

Ai

INPUT TOKEN p_{in} 

 $[0, 2)$

Token

 $[2, 8)$

omics

 $[8, 13)$

Production	,	▯Pricing	,	▯and	▯Value
▯in	▯Token	-	Met	ered	▯Intelligence

$$3 \text{ OUTPUT} \times p_{\text{out}} = \alpha p_{\text{in}}$$

Quanyan Zhu

4 TOKENS 4507 1092 272 33144

AI Tokenomics

AI Tokenomics

*Production, Pricing, and Value
in Token-Metered Intelligence*

QUANYAN ZHU

AI Tokenomics: Production, Pricing, and Value in Token-Metered Intelligence

Copyright © 2026 Quanyan Zhu. All rights reserved.

No part of this book may be reproduced in any form or by any electronic or mechanical means, including information storage and retrieval systems, without permission in writing from the author, except for brief quotations in reviews and scholarly works.

The author has made every effort to ensure the accuracy of the material in this book. Prices, products, and market facts are current as of the dates given in the text and will have changed since. Every monetary figure is either cited to a dated source or labeled hypothetical.

Version of September 22, 2026.

Typeset by the author in Libertinus Serif using $\LaTeX$. Icons in figures are from Font Awesome Free (fontawesome.com), used under the Creative Commons Attribution 4.0 License.

About This Sample

This sample contains the preface and Part I (Chapters 1–3) of *AI Tokenomics*, typeset exactly as in the complete manuscript. The contents that follow list all 21 chapters and four appendices of the complete manuscript, which currently runs to about 710 pages; page numbers in that list refer to the complete manuscript. Cross-references from the sample chapters to later chapters give the chapter and section numbers of the complete manuscript. The bibliography at the end lists only the works cited in the sample chapters.

Preface

Intelligence as infrastructure

We draw electricity from the power grid, data from wireless networks, and water from a municipal utility. None of these arrives as a product we own. Each arrives as a metered service: kilowatt-hours, gigabytes, cubic meters. Each rests on large, shared, capital-intensive infrastructure whose capacity must be built ahead of demand. And each is priced in ways that decide who uses how much, when, and for what.

Machine intelligence is becoming a service of the same kind. A question answered by a language model, a document summarized, an invoice reconciled, or a security alert triaged by a team of agents is delivered through an inference service and metered in tokens. Applications draw on that service the way appliances draw on the grid. The token is the kilowatt-hour of this new utility. It measures a flow of computation, it is billed by the million, and it links what an application consumes to the silicon, energy, and capital that produce it. As AI tools and services spread into every function of an organization, tokens are becoming part of the infrastructure for intelligence.

Infrastructure raises economic questions that the technology alone does not answer. How should a scarce and costly capacity be priced? What does efficient use look like, and how would we know that a system uses it efficiently? Who should pay for peak demand, for reliability, and for the failures that cheap service can cause? How much of what is spent creates value, and for whom? For electricity, telecommunications, and water, these questions produced a body of economics and engineering that still shapes how those systems are built, sold, and regulated. For machine intelligence, the same questions are only now being asked.

Why the economics comes next

The economics of a technology is the step that follows its development. It is also the stepping stone to its governance and regulation. The pattern is familiar. Wireless communication became a mass service only once spectrum was allocated and priced, data plans were designed, and networks were dimensioned against measured demand; the

rules that now govern it rest on that economic understanding. Cloud computing turned servers into metered services, and with them came pay-as-you-go pricing, reserved and spot capacity, and a management practice devoted to controlling the bill. Power systems went furthest: wholesale markets, peak and time-of-use pricing, capacity procurement, and regulation designed around the physics of the grid.

Machine intelligence has reached the same point. Its capabilities have been demonstrated; its deployment at scale is under way. What organizations lack is an economic understanding of the service they are adopting. An enterprise deciding whether to roll out an agent needs to know what the agent will consume, how that consumption will grow as tasks lengthen, what the service will cost at the prices it will actually pay, and whether the outcomes are worth more than the tokens. A provider needs to know how to price a service whose cost depends on hidden reasoning and whose value depends on results it does not observe. A regulator needs to know which features of the market concern competition, which concern risk, and which are simply the arithmetic of a new utility. These questions should be understood before wide adoption, not after the first surprising bill or the first costly failure.

A fragmented literature

The subject is also an important research topic, and its literature is fragmented. Computer scientists study how inference is served, batched, cached, and routed, and how agents spend tokens. Economists study pricing, contracts, market structure, and the effect of AI on productivity and labor. Engineers bring queueing, control, and network optimization. Management scholars and practitioners study total cost of ownership, financial operations, and adoption. Operations researchers contribute capacity planning, resource allocation, and mechanism design. Each community holds part of the answer, and each tends to treat the others' parts as given. A systems paper reports cost per million tokens and stops before value; an economics paper models a price and abstracts from the machine that sets its cost floor.

This book brings these strands together around a single object, the token, and a single organizing idea, the chain from *production* (how silicon, energy, and capital become tokens) through *consumption* (how applications and agents spend them) to *value* (what the spending buys), with pricing, markets, governance, and risk built around that chain. Throughout, the aim is to state models carefully enough that their assumptions are visible, to connect them to the fields in which the underlying problems were first solved, and to separate what is established from what is plausible and from what is open.

What this book contains

The book has twenty-one chapters in seven parts, followed by four appendices.

- **Part I, Foundations** (Chapters 1 to 3), defines AI tokenomics, explains for readers

without a systems background how inference works, and examines the token as a unit of account, including how a customer can verify a bill.

- **Part II, Production** (Chapters 4 and 5), develops the cost of producing tokens, the throughput–latency frontier and optimal utilization, and the procurement of capacity, hosting choices, energy, water, and power.
- **Part III, Consumption** (Chapters 6 to 8), models token demand, context, caching, and retrieval; reasoning, agents, and branching consumption; and an algebra for estimating and forecasting the cost of a workload.
- **Part IV, Value** (Chapters 9 to 12), turns from tokens to outcomes: total cost and quality floors, routing and cascades, token allocation and shadow prices in agentic workflows, and the valuation and attribution of agents.
- **Part V, Pricing and Markets** (Chapters 13 to 16), treats price lists and menus, token-based, value-based, and hybrid contracts, demand, rebound, and market structure, and markets for compute and for agents.
- **Part VI, Governance, Risk, and Society** (Chapters 17 to 19), covers internal markets, budgets, and FinOps; risk, liability, and insurance; and the political economy of tokenized machine labor.
- **Part VII, Practice and Prospect** (Chapters 20 and 21), works integrated case studies end to end and closes with a research agenda.

Three running examples connect the chapters: a high-volume customer-support assistant, an invoice-reconciliation agent, and a multi-agent security-operations workflow (Section 1.4). The second is calibrated on a real experiment of 480 metered runs, so its numbers are measured rather than assumed. Appendix A reviews probability, queueing, branching processes, heavy tails, and convex optimization; Appendix B reviews the economics the book uses; Appendix C contains laboratory exercises that run offline; and Appendix D is a glossary.

Who this book is for

The book is written for several audiences, and it is organized so that each can find a path through it.

- **Those who price AI services**, at model providers, platforms, and companies that sell AI-enabled products, will find the cost structure behind a price list, the theory of menus and screening, and the trade-offs among token-based, outcome-based, and hybrid contracts (Parts II and V).
- **Those who plan and build efficient AI systems**, including machine-learning and platform engineers and architects, will find models of serving cost, consumption,

caching, routing, and workflow allocation that turn engineering choices into dollars and quality (Parts II–IV).

- **Students and researchers** in computer science, economics, engineering, management, and operations research will find a textbook with worked examples, exercises, laboratory exercises, and a map of open problems that connects their field to the others ([Chapter 21](#)).
- **Policy makers and regulators** will find the economics of market structure, competition, liability, insurance, compute governance, and labor, stated in terms that separate structural features of the market from transient prices (Parts V and VI).
- **Practitioners in finance, procurement, and operations**, including FinOps and cloud-cost teams, will find total cost of ownership, the choice of cost denominator, chargeback and budgets, and an enterprise program that moves from visibility to optimization to value ([Chapters 9, 17 and 20](#)).
- **Managers and executives**, including chief executives deciding where and how far to deploy AI, will find the questions to ask before a rollout, the metrics that reward the right behavior, and the risks that a token bill does not show ([Chapters 1, 9, 17, 18 and 20](#)).

The mathematics is kept at the level needed to make an argument precise: elementary probability, constrained optimization, and introductory microeconomics. No background in how models are served is assumed. Readers who want a course can follow the three paths in [Table 1.7](#), for engineering and systems, economics and management, and a research seminar. Policy makers may read [Chapters 1, 3, 9, 15, 16, 18, 19 and 21](#). Executives who want the argument without the proofs may read [Chapters 1, 9, 13, 17 and 20](#), skipping propositions and returning to them when a result matters to a decision.

Using this book

Every chapter opens with learning objectives and closes with key takeaways, notes on the literature, and exercises. Worked examples are fully computed, and every number in them is reproduced by a script that accompanies the book; the laboratory exercises of [Appendix C](#) run offline on the same data, so no account with any provider is needed.

Prices, products, and market facts change quickly. The book therefore keeps them in dated *In Practice* boxes, each marked with the month in which it was checked, and every other monetary figure is either cited or labeled hypothetical. The structural results do not depend on any price list: that utilization enters unit cost hyperbolically, that agentic consumption can diverge, that replayed context grows quadratically, and that token savings, cost savings, and value are three different quantities.

Relationship to other work

This textbook is the teaching treatment of a framework whose research form appears in the author’s monograph (Zhu, 2026b) and paper (Zhu, 2026d). The book adds worked examples, exercises, laboratory material, and chapters on pricing, contracts, markets, governance, and risk, and it draws on the author’s related work where it is relevant, cited in the third person. Several frameworks from practice recur, among them the production–consumption–value decomposition, the Five-Layer Tokenomics Stack, and Big-T notation (Tokenomics Foundation, 2026b,e; Tokenomics Foundation and Neff, 2026). They are cited as practitioner constructs and kept distinct from the models developed here.

A note on terminology: the word *tokenomics* was used first in cryptocurrency, for the supply schedules and incentives of digital assets. That usage is unrelated to the subject of this book beyond the shared word.

Quanyan Zhu
New York University
September 2026

Contents of the Complete Manuscript

I FOUNDATIONS: TOKENS AS AN ECONOMIC RESOURCE

1	The Economic Turn in Machine Intelligence	3
2	How Inference Works: A Primer for Economists and Managers	29
3	The Token as a Unit of Account	53

II PRODUCTION: THE SUPPLY OF INFERENCE

4	The Cost of Producing Tokens	81
5	Capacity, Hosting, and Energy	107

III CONSUMPTION: THE DEMAND FOR TOKENS

6	Token Demand: Context, Caching, and Retrieval	139
7	Reasoning, Agents, and Branching Consumption	165
8	Estimating Workload Cost: From Big-T to an Expected-Cost Algebra	193

IV VALUE: FROM TOKENS TO OUTCOMES

9	Total Cost, Outcomes, and Value	223
10	Routing, Cascades, and the Deployment Problem	247
11	Token Allocation in Agentic Workflows	273
12	Valuing and Attributing Agents	301

V PRICING AND MARKETS

13	Pricing Token-Metered Services	329
14	Token-Based, Value-Based, and Hybrid Contracts	357
15	Demand, Rebound, and Market Structure	387
16	Token Markets and Agent Economies	411

VI GOVERNANCE, RISK, AND SOCIETY

17	Internal Markets, Budgets, and FinOps	441
18	Risk, Liability, and Insurance	471
19	Agentic Capital and the Political Economy of Tokens	501

VII PRACTICE AND PROSPECT

20	Case Studies in AI Tokenomics	529
21	Open Problems and a Research Agenda	553

APPENDICES

A	Probability, Queueing, and Optimization	573
B	Economics Primer	595
C	Laboratory Exercises	615
D	Glossary	635
	Bibliography	651
	Index	687

PART I

Foundations: Tokens as an Economic Resource

*What a token is, how it is produced by inference hardware, and
why it has become the unit through which machine intelligence is
metered, priced, and governed.*

The Economic Turn in Machine Intelligence

Foundation models moved from demonstration to production, and a production system has a unit-economics problem. Once a model is deployed, its cost is dominated by inference. That cost is metered in tokens and billed by usage, and it is set by design decisions that are made long before anyone reads an invoice. This chapter defines AI tokenomics as the study of how tokens are produced, consumed, priced, valued, allocated, and governed. It separates the subject from cloud cost accounting and FinOps, and from the unrelated tokenomics of cryptocurrency. It then introduces the production–consumption–value chain that organizes the book and places pricing, markets, and governance around it. Three running examples are fixed with their parameters: a high-volume support assistant, an invoice-reconciliation agent calibrated on 480 real runs, and a multi-agent security-operations workflow. The chapter closes with three preliminary claims, each illustrated with a computed example and proved later in the book: token savings are not automatically cost savings, cost savings are not value, and agentic consumption can diverge.

LEARNING OBJECTIVES

- Explain why inference rather than training dominates the cost of a deployed model, and what usage-based pricing changes for the buyer.
- Define AI tokenomics and distinguish it from cryptocurrency tokenomics, cloud cost accounting, and FinOps.
- Explain why token expenditure and economic value are distinct quantities, and compute the break-even quality tolerance of a cost-saving change.
- Describe the production–consumption–value chain, locate a given decision on it, and name the chapters that analyze it.
- Describe the three running examples, say which of their parameters are measured and which are hypothetical, and state the economic questions each raises.

- Plan a route through the book for an engineering course, an economics or management course, or a research seminar.

A support organization runs a four-week pilot of an AI assistant on a tenth of its traffic. The pilot resolves 78% of the conversations it handles without a human, the model bill is \$2,880 a month, and the steering committee approves a full launch. Three months later the finance team circulates two numbers. The monthly model bill is now \$28,800, ten times the pilot figure, and \$37,800 once the retrieval, evaluation, and observability services around the model are counted. The resolution rate reported to leadership is still 78%. The chief financial officer asks the obvious question: the bill grew tenfold and the metric did not move, so what did the money buy?

Both numbers are correct, and neither answers the question. The bill grew tenfold because the traffic did; the design did not change. The resolution rate did not move because it is a ratio, and a ratio that holds steady as volume grows is a sign that the pilot was representative, not a sign of failure. Whether the launch was worth \$37,800 a month depends on something that appears in neither report: what the 50,000 daily conversations would have cost without the assistant. If a human handoff costs \$4, the assistant covers its full daily cost of \$1,260 once it avoids 315 handoffs a day, which is 0.63% of the conversations it sees. That is a low bar, and the assistant probably clears it by a wide margin. But nobody in the meeting can say by how much, because the organization measured tokens and a ratio, not the counterfactual.

This chapter is about the gap between those two kinds of number. [Section 1.1](#) explains why the gap opened when models moved into production. [Section 1.2](#) defines the subject that studies it, and [Section 1.3](#) introduces the chain of production, consumption, and value that organizes the book. [Section 1.4](#) fixes three running examples, one of which is the assistant above. [Section 1.5](#) states three claims that the rest of the book proves, and [Section 1.6](#) explains how to read it. All figures in the opening story are hypothetical and follow running example R1.

1.1 From Capability to Deployability

For most of its history, machine learning measured progress with one kind of question: can a model perform a task, and how well? Benchmarks, held-out test sets, and leaderboards answer that question. They do not answer whether a system built around the model should be deployed. A production deployment must meet several conditions at once, and capability is only the first. The system must produce acceptable outputs on the real task distribution, not on a benchmark proxy. It must respond within the latency the product requires. It must fail in ways the organization can detect, bound, and recover from. Its consumption must be attributable to an owner. And the value of what it produces must exceed its full cost, which includes much that never appears on an inference invoice.

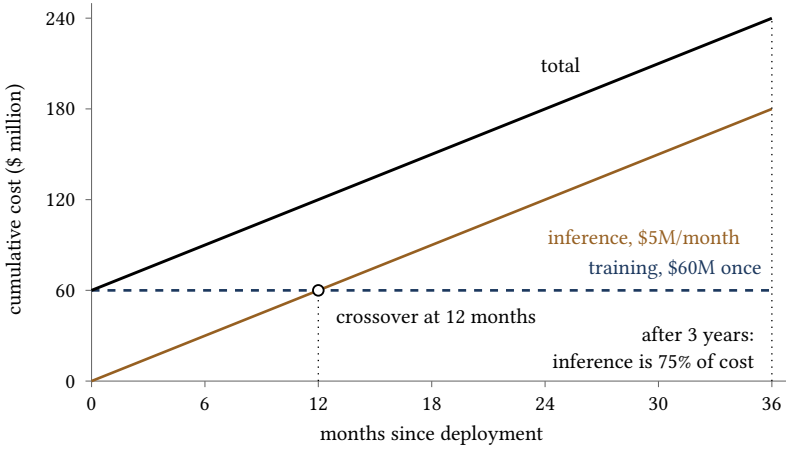


FIGURE 1.1. Cumulative cost of a deployed model with a one-time training cost and a constant inference cost per month (all figures hypothetical). Training cost is fixed; inference cost grows with use, so its share of lifetime cost rises toward one.

This book is about the last condition. Every industrial technology eventually faces it. What is distinctive about generative AI is that its cost structure fits badly with the accounting habits organizations bring to it. The rest of this section explains why.

1.1.1 *Why Inference, Not Training, Dominates Deployed Cost*

Public discussion of AI costs has focused on training, which is large, visible, and easy to report: a frontier model is trained once, at a cost that can be stated as a single figure. To the organization that deploys the model, however, training is a fixed cost incurred in the past, often by someone else. What it pays every day is inference, the computation that turns each prompt into a response. Training cost is paid once; inference cost grows with every user, every request, and every model call.

The asymmetry follows from arithmetic. Let a model cost F to train and c per period to serve at a given volume. Cumulative cost after t periods is $F + ct$, and the inference share of it is $ct/(F + ct)$, which rises toward one as the deployment runs. Figure 1.1 uses hypothetical round numbers: a one-time training cost of \$60 million and inference spending of \$5 million a month. Inference spending equals training cost after 12 months and is 75% of cumulative cost after three years. A deployment that grows, or that moves to agentic workloads with many model calls per task, reaches the crossover sooner.

Measurements are consistent with the arithmetic. Patterson et al. (2022) report that at Google, in one week of April in each of 2019, 2020, and 2021, about three-fifths of the energy used for machine learning went to inference and two-fifths to training, a split they attribute to the many services with billions of users that rely on machine learning. The fact has design consequences too. Sardana et al. (2024) show that once expected inference demand is added to the scaling-law objective, the compute-optimal model for

a heavily used deployment is smaller and trained on more data than the training-only optimum. How much a model will be used is thus a design input to the model itself.

A second contrast matters as much as the first. Conventional software is an information good, with high fixed cost and nearly zero marginal cost (Shapiro and Varian, 1998): the millionth copy of a spreadsheet program costs its vendor almost nothing. A model-based service has a strictly positive marginal cost. Every answer consumes accelerator time, memory, and energy, and a more elaborate answer consumes more of each. Software economics has long assumed that scale drives unit cost toward zero, and that assumption does not hold here. Chapter 2 explains the mechanics of this marginal cost, and Chapter 4 derives it from the hardware.

✂ IN PRACTICE (AS OF SEPTEMBER 2026)

- *Token volumes.* At Google I/O on 19 May 2026, Sundar Pichai reported that Google processed more than 3.2 quadrillion tokens a month across its surfaces, against roughly 480 trillion a year earlier and 9.7 trillion two years earlier (Pichai, 2026).
- *Enterprise spending.* Menlo Ventures estimates that companies spent \$37 billion on generative AI in 2025, up from \$11.5 billion in 2024; of this, \$18 billion went to infrastructure, including \$12.5 billion to foundation-model APIs (Tully et al., 2025). The estimate comes from a venture investor's survey and market model, not from audited accounts.
- *Budgets.* Deloitte reports that AI is the fastest-growing expense in corporate technology budgets, that some firms say it consumes up to half of their IT spend, and that cloud bills rose 19% in 2025 for many enterprises (Merizzi et al., 2026b,a).

1.1.2 Usage-Based Pricing and Its Consequences

Most model access is sold by the token. A provider posts a price p_{in} per input token and a higher price p_{out} per output token, and a period's model charge is

$$C_m = p_{\text{in}}N_{\text{in}} + p_{\text{out}}N_{\text{out}}, \quad (1.1)$$

where N_{in} and N_{out} are the input and output tokens metered in the period. A token is a fragment of text, typically a word or part of a word, defined by the model's tokenizer (Chapter 3). Token volume in turn is a product of design and demand: users U , requests per user r , model calls per request k , and tokens per call. Chapter 6 develops this identity. The point here is that most factors of the product are chosen by engineers.

Usage-based pricing is a reasonable way to sell a service whose marginal cost is positive, but it changes who bears which risk. Under a seat license, the vendor bears the risk that a customer uses the product heavily, and the customer's bill is known in advance. Under token pricing, four risks move to the customer.

1. *Volume risk*. Spending scales with adoption. A successful launch, as in the opening story, multiplies the bill.
2. *Architectural risk*. Spending scales with design choices: the number of calls per request, the context replayed on each call, whether retries are bounded, and whether agents may delegate to other agents. The same task can differ in cost by an order of magnitude between two designs.
3. *Behavioral risk*. Reasoning models and agents decide for themselves how many tokens to spend, so a prompt edit or model update can change consumption without any change in demand ([Chapter 7](#)).
4. *Definition risk*. The unit itself can change. A new tokenizer can turn the same text into more tokens at an unchanged price per token, and reasoning tokens that the customer never sees are billed as output ([Chapter 3](#)).

Two further consequences follow. Cost becomes visible at the level of a single call, so every engineer can affect it and none owns it. And the metered quantity becomes an obvious target, which invites the familiar failure of metrics that become goals: teams cut tokens in ways that cut quality, as [Section 1.5](#) illustrates. [Chapter 13](#) examines how posted prices, discounts, subscriptions, and committed-spend contracts shape these risks in practice, and [Chapter 14](#) asks when a customer should pay for tokens at all rather than for outcomes.

IN PRACTICE (AS OF SEPTEMBER 2026)

Unit prices for a fixed level of capability have fallen steeply. Epoch AI estimates that, over the three years to March 2025, the price of reaching a given benchmark performance fell by between 9 and 900 times per year depending on the threshold, with the fastest declines in the most recent year ([Cottier et al., 2025](#)). The 2025 AI Index reports that the inference cost of a system performing at the level of GPT-3.5 fell more than 280-fold between November 2022 and October 2024 ([Maslej et al., 2025](#)). Using OpenRouter data, [Demirer et al. \(2026\)](#) find that the price of intelligence has fallen roughly a thousandfold and that open-source models cost about 90% less than comparable closed models. Over the same period, the total spending figures in the previous box rose. Falling unit prices and rising bills are consistent: [Chapter 15](#) develops the demand response that connects them.

1.1.3 *The Gap between Token Accounting and Economic Value*

An organization that has instrumented its token consumption can say how many tokens were used, by which application, on which model, and at what charge. That is necessary. It is not yet economics, for three reasons.

Consumption is determined by architecture and is largely invisible. The quantity of inference work a system performs is a property of its control flow: how many model calls a user-visible request triggers, how much history each call replays, and how delegation and retries are bounded. These decisions are usually made early, by engineers who have no cost information at that moment, and their effects are hidden inside a single line of the invoice.

The metered unit is not the economic unit. Tokens correlate with computational work, which makes them a legitimate billing unit. But they are not a unit of computation, energy, or value, and because each model has its own tokenizer, they are not even comparable across providers (Chapter 3). A practice that optimizes the metered quantity optimizes a proxy whose relation to the quantity of interest is unstable.

Cost and quality are coupled, and the coupling is asymmetric. Most available cost reductions, such as smaller models, shorter contexts, aggressive caching, and fewer verification passes, are also potential quality reductions. The saving is immediate, measurable, and credited to the team that made the change. The quality loss is delayed, diffuse, and borne by someone else: human reviewers, escalation queues, customers. A program that measures only the saving will systematically overstate its success. Chapter 9 prices that loss, and Section 1.5 shows a case in which it reverses the sign of a redesign.

KEY IDEA

Inference makes machine intelligence a good with positive marginal cost, and usage-based pricing moves the risk of that cost to the buyer. What the buyer pays is determined by design decisions, while what it gains is determined by outcomes, and a token count measures neither.

1.2 What AI Tokenomics Is

The term *AI tokenomics* has come into use among practitioners and researchers for the body of practice and analysis concerned with the economics of token-metered AI. This section states the working definition the book uses, sets it beside definitions in circulation, and separates the subject from two neighbors with which it is often confused.

1.2.1 Definition and Scope

DEFINITION 1.1 (AI tokenomics). *AI tokenomics* is the study of how tokens, the units in which the work of machine-learning models is metered, are produced, consumed, priced, valued, allocated, and governed, and of the decisions that change the relationship between what token-metered systems cost and what they are worth.

The definition is a working one. It names seven verbs because each identifies a distinct economic mechanism with its own models and instruments, and each is the

subject of part of the book. It names the token because the token is the unit through which those mechanisms interact, not because the token is the object of optimization. The object of optimization is the relationship between cost and worth, which the token merely meters.

The definition builds on two earlier statements by the author. The research paper of [Zhu \(2026d\)](#) describes AI tokenomics as the study of the generation, consumption, pricing, valuation, allocation, optimization, and governance of tokens within AI ecosystems. It treats tokens as economic resources that mediate between computational effort, workflow performance, organizational value, and market structure. The monograph of [Zhu \(2026b\)](#) organizes the economics of token-based systems around production, consumption, and value. This book adapts the models of both works and rewrites them for teaching.

Practitioner definitions are broader in purpose and narrower in method. The Tokenomics Foundation, in a definition it labels a working draft, describes AI tokenomics as the discipline of turning energy and capital into AI capability, consuming that capability efficiently across an organization, and realizing measurable business value ([Tokenomics Foundation, 2026b,f](#)). Deloitte’s report on the “pivot to tokenomics” treats tokens as the unit in which enterprises must now forecast, monitor, and govern AI spending ([Merizzi et al., 2026a](#)). These are reported practitioner constructs. The book cites them where it uses them and keeps them separate from its own models, which are stated as definitions and propositions and proved.

The word has other academic uses. [Salim et al. \(2026\)](#) use *tokenomics* for the measurement of where tokens are consumed in a multi-agent software-engineering system, finding that code review, not code generation, accounts for most of the tokens. [Chen et al. \(2026\)](#) survey *token economics* for LLM agents, treating tokens as production factors, exchange media, and accounting units. The book’s usage is compatible with both and wider than either.

Levels of analysis. The relevant unit of analysis changes with scale, and the book moves between four levels ([Zhu, 2026d](#)). At the *task* level, the question is how difficulty, context, and output requirements generate token demand and quality. At the *workflow* level, it is how tokens should be allocated across interdependent stages whose qualities complement or substitute for one another. At the *enterprise* level, it is how budgets, internal prices, and controls should govern consumption among teams with private information. At the *market* level, it is how prices, contracts, and allocation mechanisms shape access to machine intelligence and the distribution of its gains.

Three kinds of content. The book mixes material of three kinds and keeps them distinguishable. *Reported constructs* come from practitioner sources and are cited at the point of use. *Established results* from queueing theory, branching processes, contract theory, index numbers, and cooperative games are stated with citations and used as tools. *Constructions of the book*, meaning its models, propositions, and worked analyses, are attributed to no practitioner organization. Every monetary figure is either cited or labeled hypothetical. Facts that change quickly, such as posted prices and product

TABLE 1.1. Cloud cost accounting, FinOps for AI, and AI tokenomics compared. The three are complements: each supplies information or authority that the others lack.

		Cloud cost accounting	FinOps for AI	AI tokenomics
Object		Charges and cost centers	Realized spend against plan	Conditions that generate cost and value
Typical question		What was spent, and where is it booked?	Why did spend differ from plan, and who owns the variance?	Which design, product, or contract decision changes cost per unit of value?
Timing		After the period	During and after the period	Before and during design
Unit		Dollars by account	Dollars and tokens by owner	Cost per accepted outcome, value net of risk
Instruments		Ledgers, allocation keys	Tagging, showback, budgets, anomaly alerts, commitments	Cost and demand models, routing, caching, allocation, pricing and contract design
Failure if used alone		Accurate records of an unexplained bill	Finds a problem without owning the technical fix	Improves a design that nobody sustains

features, appear only in dated *In Practice* boxes, and no structural result depends on them.

1.2.2 *Tokenomics versus FinOps*

Cloud financial operations, or *FinOps*, is an established practice for managing variable cloud spending. It assigns costs to owners, forecasts and budgets, detects anomalies, manages commitments, and runs an operating cadence that reviews variances with the teams responsible (Storment and Fuller, 2023; *FinOps Foundation*, 2024). Extending FinOps to AI spending is natural and is under way. Beneath both lies cloud cost accounting, the recording and allocation of charges to cost centers.

The three are often presented as competing labels for one activity. They are better understood as answering different questions with different instruments (Table 1.1). Cost accounting records what was spent. FinOps is a *control* practice that operates on realized spend and asks what happened against plan and who owns the variance. Tokenomics is a *design* practice that operates on the conditions that generate spend and value and asks which technical, product, or contractual decision changes the economics. The Tokenomics Foundation reaches a similar conclusion: FinOps for AI brings depth in allocation, forecasting, and accountability, tokenomics brings breadth across the chain from capacity to value, and neither is a subset of the other (*Tokenomics Foundation*, 2026a).

The division matters for the opening story. FinOps could have flagged the tenfold increase in the bill, attributed it to the support assistant, and confirmed that it matched the forecast launch volume. It could not have told the committee whether the launch

created value, whether a cheaper route would have met the same quality, or how to redesign the assistant. Those are tokenomics questions. [Chapter 17](#) returns to the division of labor inside an organization, including the practice of “shifting left,” which means putting cost information in front of the engineers who make design decisions.

KEY IDEA

FinOps controls realized spending; tokenomics designs the conditions that generate spending and value. An organization with only the first can identify a problem it cannot fix, and one with only the second can make improvements that nobody sustains.

1.2.3 A Note on Cryptocurrency Tokenomics

The word *tokenomics* originated in cryptocurrency, where it denotes the design of token supply schedules, issuance and burning, staking incentives, distribution, and on-chain governance for digital assets whose scarcity is enforced by a consensus protocol. That is a different subject. The tokens of this book are units of text segmentation used to meter computation. They are not transferable assets, they are not owned, and their supply is not governed by a protocol. The overlap is lexical, and it causes confusion in literature searches and in interdisciplinary conversation. Where the book discusses tokenized or decentralized markets for compute ([Chapter 16](#)), it says so explicitly. Otherwise, *token* always means a metered unit of model input or output.

1.3 The Production–Consumption–Value Chain

The economics of a token-metered system can be organized around three domains, each governed by a different mechanism and admitting different instruments. The decomposition follows the Tokenomics Foundation’s framing ([Tokenomics Foundation, 2026d](#)) and the monograph of [Zhu \(2026b\)](#). The book develops each domain with formal models and adds the institutions that act on all three.

DEFINITION 1.2 (Production–consumption–value chain). *Production* converts capital, energy, and hardware into delivered tokens at some unit cost and latency. *Consumption* converts tasks into tokens through the architecture of an application or agent. *Value* converts the resulting outputs into accepted outcomes and their worth, net of the cost of failures. *Pricing, markets, and governance* are the institutions that transmit costs, allocate capacity, and control exposure across the three domains.

[Figure 1.2](#) maps the chain onto the book, and [Table 1.2](#) lists the question, governing mechanism, and representative measure of each domain.

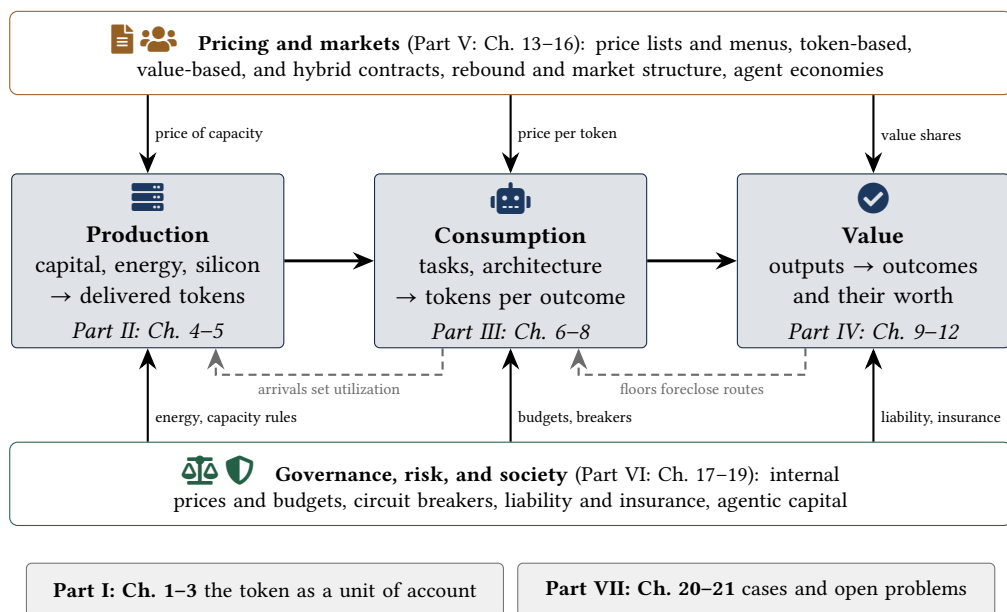


FIGURE 1.2. The production–consumption–value chain as the organizing map of the book. Solid arrows between the domains carry the forward flow; the dashed connectors beneath them carry couplings that run the other way, which is why optimizing one domain in isolation is unreliable. Pricing and markets transmit costs and incentives into all three domains, and governance controls exposure in all three. Part I supplies the unit of account, and Part VII applies the whole chain to cases.

1.3.1 Production: Turning Energy and Silicon into Tokens

Production begins with an hourly cost: accelerators amortized over their useful life, power and cooling, floor space, networking, and staff. Hardware sets how many tokens that hour can deliver, and scheduling software determines how many it does deliver. The key relation is hyperbolic. If a pool of hardware costs C_f per day, can deliver $N_{\max}$ tokens a day, and runs at utilization u , the capacity cost of a delivered token is $C_f/(uN_{\max})$ plus any variable cost. Halving utilization doubles unit cost. In the dedicated pool used with R1 (all figures hypothetical), $C_f = \$2,400$ a day and $N_{\max} = 1.2 \times 10^9$ tokens, so a token costs \$8.00 per million at $u = 0.25$ and \$2.67 per million at $u = 0.75$.

It does not follow that the pool should run full. Requests arrive at random, and as utilization approaches one, queues and delay grow without bound. Once delay has a price, the cost-minimizing utilization is strictly below one ([Theorem 4.12](#)). Latency targets also turn raw throughput into *goodput*, the tokens delivered within their service-level objectives. [Chapter 4](#) builds these results from the hardware up. [Chapter 5](#) turns to procurement, meaning whether to buy tokens through an API, rent accelerators, or own infrastructure under uncertain demand, and to energy as both a cost and an externality.

TABLE 1.2. The three domains of the chain: the question each poses, the mechanism that governs it, a representative measure, and the chapters that model it.

Domain	Question	Governing mechanism	Representative measure	Chapters
Production	How efficiently do capital and energy become delivered tokens?	Fixed cost spread over utilization; throughput-latency frontier; delay cost	Unit cost of a delivered token $\bar{c}(u)$	4, 5
Consumption	How many tokens does an outcome require?	Call multiplicity, context replay, caching, reasoning, delegation	Tokens per request; mean branching factor m	6–8
Value	What outcome results, and what is it worth?	Quality floors, fallbacks, routing, allocation, attribution	Cost per accepted outcome; net value W	9–12

1.3.2 Consumption: How Applications and Agents Spend Tokens

Consumption is governed by an accounting identity. Daily tokens equal requests, times model calls per request, times tokens per call. R1’s baseline design serves 50,000 requests a day with four calls of 1,500 tokens each, which is $50,000 \times 4 \times 1,500 = 3 \times 10^8$ tokens. Every factor is a design choice. Conversation history replayed on each call makes context a recurring cost, not a one-time one. Prompt caching, reuse of attention state, and semantic caching remove tokens in different ways and with different error profiles. Retrieval substitutes a few relevant passages for a long context ([Chapter 6](#)).

Reasoning models and agents make consumption endogenous, because the model chooses how much to spend. Two results of [Chapter 7](#) set the scale. When each round of an agent loop replays the history, a loop with K calls consumes on the order of K^2 tokens ([Theorem 7.6](#)). When agents delegate to sub-agents, expected consumption is finite exactly when the mean number of sub-agents each invocation spawns, the branching factor m , is below one ([Theorem 7.13](#)). [Chapter 8](#) turns these growth laws into an algebra of expected cost for estimating and budgeting workloads.

1.3.3 Value: From Tokens to Outcomes

The value domain changes the denominator. Cost per token is a statement about procurement; cost per request is a statement about an application; neither says whether the request was answered correctly or what the answer was worth. [Chapter 9](#) builds a total-cost-of-ownership decomposition and a ladder of denominators that ends at the economic cost per resolved case, where every failure is charged at the cost of the fallback that completes it. It shows that a quality floor is a price of failure that has not been written down ([Proposition 9.4](#)). It closes with the design of experiments that identify value and with pilots as real options.

The remaining chapters of Part IV optimize against that denominator. [Chapter 10](#)

sends each request to the cheapest route that meets its quality floor, through routing and cascades. [Chapter 11](#) allocates tokens across the stages of a workflow and interprets the multipliers as shadow prices. [Chapter 12](#) asks how the surplus of a human–AI team should be attributed to its members.

1.3.4 Pricing, Markets, and Governance around the Chain

Three institutions act on the whole chain. *Pricing* transmits production cost to consumers and value back to producers. [Chapter 13](#) reads price lists, discounts, and menus of subscriptions and committed spend. [Chapter 14](#) treats the choice between paying for tokens and paying for outcomes as a contract design problem. It shows that per-token markups reward token-hungry configurations ([Theorem 14.12](#)) and derives the optimal hybrid tariff together with a test of whether a token charge belongs in it ([Theorem 14.19](#)). *Markets* aggregate these decisions. Falling prices can raise total spending ([Chapter 15](#)), and agents that buy compute and services from other agents create markets of their own ([Chapter 16](#)). *Governance* controls exposure inside and beyond the firm. Internal prices, budgets, and circuit breakers allocate shared capacity ([Chapter 17](#)); liability rules and insurance handle losses that no token refund repairs ([Chapter 18](#)); and the substitution of token costs for salaries raises questions of distribution and policy ([Chapter 19](#)).

The domains are coupled in both directions, and most analytical errors occur at the couplings. A production choice made for unit cost, such as a longer batching window, raises latency, which constrains the application’s viable architecture. A consumption choice, such as aggressive prefix caching, changes the arrival process the serving layer sees and therefore the utilization at which capacity is priced. A value choice, such as a higher quality floor for a regulated workflow, rules out routes that the consumption layer would otherwise use. The three domains are also usually owned by different functions (infrastructure, product engineering, and the business unit), with different budgets and reporting cycles. The economically correct decision often requires a trade across an organizational boundary that no single owner is empowered to make.

KEY IDEA

Production, consumption, and value are governed by different mechanisms and owned by different functions, but they are coupled in both directions. An improvement measured within one domain can be a loss for the chain, so every claimed saving must be checked against the domains it does not measure.

1.4 Running Examples

Three running examples recur throughout the book ([Table 1.3](#)). They are chosen to stress different parts of the chain, and they differ in their evidentiary status. R1 is a hypothetical case with round parameters, designed to make arithmetic transparent. R2

TABLE 1.3. The three running examples at a glance.

	R1: support assistant	R2: invoice agent	R3: security operations
Status of data	Hypothetical	Measured (480 runs)	Hypothetical scenario
Unit of work	Customer request	Reconciled invoice	Security alert
Scale	50,000 requests/day	40 tasks, 2 replicates	1,000 alerts/day
Stresses	Context, caching, utilization, quality floors	Hidden reasoning, model choice, contracts	Workflow reliability, delegation, attribution, insurance
Main chapters	2, 3, 4, 6, 9, 10, 20	3, 7, 8, 9, 14, 18	4, 7, 9, 11, 12, 18

TABLE 1.4. Parameters of running example R1 (all hypothetical). The redesign cuts calls, trims prompts, and adds a semantic cache.

Symbol	Meaning	Baseline	Redesign
Q	Requests per day	50,000	50,000
k	Model calls per request	4	1
h	Cache hit rate (hits need no model call)	0	0.25
	Input tokens per call	1,200	600
	Output tokens per call	300	200
$p_{\text{in}}, p_{\text{out}}$	List prices (\$ per million tokens)	2, 8	2, 8
	Adjacent cost per day (platform, retrieval, evaluation, observability)	\$300	\$420
s	First-contact resolution rate	0.78	0.76
c_H	Cost per escalation to a human agent	\$4	\$4

Dedicated-pool variant (Chapter 4): fixed cost $C_f = \$2,400$ per day, capacity $N_{\text{max}} = 1.2 \times 10^9$ tokens per day, variable cost $c_v = 0$.

is calibrated on a real experiment whose data are used as measured. R3 is a hypothetical scenario whose parameters follow a published case study. The parameters below are fixed for the whole book; a chapter that needs more states the additional values and labels them hypothetical.

1.4.1 R1: A Customer-Support Assistant

R1 is a high-volume assistant that answers customer-support requests and escalates what it cannot resolve to human agents. It is the book’s laboratory for context and caching, for the cost of producing tokens, and for pricing quality. All of its parameters are hypothetical and are listed in Table 1.4. The list prices of \$2 per million input tokens and \$8 per million output tokens are round numbers chosen for arithmetic, not quotations of any provider. The case follows the support-assistant analysis of Zhu (2026b).

RUNNING EXAMPLE R1: THE DAILY BILL, BEFORE AND AFTER

Step 1: baseline tokens. The baseline makes $50,000 \times 4 = 200,000$ calls a day, with $200,000 \times 1,200 = 240$ million input tokens and $200,000 \times 300 = 60$ million output tokens, 300 million in all.

Step 2: baseline cost. By Eq. (1.1), $C_m = 240 \times \$2 + 60 \times \$8 = \$960$ a day. With \$300 of adjacent cost, total cost is \$1,260 a day. The metered charge is \$3.20 per million tokens.

Step 3: redesign tokens. A quarter of requests are answered from the cache, so 37,500 reach the model, each with one call. Input tokens fall to $37,500 \times 600 = 22.5$ million and output tokens to $37,500 \times 200 = 7.5$ million, 30 million in all, a 90% reduction.

Step 4: redesign cost. $C_m = 22.5 \times \$2 + 7.5 \times \$8 = \$105$ a day, 89.1% below the baseline. Adjacent cost rises to \$420 for the cache and its evaluation harness, so total cost is \$525 a day, 58.3% lower, a saving of \$735 a day.

Step 5: outcomes. The baseline resolves $0.78 \times 50,000 = 39,000$ requests and escalates 11,000; the redesign resolves 38,000 and escalates 12,000. Section 1.5 prices the difference.

The pilot in the opening story is the baseline design on a tenth of the traffic: 5,000 requests, 30 million tokens, and \$96 of model charges a day, or \$2,880 over a 30-day month. At full traffic the same design costs \$960 a day, \$28,800 a month, and \$37,800 with adjacent costs.

Questions later chapters answer. What does one R1 request compute, and where does its time and memory go (Chapter 2)? Why can a 90% token saving lose money, and what do token counts miss (Chapter 3)? What does a delivered token cost on a dedicated pool, and why does the pool run below capacity (Chapter 4)? When is the pool cheaper than metered access (Chapter 5)? How do calls, context, and cache hit rates combine in token demand (Chapter 6)? What is the full cost of ownership, and how large a quality regression can a saving afford (Chapter 9)? Which routes should serve which requests (Chapter 10)? Chapter 20 assembles the answers.

1.4.2 R2: An Invoice-Reconciliation Agent

R2 is an agent that reconciles vendor invoices. It applies tiered discounts, currency conversion, proration, a credit note, tax, and a late fee in the right order to reach an exact amount payable. Unlike the other examples, R2 rests on a real experiment (Zhu, 2026c). Forty synthetic reconciliation tasks with known answers were each run twice under three prompting strategies on two models, for 480 runs in all. Under the *direct* strategy the agent answers in one call at low reasoning effort. Under *reasoned* it answers in one call at medium effort, with instructions to reason step by step. Under *verified*, a reasoned call is followed by an independent reviewing call. A response counts as correct if it is within one cent of the truth. The provider metered input, output, and reasoning

TABLE 1.5. Running example R2: measured results of the 480-run invoice-reconciliation experiment (Zhu, 2026c), costed at the calibration prices. Tokens are mean billed tokens per run (input plus output); reasoning tokens are part of output. Each cell has 80 runs.

Model	Strategy	Accuracy	Input	Output (reas.)	Tokens	\$/run	\$/correct
gpt-5.4	direct	0.512	397	796 (784)	1,193	0.0129	0.0252
gpt-5.4	reasoned	0.550	399	2,018 (1,387)	2,417	0.0313	0.0569
gpt-5.4	verified	0.575	1,456	3,696 (2,517)	5,152	0.0591	0.1028
gpt-5.4-mini	direct	0.087	397	897 (884)	1,294	0.0043	0.0495
gpt-5.4-mini	reasoned	0.287	399	2,673 (2,251)	3,072	0.0123	0.0429
gpt-5.4-mini	verified	0.312	1,223	4,579 (3,848)	5,802	0.0215	0.0689

tokens for every run; reasoning tokens are billed as output tokens and counted within them, and no cached tokens occurred.

Costs use the *calibration prices* of the experiment: \$2.50 and \$15.00 per million input and output tokens for gpt-5.4, and \$0.75 and \$4.50 for gpt-5.4-mini. They were posted prices when the experiment was run, and they are used here as fixed parameters of the case, not as current market prices. Table 1.5 reports the six cells.

RUNNING EXAMPLE R2: THREE FACTS FROM THE DATA

Hidden reasoning is most of the bill. Across all 480 runs, reasoning tokens are 61.6% of billed tokens and 71.9% of billed cost. The customer never sees the tokens that make up almost three-quarters of what it pays for.

Capability saves tokens. Under every strategy, the larger model uses fewer tokens than the mini model (1,193 against 1,294 direct, 2,417 against 3,072 reasoned, 5,152 against 5,802 verified) and is far more accurate. The mini model is cheaper per run but not per correct answer. Its direct strategy costs \$0.0043 a run, yet \$0.0495 per correct answer, 1.96 times the \$0.0252 of the larger model's direct strategy.

Elaboration is expensive. For gpt-5.4, moving from direct to verified multiplies tokens by 4.32 and cost per run by 4.57, and raises accuracy by 6.2 percentage points.

The data are small and synthetic: forty tasks from one generator, two models from one provider, and one set of prompts. They show how token counts, costs, and correctness co-vary in a real agent, not how any model performs in general. Later chapters add a few hypothetical constants to the case: a value of \$400 per correct reconciliation, cost wedges for the three strategies, and a provider margin (Chapter 14), as well as a cost of correcting a wrong answer (Chapter 9).

Questions later chapters answer. Where does the bill go, and how could a customer audit reasoning tokens it cannot see (Chapter 3)? Why can the more capable model be the cheaper one, and how should reasoning effort be chosen (Chapter 7)? Can the cost per

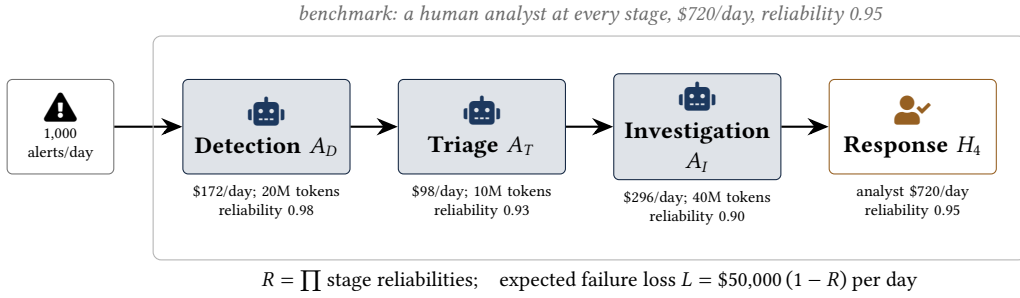


FIGURE 1.3. Running example R3. Four sequential stages process 1,000 alerts a day. Agents replace the analysts at the first three stages; a human analyst keeps the response stage. Agent costs are a subscription plus tokens at a blended \$5 per million. All parameters are hypothetical and follow Zhu (2026a).

TABLE 1.6. Parameters of running example R3 (hypothetical scenario after Zhu, 2026a).

Worker	Subscription	Tokens/day	Cost/day	Reliability
Human analyst (any stage)	\$120k salary $\times$ 1.5 overhead / 250 days		\$720	0.95
Detection agent A_D	\$72	20M	\$172	0.98
Triage agent A_T	\$48	10M	\$98	0.93
Investigation agent A_I	\$96	40M	\$296	0.90
Blended token price \$5 per million; value \$15 per processed alert; failure loss \$50,000 $(1 - R)$.				

invoice be predicted from a description of the workflow (Chapter 8)? What is the cost per correct answer once errors must be fixed (Chapter 9)? Should the provider charge per token, per correct invoice, or both, and what does the data say about the optimal token price (Chapter 14)? When does an accuracy level rule out giving the agent authority to pay (Chapter 18)?

1.4.3 R3: A Multi-Agent Security-Operations Workflow

R3 is a security operations center (SOC) that processes 1,000 alerts a day through four sequential stages: detection, triage, investigation, and response recommendation (Fig. 1.3). The benchmark staffs each stage with a human analyst. The deployment replaces the first three stages with AI agents and keeps a human at the fourth. All values are hypothetical scenario parameters that follow the case study of Zhu (2026a) and are listed in Table 1.6. The workflow succeeds on a day only if every stage performs reliably, so end-to-end reliability R is the product of the stage reliabilities. Its net value is

$$W = V - C - L, \quad L = \$50,000 (1 - R), \quad (1.2)$$

where V is the gross value of processed alerts, C is daily operating cost, and L is expected failure loss.

RUNNING EXAMPLE R3: BENCHMARK AND DEPLOYMENT

Human benchmark. Reliability is $R = 0.95^4 = 0.8145$. Cost is $4 \times \$720 = \$2,880$ and expected failure loss is $L = 50,000 \times 0.1855 = \$9,275$. With gross value $V = 1,000 \times \$15 = \$15,000$, net value is $W = \$2,845$ a day.

Agents at stages 1–3. Reliability falls to $R = 0.98 \times 0.93 \times 0.90 \times 0.95 = 0.7792$, and expected loss rises to \$11,038. Cost falls to $172 + 98 + 296 + 720 = \$1,286$. The agents' 70 million tokens a day cost \$350, 61.8% of their \$566 combined cost. The source case reports gross value $V = \$16,884$ for this team, which reflects complementarities between agents and the analyst. Net value is $W = 16,884 - 1,286 - 11,038 = \$4,560$, a surplus of \$1,715 a day over the benchmark.

Attribution. The Shapley values of the agents, anchored to the human benchmark, are \$810 for A_D , \$620 for A_T , and \$285 for A_I ; they sum to the surplus.

The deployment is less reliable than the benchmark and still more valuable, because the priced objective already charges the extra failures through L . Gross value in R3 depends on which workers form the team, and later chapters take the coalition values from the source case rather than recomputing them.

Questions later chapters answer. How do latency and reliability compose across stages, and when does redundancy pay (Chapter 4)? What happens to consumption when the investigation agent delegates to sub-agents (Chapter 7)? What does a reliability floor cost (Chapter 9)? How should a token budget be allocated across the stages (Chapter 11)? How should surplus be attributed and agents priced (Chapter 12)? How large is the exposure to loss, and how can it be insured (Chapter 18)?

1.5 Three Preliminary Claims

Three claims recur in the book. Each contradicts an intuition common in practice, each is illustrated here with the running examples, and each is made precise in a later chapter.

Claim 1: Token savings are not automatically cost savings. A reduction in tokens becomes a reduction in cost only through prices, capacity, and the costs that surround the model. The R1 redesign shows three separate gaps.

1. *The price mix.* Tokens fall by 90%, but the model charge falls by 89.1%, because the redesign cut cheap input tokens faster than expensive output tokens. The metered cost per million tokens rises from \$3.20 to \$3.50.
2. *Adjacent cost.* The cache and its evaluation harness add \$120 a day, so the 89.1% cut in model charges is a 58.3% cut in total cost.
3. *Fixed capacity.* Suppose R1 runs on its dedicated pool, costing \$2,400 a day with capacity for 1.2 billion tokens. The baseline's 300 million tokens put the pool at $u = 0.25$ and a unit cost of \$8.00 per million. The redesign's 30 million tokens put it

at $u = 0.025$ and \$80.00 per million. The daily bill is \$2,400 either way. The saving appears only when the pool shrinks, and capacity comes in whole accelerators and multi-month commitments (Fig. 1.4a).

Chapter 4 develops the utilization mechanism, using the same pool with a redesign that omits the cache. Chapter 5 develops the procurement decision, and Chapter 9 the total cost of ownership. A fourth gap opens over time: a cheaper task invites more tasks, and Chapter 15 derives when an efficiency gain raises total spending.

Claim 2: Cost savings are not value. A cost saving that lowers quality must be compared with the cost of the failures it adds. The R1 redesign saves \$735 a day, but resolution falls from 0.78 to 0.76, which means $0.02 \times 50,000 = 1,000$ more escalations a day, costing \$4,000. The redesign loses \$3,265 a day, and every cost metric improved. The break-even calculation is simple. The saving could pay for at most $735 / (50,000 \times 4) = 0.37$ percentage points of lost resolution. Put the other way, a two-point regression breaks even only if an escalation costs less than $735 / 1,000 = \$0.735$. Chapter 9 shows that one percentage point of success is worth one percent of the failure cost per request, and that every quality floor implies such a price (Proposition 9.4). The converse also holds. In R3, the deployment is less reliable than the human benchmark and still adds \$1,715 a day, because its cost and value gains exceed the extra expected loss. Value is not monotone in cost, and it is not monotone in any single quality metric either.

Claim 3: Agentic consumption can diverge. When a model decides how much work to do, small changes in behavior can produce large, and even unbounded, changes in consumption. Two mechanisms are illustrated here with hypothetical parameters.

1. *Replay.* An agent loop starts from a 2,000-token task prompt, and each round adds 1,000 tokens of tool output and reasoning, all replayed on the next call. Ten rounds consume 65,000 input tokens. Forty rounds consume 860,000: four times the rounds, 13.2 times the tokens. The growth is quadratic (Theorem 7.6).
2. *Branching.* Suppose R3's investigation agent is redesigned to delegate. Each invocation uses 8,000 tokens and spawns 0, 1, 2, or 3 sub-investigations with probabilities 0.5, 0.3, 0.1, 0.1. The mean branching factor is $m = 0.8$, so an alert triggers $1 / (1 - m) = 5$ invocations and 40,000 tokens on average, the same as the original agent. A prompt edit shifts the probabilities to 0.45, 0.30, 0.10, 0.15. Then $m = 0.95$, an alert triggers 20 invocations and 160,000 tokens, and the agent's daily cost rises from \$296 to \$896, cutting the workflow surplus from \$1,715 to \$1,115 if nothing else changes. At $m \geq 1$ no finite expected budget exists (Theorem 7.13; Fig. 1.4b).

Chapter 7 proves both results, relates the cost threshold to the threshold at which errors proliferate through the same delegation tree, and designs the budgets and breakers that bound consumption.

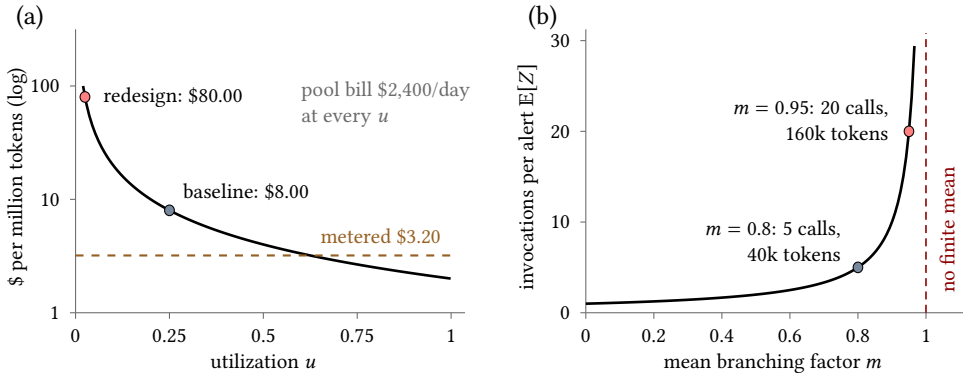


FIGURE 1.4. (a) Claim 1: on R1’s dedicated pool (hypothetical), unit cost is hyperbolic in utilization, so cutting tokens by 90% raises the cost per token tenfold and leaves the daily bill unchanged until capacity shrinks. The pool beats the metered price only above $u = 0.625$. (b) Claim 3: expected invocations per alert in a delegation tree are $1/(1 - m)$. The prompt edit that moves R3’s investigation agent from $m = 0.8$ to $m = 0.95$ quadruples expected consumption, and at $m \geq 1$ it is infinite.

KEY IDEA

Tokens, cost, and value are three different quantities. Tokens become cost through prices, capacity, and adjacent spending; cost becomes value only after failures are priced; and when models choose their own consumption, the mapping from task to tokens can itself be unstable.

1.6 How to Read This Book

The book has twenty-one chapters in seven parts, plus appendices. Part I (Chapters 1 to 3) establishes the token as a unit of account and explains the machine that produces it. Chapter 2 is written for readers without a systems background; readers who work on inference systems can skim it but should read its final section, which maps each mechanism to the economic variable it becomes. Parts II–IV follow the chain. Part V treats pricing and markets, and Part VI governance, risk, and society. Part VII applies the book to integrated cases (Chapter 20) and closes with open problems (Chapter 21). Appendix A reviews probability, queueing, branching processes, and convex optimization. Appendix B reviews demand, index numbers, contract theory, mechanism design, and cooperative games. Appendix C contains laboratory exercises, one of which uses the R2 data set, and Appendix D is a glossary.

TABLE 1.7. Suggested paths for a fourteen-week course. Numbers are chapters; letters are appendices.

	Engineering and systems	Economics and management	Research seminar
Audience	Graduate students in computer science and ML engineers	MBA and economics students, product and finance leaders	Doctoral students and researchers
Foundations	1, 2, 3; A	1, 3 (2 as reading); B	1, 3; A and B as needed
Core	4, 5, 6, 7, 8, 10	9, 13, 14, 15, 17	7, 11, 12, 14, 16, 18
Extensions	11, 17, 18	12, 18, 19	4 (optimal utilization), 15
Practice	20 (R1, R2); labs in C	20 (all cases, enterprise program)	21; a project from its research problems
Emphasis	Serving cost, consumption laws, routing	Denominators, pricing, contracts, governance	Proofs, open problems, the author's source papers

1.6.1 Chapter Dependencies

Figure 1.5 arranges the chapters in lanes that follow the chain. Within a lane, each chapter builds on the one above it. Chapters 1–3 are prerequisites for every lane. Dependencies across lanes are listed in gray inside each box. A reader interested in one domain can follow its lane from the top after Part I, consulting the cross-lane prerequisites as needed.

1.6.2 Suggested Course Paths

The book supports a two-semester sequence, but most readers will use a path through it. Table 1.7 gives three paths for a fourteen-week course. Each begins with the core: Chapters 1 and 3, and Chapter 2 for readers without a systems background. Each ends with at least one integrated case from Chapter 20. Every chapter has worked examples on the running examples and exercises with hints on about half. The numbers in worked examples are produced by scripts that accompany the book, so instructors can change a parameter and regenerate an example.

Short courses can be cut from these paths. A four-week module on AI unit economics can use Chapters 1, 3, 9, and the R1 case of Chapter 20. A module on agents can use Chapters 1, 7, 14, and 18. Readers who prefer to start from a problem can begin with any running example in this chapter and follow the forward references attached to it.

KEY IDEA

Read the book along the chain, not around it. Every lane starts from the token as a unit of account and ends in the integrated cases, where production, consumption, value, pricing, and governance must be decided together.

KEY TAKEAWAYS

- Once a model is deployed, inference dominates its cost. Unlike conventional software, a model-based service has a strictly positive marginal cost, and usage-based pricing moves volume, architectural, behavioral, and definition risk to the buyer.
 - AI tokenomics is the study of how tokens are produced, consumed, priced, valued, allocated, and governed, and of the decisions that change the relation between what token-metered systems cost and what they are worth. It is a design practice, complementary to FinOps, which controls realized spending. It is unrelated to cryptocurrency tokenomics.
 - The production–consumption–value chain separates three mechanisms: utilization and delay in production, architecture and delegation in consumption, and quality, fallbacks, and attribution in value. Pricing, markets, and governance act on all three, and the domains are coupled in both directions.
 - Three running examples recur: R1, a hypothetical support assistant (\$960 to \$105 a day in model charges); R2, an invoice agent calibrated on 480 real runs (hidden reasoning is 71.9% of the bill); and R3, a hypothetical security-operations workflow (surplus \$1,715 a day at lower reliability).
 - Token savings are not automatically cost savings; cost savings are not value (R1’s \$735 saving could afford only 0.37 points of lost resolution and lost two points); and agentic consumption can diverge (a mean branching factor of 0.95 quadruples consumption relative to 0.8, and at one or above there is no finite mean).
-

Notes and Further Reading

The economics of AI. The macroeconomic literature on AI asks how large its productivity effects will be and when they will appear. [Acemoglu \(2025\)](#) derives modest aggregate estimates from a task-based framework. [Brynjolfsson et al. \(2021\)](#) explain why general-purpose technologies first depress and later raise measured productivity, because firms must invest in intangible complements. The latter argument applies directly to the pilots and redesigns of this chapter, whose costs appear before their benefits. Micro evidence has accumulated quickly; [Brynjolfsson et al. \(2025\)](#), who study a conversational assistant in customer support, are the natural empirical companion to R1, and [Chapter 9](#) surveys the rest. On the economics of goods with high fixed and low marginal cost, [Shapiro and Varian \(1998\)](#) remains the standard reference, and the contrast with positive marginal cost motivates much of the book. The rebound effect has a long history in energy economics ([Jevons, 1865](#); [Alcott, 2005](#)).

Inference cost and its measurement. [Sardana et al. \(2024\)](#) bring inference demand into scaling laws, and [Patterson et al. \(2022\)](#) report measured energy splits between training

and inference. On the systems side, Pope et al. (2023) and Kwon et al. (2023) explain what makes inference expensive and how serving software reduces the cost; Chapters 2 and 4 build on both. Price measurement is younger. Cottier et al. (2025) and Gundlach et al. (2025) estimate price declines at constant capability, the AI Index tracks inference cost among many other indicators (Maslej et al., 2025), and Demirer et al. (2026) describe the business-to-business market for inference from marketplace data. The question of what such indexes should hold fixed is taken up in Chapter 3.

FinOps and practitioner frameworks. Storment and Fuller (2023) is the standard account of cloud FinOps, and the FinOps Foundation maintains the framework of capabilities and phases (FinOps Foundation, 2024). For AI specifically, the Tokenomics Foundation publishes a working definition (Tokenomics Foundation, 2026b,f), the production–consumption–value framing (Tokenomics Foundation, 2026d), an essay on how tokenomics and FinOps fit together (Tokenomics Foundation, 2026a), a five-layer stack (Tokenomics Foundation, 2026e), Big-T notation for token-consumption growth (Tokenomics Foundation and Neff, 2026), and an enterprise case sequenced as visibility, optimization, and value (Tokenomics Foundation, 2026c). Deloitte’s report and its web version frame token spending as a new category of enterprise cost (Merizzi et al., 2026a,b). The book reports these constructs where it uses them and evaluates two of them formally: Big-T in Chapter 8 and the five-layer stack in Chapter 11.

The word tokenomics. In software engineering, Salim et al. (2026) use the term for measuring where an agentic development system spends its tokens. Chen et al. (2026) survey token economics for LLM agents from computing and economic perspectives. The author’s research paper (Zhu, 2026d) and monograph (Zhu, 2026b) develop the economic framing that this book adapts. The running examples draw on the author’s work on pricing agentic AI (Zhu, 2026c) and on valuing agents in human–AI workflows (Zhu, 2026a).

Exercises

EXERCISE 1.1. For an AI product you use, list every activity that consumes tokens on your behalf, including those you do not see (system prompts, retrieval, reasoning, tool calls, retries, safety checks). Place each on the production–consumption–value chain and say who controls it: you, the application developer, or the model provider.

Hint: Reasoning and replayed conversation history are the most commonly overlooked. Ask for each item whether it would change if you rewrote your request.

EXERCISE 1.2. Explain in one paragraph why a 50% reduction in tokens per request need not reduce a firm’s AI spending, and in a second paragraph why it need not increase profit even if spending falls.

Hint: For spending, consider capacity that is paid for whether used or not, the mix of input and output tokens, and demand that grows when tasks become cheaper. For

profit, consider quality and the cost of failures.

EXERCISE 1.3. A pilot costs \$2,000 per month in model charges, and a business unit claims it saves 300 staff hours per month. List the additional information you would need before calling the pilot profitable, and for each item say whether it affects the cost side, the value side, or both.

EXERCISE 1.4. Compare seat-based software licensing with token-metered pricing. Which of the four risks of Section 1.1 move from vendor to customer under token pricing? Which move back under a committed-spend contract with a monthly allowance and overage charges?

EXERCISE 1.5. Using the parameters of R1, compute the daily tokens and model charges of a pilot of the *redesigned* assistant on 20% of traffic. Compare the result with the baseline pilot on 10% of traffic in the opening story, and explain why the comparison says nothing about whether either design is better.

Hint: The redesigned pilot uses 6 million tokens and \$21 a day.

EXERCISE 1.6. Repeat the break-even calculation of Claim 2 with an escalation cost of \$1 instead of \$4. How large a regression in resolution could the \$735 saving now afford? At what escalation cost is the observed two-point regression exactly offset?

EXERCISE 1.7. Classify each of the following as primarily a cost-accounting, FinOps, or tokenomics activity, and explain which chain domain it acts on: (a) tagging API keys by product team; (b) replacing four model calls with one; (c) negotiating a committed-spend discount; (d) setting a quality floor for insurance-claim summaries; (e) investigating why last month's bill exceeded forecast; (f) choosing whether to rent or buy accelerators.

EXERCISE 1.8. An agent invocation uses 8,000 tokens and spawns 0, 1, 2, or 3 sub-agents with probabilities 0.6, 0.2, 0.1, 0.1. Compute the mean branching factor, the expected number of invocations per task, and the expected tokens per task. Which single probability change would you monitor most closely, and why?

Hint: $m = 0.7$. Expected tokens per task are $8,000/(1 - m)$.

EXERCISE 1.9. From Table 1.5, rank the six R2 configurations by cost per run and by cost per correct answer. Then suppose every wrong answer must be corrected by a person at \$12. Compute the expected cost per invoice of each configuration and rank them again. What do the three rankings imply about choosing a model by its price per token?

Hint: With the correction cost, the expected cost per invoice is the cost per run plus $(1 - \text{accuracy}) \times \12 . The larger model with the verified strategy costs about \$5.16.

EXERCISE 1.10. Locate each of the following decisions on Fig. 1.2 and name one coupling through which it affects another domain: a power-purchase agreement for a data center; enabling prompt caching; raising a quality floor in a regulated workflow; switching from

per-token to per-outcome pricing with a vendor; installing a circuit breaker on agent spending.

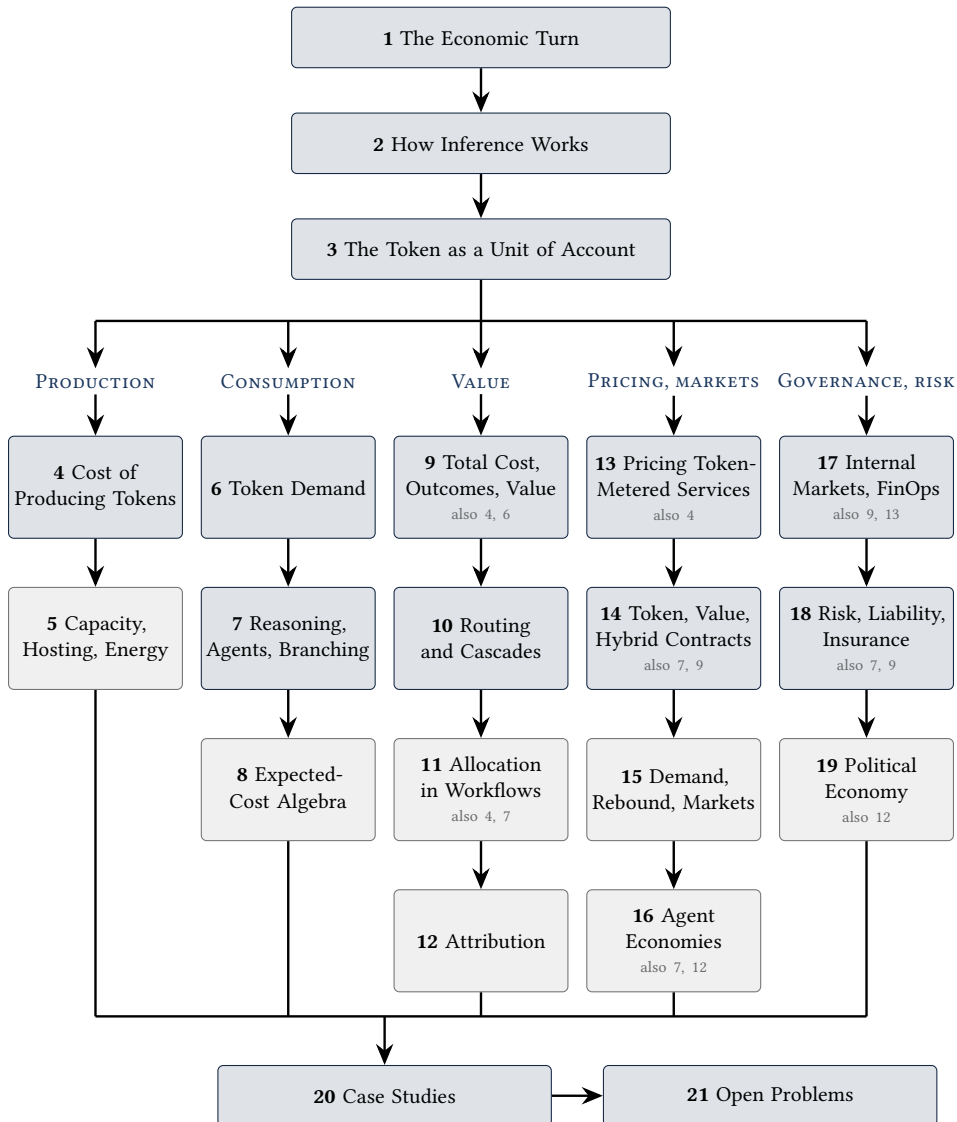


FIGURE 1.5. Chapter dependencies. After Part I, the chapters fall into five lanes that follow the chain and the institutions around it; each chapter builds on the one above it, and cross-lane prerequisites are listed in gray inside a box. Shaded boxes form the core used by every course path in Table 1.7. All lanes feed the case studies.

How Inference Works: A Primer for Economists and Managers

Economic models of AI are only as good as their picture of the machine. This chapter explains, for readers without a systems background, what a language model computes, what a token is, how a request is processed in a prefill and a decode phase, how batching and caching share hardware across users, and how reasoning, retrieval, and tool use turn one request into many model calls. A single request of the customer-support assistant (R1) is followed through the machine with every number computed: its arithmetic, its memory, its latency, and its cost at different batch sizes. The chapter ends with a map from each mechanism to the economic variable it becomes and the chapter in which that variable is modeled.

LEARNING OBJECTIVES

- Describe what a language model computes when it generates text, and estimate the arithmetic a forward pass requires from the parameter count.
 - Describe the prefill and decode phases and explain why output tokens cost more than input tokens.
 - Compute the key-value (KV) cache of a request from a model's architecture, and explain why memory rather than arithmetic usually limits how many users share an accelerator.
 - Explain batching and continuous batching, and read the throughput-latency trade-off off a table of batch sizes.
 - Explain what quantization, speculative decoding, and mixture-of-experts architectures change, and which cost term each one moves.
 - Describe how reasoning, retrieval, and tool use multiply model calls and tokens.
 - Map each serving mechanism to the economic variable it determines.
-

A manager who approves an AI budget sees a price list: so many dollars per million input tokens, a higher price per million output tokens, a discount for cached input, perhaps a surcharge for long prompts or for faster service. An economist who models the market sees a cost function. Neither sees the machine that produces the tokens, yet almost every feature of the price list is a fingerprint of that machine. Output costs more than input because of the way text is generated. Cached input is cheap because work has already been done and kept. Long prompts carry surcharges because they fill memory. Faster service costs more because it prevents the provider from sharing hardware across as many users.

This chapter opens the machine, just far enough to make those fingerprints legible. It does not teach anyone to build a serving system. It supplies the mechanisms that later chapters turn into economics, states each in the form in which it will be used, and computes one concrete request end to end so that the orders of magnitude are clear. [Section 2.1](#) describes the computation a model performs. [Section 2.2](#) covers tokens briefly, since [Chapter 3](#) treats their economics. [Section 2.3](#) follows one request through the prefill and decode phases and computes its time and cost. [Section 2.4](#) explains how a provider shares hardware across many requests and which engineering techniques change the cost of doing so. [Section 2.5](#) shows how reasoning, retrieval, and tool use turn one user request into many model calls. [Section 2.6](#) collects the mechanisms into a map from mechanism to economic variable.

The reader needs no background beyond high-school algebra. Wherever a result is derived formally elsewhere in the book, this chapter gives the intuition and a forward reference instead of the proof.

2.1 The Computation a Language Model Performs

2.1.1 *Next-Token Prediction*

A language model is a function with one job: given a sequence of tokens, it assigns a probability to every possible next token. A *token* is a fragment of text, often a word or part of a word, drawn from a fixed vocabulary of tens or hundreds of thousands of entries ([Section 2.2](#)). If the input is “The invoice total is”, the model might assign probability 0.31 to “\$”, 0.08 to “the”, and small probabilities to everything else. Text is generated by drawing one token from that distribution, appending it to the sequence, and asking the model again. [Figure 2.1](#) shows the loop.

Three consequences of this design carry most of the economics, and each is worth isolating.

Reading is parallel; writing is sequential. Every token of a prompt is known in advance, so the model can process all of them in one pass. The model cannot write its own answer in parallel, because token $t + 1$ depends on token t . This asymmetry is the origin of the two-phase structure of serving in [Section 2.3](#) and, through it, of the price gap between input and output tokens that [Proposition 3.6](#) derives.

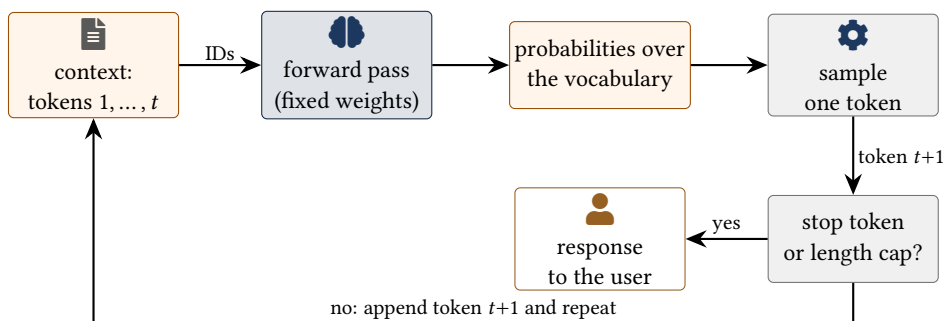


FIGURE 2.1. Generation as a loop. The model maps the tokens so far to a probability distribution over the next token; a sampler draws one; the token is appended and the loop repeats until a stop token or a length cap. Every pass through the loop is a full forward pass through the model, which is why each generated token has a cost of its own.

The weights are fixed at serving time. A model consists of a fixed set of numbers, its *parameters* or *weights*, and a fixed recipe of arithmetic that combines them with the input. Nothing a user does changes them. A model that lacks some piece of knowledge lacks it on every request until someone deploys a different model or supplies the knowledge in the input. That is why so much applied practice consists of changing what goes into the context, through instructions, examples, retrieved documents, and tool results, and why consumption is determined by the program around the model more than by the model itself (Chapter 6).

Output is stochastic. The sampler introduces randomness on purpose. A *temperature* setting sharpens or flattens the distribution before sampling, and truncation rules such as nucleus sampling discard the unlikely tail (Holtzman et al., 2020). Even at the most deterministic settings, numerical effects of batching can make identical requests produce different outputs. The economic consequence is that quality is a distribution, not a fixed property of a model. This is why the book poses cost reductions against a *quality floor* (Section 9.3) instead of assuming that a cheaper configuration delivers the same answer.

2.1.2 Parameters, Width, and Floating-Point Operations

The dominant architecture is the *transformer* (Vaswani et al., 2017). For our purposes it is a stack of n_ℓ identical *layers*. Each token enters the stack as a vector of d numbers, the model’s *width*. Each layer updates every token’s vector in two sub-steps. In *attention*, each token looks back at the vectors of the tokens before it and pulls in information from the ones it finds relevant. In the *feed-forward* block, each token’s vector is transformed on its own. Attention is split into several *heads* that look for different kinds of relevance in parallel, each working with vectors of dimension d_h . After the last layer, the final vector of the last token is converted into the probability distribution over the vocabulary.

Nearly all the parameters sit in large matrices inside the layers, and processing a token means multiplying its vector by those matrices. Counting operations gives the

most useful rule of thumb in this book (Kaplan et al., 2020):

💡 KEY IDEA

A forward pass through a dense transformer with P parameters costs about $2P$ floating-point operations per token: roughly one multiplication and one addition per parameter. Per-token work is proportional to model size, which makes the choice of model a first-order economic decision, not a tuning detail.

A 70-billion-parameter model therefore performs about 1.4×10^{11} operations for each token it reads or writes. Attention adds a second term. A token at position i compares itself with the i tokens before it in every layer, which costs about $2n_{\ell} d i$ operations. Summed over a prompt of S tokens, the comparisons cost about $n_{\ell} d S^2$ operations, so this term grows with the *square* of the prompt length, while the parameter term $2PS$ grows linearly. For short prompts the parameter term dominates and cost looks proportional to tokens. For very long prompts the quadratic term takes over; for the model used in this chapter the two terms are equal at a prompt of about 214,000 tokens (Theorem 2.1).

Parameter counts also connect inference to training. Scaling laws relate a model’s quality to its parameters and training data (Kaplan et al., 2020; Hoffmann et al., 2022). A developer who trains a model to be *compute-optimal* for training alone chooses a larger model than one who also counts the cost of serving it for years: once expected inference demand enters the objective, the better choice is a smaller model trained on more data (Sardana et al., 2024). Serving cost thus reaches back into the design of the model before a single user arrives, a point Chapter 4 develops.

2.2 Tokens and Tokenizers

2.2.1 Subword Tokenization

Before the model sees any text, a *tokenizer* converts it into a sequence of integers, one per token. The standard construction, byte-pair encoding, starts from single bytes and repeatedly merges the most frequent adjacent pair in a training corpus into a new vocabulary entry (Sennrich et al., 2016); related methods and toolkits reach similar vocabularies (Kudo and Richardson, 2018). The result is a vocabulary of *subword* units. A common English word is usually one token, a rare word or a long number several, and the leading space of a word is often part of its token. The model never sees characters, only these integers, and every downstream quantity in this chapter, from arithmetic to memory to the bill, is counted in them.

2.2.2 Why Token Counts Differ across Models and Languages

The vocabulary is learned from a particular corpus, so it is efficient for text that resembles that corpus and inefficient for text that does not. The same sentence can take several times as many tokens in one language as in another, and a different model family’s tokenizer will split it differently again (Petrov et al., 2023). Two consequences matter here. First, since serving work is proportional to tokens, a language or a data format that tokenizes poorly is more expensive to serve, not merely more expensive to buy: [Exercise 2.3](#) computes how a threefold token premium propagates through prefill, memory, and decode. Second, token counts from different model families are not in a common unit. [Chapter 3](#) develops both consequences, measures them across languages and tokenizers ([Section 3.2](#)), and shows why a fixed exchange rate between tokenizers must misstate some texts.

2.3 The Anatomy of a Request

2.3.1 Prefill and Decode

Serving one request proceeds in two phases with very different resource profiles ([Fig. 2.2](#)).

In *prefill*, the whole assembled input, meaning the system instructions, any retrieved documents, the conversation so far, and the user’s message, passes through the model in one go. Because every token of the input is already known, the model processes all S of them in parallel as a few very large matrix multiplications. The output of prefill is the probability distribution for the first token of the answer, plus a record of intermediate results for every input token, the KV cache described below.

In *decode*, the model writes the answer one token per step. Each step runs the model on a single new token, looks back through the record of all earlier tokens, and appends one more entry to that record. A decode step does little arithmetic: one token’s worth, about $2P$ operations. It must nevertheless touch every weight of the model, which means streaming the entire parameter set out of memory for every token produced. Decode is therefore limited by how fast the accelerator can move bytes between memory and its arithmetic units, its *memory bandwidth*, and not by how fast it can multiply ([Pope et al., 2023](#)).

The two phases define the two latency measures that matter to users, and they are not interchangeable. *Time to first token* (TTFT) is the wait before anything appears; it consists of queueing plus prefill and is what a user experiences as responsiveness. *Time per output token* (TPOT) is the interval between successive tokens once text starts to flow; it is the duration of one decode step and is what a user experiences as speed. A request that generates G tokens has an end-to-end latency of about

$$\text{latency} \approx \text{TTFT} + G \times \text{TPOT}. \quad (2.1)$$

[Section 2.4](#) shows that the provider’s main efficiency lever, batching, moves these two measures differently, and [Chapter 4](#) turns that into the throughput–latency frontier.

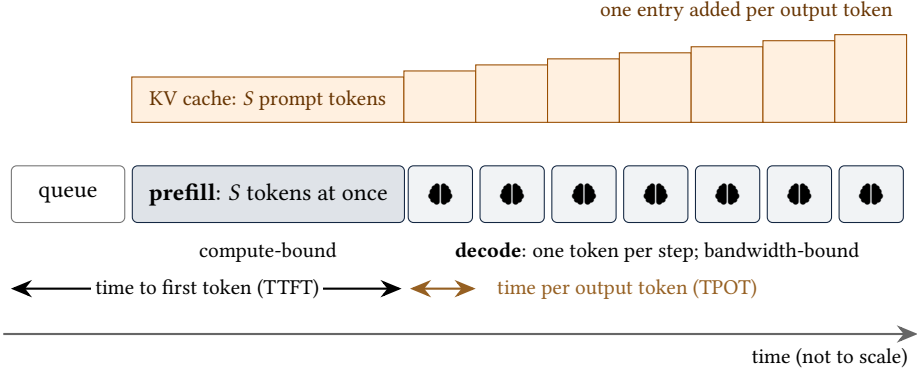


FIGURE 2.2. The anatomy of a request. After waiting in a queue, the whole prompt is processed in one prefill pass, which writes the key–value (KV) cache for every prompt token and yields the first output token. Decode then produces one token per step; each step reads the model’s weights and the growing cache and adds one entry to the cache. Time to first token is set by queueing and prefill; time per output token by the length of a decode step. In the walkthrough of [Theorem 2.1](#), prefill takes 0.34 seconds and each decode step about 0.18 seconds, so the 300 decode steps take more than 150 times as long as the prefill.

2.3.2 The KV Cache

Attention requires each new token to look back at every earlier token. Recomputing the earlier tokens’ contributions at every step would make each step as expensive as a fresh prefill. Instead, the model stores, for every earlier token, in every layer and every attention head, two vectors called the *key* and the *value*. This store is the *key–value cache*, or KV cache. Prefill writes it for all prompt tokens; each decode step reads all of it and appends one entry.

The cache trades memory for arithmetic, and its size follows directly from the architecture. With n_ℓ layers, n_{kv} key–value heads per layer, head dimension d_h , and b_{kv} bytes per stored number, each token of context occupies

$$\kappa = 2 n_\ell n_{kv} d_h b_{kv} \quad (2.2)$$

bytes, where the factor 2 counts the key and the value. A sequence with S tokens of context holds κS bytes. This is the quantity κ that appears in [Proposition 3.6](#) and [Theorem 4.3](#).

The number of key–value heads is a design choice with large economic consequences. In the original transformer every attention head kept its own keys and values. *Grouped-query attention* lets several query heads share one key–value head, which cuts n_{kv} , and hence the cache, by the sharing factor with little loss of quality ([Ainslie et al., 2023](#)). Other architectures compress the cache further, for example by storing a low-dimensional latent from which keys and values are reconstructed ([DeepSeek-AI, 2024](#)). Serving systems also manage the cache as an economic resource: they allocate it in fixed-size pages to avoid

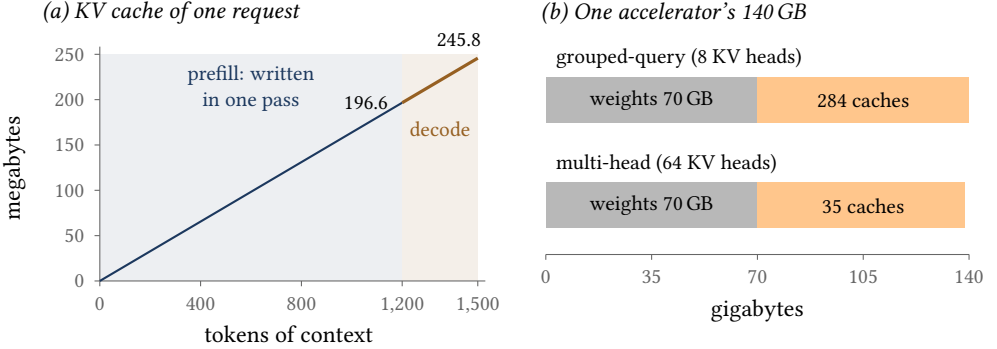


FIGURE 2.3. The KV cache of the walkthrough model (80 layers, 8 key–value heads of dimension 128, 8-bit storage: 163,840 bytes per token; all hypothetical). (a) One R1 call: prefill writes 196.6 MB for the 1,200-token prompt at once, and the 300 decode steps add 49 MB more. Without grouped-query attention the cache would be eight times larger, 1.97 GB at the end. (b) After 70 GB of 8-bit weights, the remaining 70 GB holds the caches of 284 sequences of 1,500 tokens with grouped-query attention, but only 35 with a key–value head per query head. Computed by `tools/numbers/inference/numbers.py`.

fragmentation (Kwon et al., 2023), evict entries that attention rarely uses (Zhang et al., 2023), or keep only a window of recent tokens plus a few initial ones (Xiao et al., 2024). Each of these raises the number of sequences that fit in memory, which, as Section 2.4 shows, lowers the cost per token.

Figure 2.3 shows the cache for the request computed below. Panel (a) shows its growth through a single request. Panel (b) shows why the cache, not the weights, decides how many users an accelerator can serve at once.

2.3.3 Where the Time and Money Go

We now follow one request through a hypothetical machine and compute every quantity. The request is a single model call of running example R1, the customer-support assistant (Section 1.4), with the call size used throughout the book. The model and hardware are deliberately the same as in the roofline example of Chapter 4 (Theorem 4.4), so the two chapters can be read together. Table 2.1 lists the parameters.

EXAMPLE 2.1 (One R1 call, computed end to end).

1. *The bill.* At R1's prices the call is metered at $1,200 \times \$2/10^6 + 300 \times \$8/10^6 = \$0.0024 + \$0.0024 = \$0.0048$. Input is 80% of the tokens and half of the charge.
2. *Prefill arithmetic.* The parameter term is $2PS = 2 \times (7 \times 10^{10}) \times 1,200 = 1.68 \times 10^{14}$ operations. The attention term is about $n_t d S(S+1) = 80 \times 8,192 \times 1,200 \times 1,201 = 9.4 \times 10^{11}$ operations, 0.56% of the total, so for a prompt this short cost is effectively linear in tokens. (The attention share would be 13% at $S = 32,000$ and 38% at $S = 128,000$.)

TABLE 2.1. Parameters of the walkthrough in [Theorem 2.1](#). The token counts and prices are R1’s ([Section 1.4](#)); the model and accelerator are hypothetical, of realistic orders of magnitude, and identical to [Table 4.1](#).

Quantity	Symbol	Value
Prompt length; generated length (R1 call)	$S; G$	1,200; 300 tokens
Prices per million input and output tokens (R1)	$p_{\text{in}}; p_{\text{out}}$	\$2; \$8
Dense parameters; bytes per parameter (8-bit)	$P; b$	70×10^9 ; 1
Layers; query heads; key–value heads; head dimension	$n_{\ell}; -; n_{\text{kv}}; d_h$	80; 64; 8; 128
Width	d	8,192
Bytes per cached number (8-bit KV cache)	b_{kv}	1
Effective compute	Π	5×10^{14} operations/s
Effective memory bandwidth	W	400 GB/s
Memory capacity	M_{mem}	140 GB
All-in accelerator cost (Theorem 4.1)	h	\$1.94/hour

3. *Prefill time.* At $\Pi = 5 \times 10^{14}$ operations per second the arithmetic takes $1.68 \times 10^{14} / (5 \times 10^{14}) = 0.336$ seconds. Prefill must also read the 70 GB of weights once, which at $W = 400$ GB/s takes 0.175 seconds, so arithmetic is the binding constraint: prefill is *compute-bound*. With no queue, the time to first token is about 0.34 seconds.
4. *The KV cache.* By [Eq. \(2.2\)](#), $\kappa = 2 \times 80 \times 8 \times 128 \times 1 = 163,840$ bytes per token. After prefill the cache holds 1,200 $\kappa = 196.6$ MB; after the last output token it holds 1,500 $\kappa = 245.8$ MB. With 64 key–value heads instead of 8 it would reach 1.97 GB.
5. *Decode alone on the accelerator.* Suppose this is the only request being served (batch size $B = 1$). Each step must read the 70 GB of weights plus the cache, about 70.2 GB, which takes $70.2/400 = 0.1755$ seconds. The arithmetic of the step, $2P = 1.4 \times 10^{11}$ operations, takes only 0.28 milliseconds; the accelerator’s arithmetic units spend more than 99.8% of the step waiting for data. Decode is *memory-bound*. The 300 steps take 52.7 seconds: 5.7 tokens per second, a TPOT of 176 milliseconds.
6. *Where the time goes.* The request occupies the accelerator for $0.336 + 52.67 = 53.0$ seconds, of which prefill is 0.6%. An input token took 0.28 milliseconds of accelerator time and an output token 176 milliseconds, 627 times as long. At $h = \$1.94$ per hour, the 53 seconds cost \$0.0286, 5.95 times what the call is metered at. Served alone, this request would lose money at any plausible price.
7. *Decode shared with 31 other requests.* Now let the step serve $B = 32$ sequences, each with about 1,350 tokens of context. A step reads $70 + 32 \times 1,350 \times 163,840$ bytes ≈ 77.1 GB and takes 192.7 milliseconds; it produces one token for each of 32 sequences, 166 tokens per second in all. Our request’s share of decode time is $300 \times 0.1927/32 = 1.81$ seconds, and with its 0.336 seconds of prefill it uses 2.14 accelerator-seconds, costing \$0.00115, 24% of its metered charge. Prefill is now 16%

of the request's cost. The output/input ratio of accelerator time per token has fallen from 627 to 21.5, exactly the value [Proposition 3.6](#) gives for a machine balance of $\Pi/W = 1,250$ operations per byte.

8. *What the user sees.* Batching barely changes TPOT (176 to 193 milliseconds), but 300 tokens at 5 tokens per second is still a minute of waiting. Splitting the model across four such accelerators with tensor parallelism (each holds a quarter of every weight matrix and they compute together) roughly quarters the bytes each must move per step. Ignoring communication overhead, TPOT falls to 48 milliseconds at $B = 32$ (21 tokens per second) and TTFT to 0.08 seconds. The cost per request does not fall: four accelerators working a quarter as long use the same accelerator-seconds. Parallelism of this kind buys latency, not cost.

The walkthrough compresses several of the book's recurring themes into a few lines. The metered charge treats an input token as a quarter of an output token, but the machine does not: at batch size one an output token costs more than 600 input tokens of accelerator time, and at batch size 32 about 21. The ratio the machine produces depends on how many requests share the hardware, which the customer does not control and cannot observe. The cost of the same request differs by a factor of 25 between batch sizes 1 and 32 on identical hardware. And a single visible number, the time a user waits, depends on the product of a token count and a hardware property.

KEY IDEA

Prefill is bounded by arithmetic and decode by memory bandwidth. Almost all of a request's accelerator time is spent writing its output, one full read of the model's weights per token, which is why output tokens cost more than input tokens and why sharing those weight reads across many users is the main source of efficiency in serving.

RUNNING EXAMPLE R1: WHAT THE SUPPORT ASSISTANT'S PROMPT COSTS THE MACHINE

In R1's baseline design each call carries 1,200 input tokens, much of them instructions and retrieved help-center text that is identical across calls. The walkthrough shows what that prompt costs: 0.34 seconds of compute-bound prefill and 197 MB of cache per call, held for the life of the request. The book's redesign of R1 halves the prompt to 600 tokens and cuts the output to 200 tokens ([Section 1.4](#)). At the machine level this halves prefill work and cache per call, and, because decode dominates, it cuts decode time by a third. Whether the redesign pays is a different question. [Chapter 9](#) shows that its 89% cut in model charges becomes a net loss once the fall in first-contact resolution is priced.

✂ IN PRACTICE (AS OF SEPTEMBER 2026)

Context windows, the maximum number of tokens a model accepts, have grown large, and their pricing shows the cost of memory. Anthropic’s model overview lists a 1-million-token context window and 128,000 maximum output tokens for Claude Fable 5.1, Claude Opus 5, and Claude Sonnet 5, and 200,000 and 64,000 for Claude Haiku 4.5; it notes that 1 million tokens is about 555,000 English words on the tokenizer introduced with Claude Opus 4.7 (Anthropic, 2026b). OpenAI’s documentation lists GPT-5.5 with a 1,050,000-token context window and 128,000 maximum output tokens, and states that prompts with more than 272,000 input tokens are priced at twice the input rate and 1.5 times the output rate for the session (OpenAI, 2026b). The long-context surcharge on output is consistent with the mechanics of this section: a longer context enlarges every decode step’s cache read and leaves room for fewer sequences in a batch (Proposition 3.6 and Exercise 2.6). Posted per-token prices are collected in the box after Proposition 3.6.

2.4 Concurrent Serving

2.4.1 *Batching and Continuous Batching*

Step 7 of the walkthrough contains the central fact of serving economics. A decode step reads the model’s weights once regardless of how many sequences it advances, so advancing 32 sequences in the same step costs barely more time than advancing one. Grouping sequences into one step is called *batching*. The number of sequences advanced together is the *batch size* B .

A restaurant kitchen offers a useful picture. Heating the oven is the expensive part; putting in one tray or ten costs about the same. A kitchen that heats the oven for every single order is slow and wasteful. A kitchen that waits until it has ten orders makes the first customer wait. Serving systems face the same tension, with one extra complication: orders differ enormously in how long they take to cook.

Table 2.2 traces the trade-off on the walkthrough’s accelerator for sequences with 1,500 tokens of context. The formula behind each row is the memory-bound step time

$$t(B) \approx \frac{bP + BS\kappa}{W}, \quad (2.3)$$

which is the first branch of Eq. (4.2). Throughput is $B/t(B)$ tokens per second, and each user receives $1/t(B)$ tokens per second.

Read down the table. From $B = 1$ to $B = 32$, throughput rises 29-fold while each user’s speed falls by less than 10%: batching at small sizes is nearly free. Beyond that, each doubling of the batch adds less throughput and costs users more speed, because the caches of the extra sequences add bytes to every step as fast as the sequences add tokens. At the memory limit the cost floor is \$0.66 per million tokens, 143 times lower than at

TABLE 2.2. Batching on the walkthrough accelerator (Table 2.1), with 1,500 tokens of context per sequence. The step time is Eq. (2.3); “arithmetic” is the time the step’s $2PB$ operations would take at Π , always far below the step time, so the step is memory-bound throughout. The memory limit is $\lfloor (140 - 70) \text{ GB} / (1,500 \kappa) \rfloor = 284$ sequences. Cost uses $h = \$1.94$ per accelerator-hour and full utilization. All values hypothetical. Theorem 4.4 performs the same calculation for 2,000-token contexts.

Batch size B	Step time (ms)	Per user (tokens/s)	Throughput (tokens/s)	Cost floor (\$ per million)	Arithmetic (ms)
1	175.6	5.69	5.7	94.64	0.3
8	179.9	5.56	44.5	12.12	2.2
32	194.7	5.14	164.4	3.28	9.0
64	214.3	4.67	298.6	1.80	17.9
128	253.6	3.94	504.6	1.07	35.8
256	332.3	3.01	770.4	0.70	71.7
284	349.5	2.86	812.6	0.66	79.5

$B = 1$, and each user receives half the speed. No row is better than another without a value for latency. That value is set by the product, not by the hardware, which is why Chapter 4 treats the operating point as an economic choice (Fig. 4.3) and derives the concavity of throughput and the cache bound formally (Theorem 4.3).

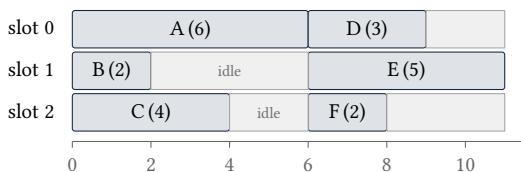
Batching is easy when all sequences start and finish together, and they do not. In *static batching*, a group of requests is admitted together and runs until its longest member finishes; slots whose request has already finished sit idle, and newly arrived requests wait for the whole group. *Continuous batching* admits a waiting request into any slot the moment it frees, at the boundary of the next decode step (Yu et al., 2022). Figure 2.4 compares the two on six requests and three slots. Continuous batching finishes the same work in 9 steps instead of 11, raises slot utilization from 67% to 81%, and lowers the mean completion time from 6.7 to 5.5 steps. The gain comes entirely from scheduling, with no change of hardware or model. Paged KV-cache memory complements it: by storing each sequence’s cache in fixed-size blocks, it avoids reserving memory for the longest possible output and so lets more sequences fit (Kwon et al., 2023).

Batching is also the mechanism behind two features of commercial offers. A *batch* or *flex* tier, in which the customer accepts results within hours instead of seconds, lets the provider run at the bottom rows of Table 2.2 and fill capacity that interactive traffic leaves idle, which is why such tiers are discounted. A *priority* or *fast* tier does the reverse. Both are prices for positions on the same trade-off, analyzed in Chapters 4 and 13.

2.4.2 Memory-Bound versus Compute-Bound Operation

The walkthrough found prefill limited by arithmetic and decode limited by memory. A simple ratio decides which constraint binds for any workload: the *arithmetic intensity*, the number of operations performed per byte moved. Prefill of S tokens performs $2PS$

(a) Static batching: 11 steps, 67% of slot-steps used



Six requests A–F, output lengths in parentheses, all waiting at step 0. In (a) a new group starts only when the longest member of the old group finishes. In (b) a waiting request takes any slot as soon as it frees.

(b) Continuous batching: 9 steps, 81% of slot-steps used

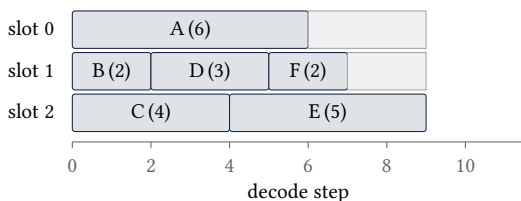


FIGURE 2.4. Static versus continuous batching on three slots. Continuous batching keeps the batch full, so the same hardware finishes the same six requests in fewer steps, and short requests are not held hostage by long ones. The slot utilization and mean completion times in the text are computed by `tools/numbers/inference/numbers.py`.

operations while reading bP bytes of weights, an intensity of $2S/b$: 2,400 operations per byte for the walkthrough prompt. A decode step with batch B performs $2PB$ operations while reading at least bP bytes, an intensity of at most $2B/b$: 2 at $B = 1$. An accelerator, in turn, can perform Π operations per second and move W bytes per second; the ratio Π/W , its *machine balance*, is 1,250 operations per byte in Table 2.1. A workload whose intensity exceeds the machine balance runs out of arithmetic first; one whose intensity falls below it runs out of bandwidth first. This is the roofline model of computer architecture (Williams et al., 2009), applied to transformer inference by Pope et al. (2023); Fig. 4.1 draws it.

Figure 2.5 presents the same comparison as time. For each workload it shows how long the step’s arithmetic would take at full compute speed and how long its data movement would take at full bandwidth. The step takes the longer of the two.

The picture explains several facts that otherwise look like marketing. Accelerator generations are increasingly differentiated by memory capacity and bandwidth rather than raw arithmetic, because decode, the phase that dominates cost, is starved of bytes (see the box in Section 4.3 for current figures). Serving systems increasingly separate the phases, running compute-bound prefill and memory-bound decode on separately provisioned pools (Zhong et al., 2024; Patel et al., 2024; Qin et al., 2025), and Theorem 4.9 derives when that pays. Techniques that interleave pieces of long prompts with ongoing decode steps reduce the interference between the phases on shared hardware (Agrawal et al., 2024). Memory, more than arithmetic, is the wall modern AI workloads hit (Gholami et al., 2024).

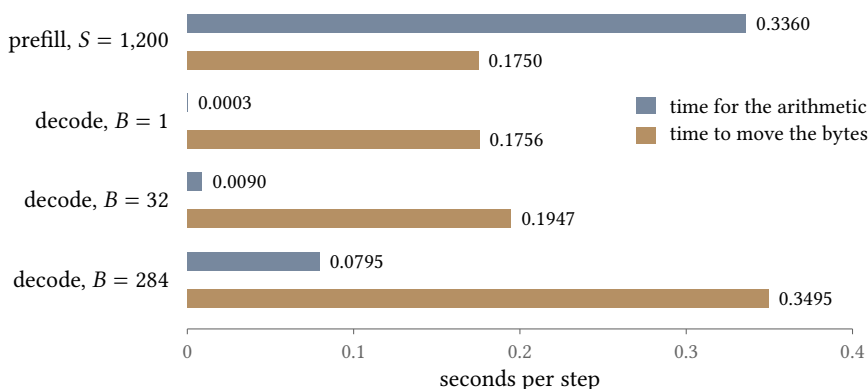


FIGURE 2.5. Compute-bound and memory-bound work on the walkthrough accelerator ($\Pi = 5 \times 10^{14}$ operations/s, $W = 400$ GB/s; contexts of 1,500 tokens in decode; all hypothetical). For prefill the arithmetic takes longer than reading the weights, so compute binds. For decode, moving the weights and caches takes 600 times longer than the arithmetic at $B = 1$ and still more than four times longer at the memory limit $B = 284$, so bandwidth binds at every feasible batch size.

💡 KEY IDEA

The cost of a token depends on what else the hardware is doing. The same request on the same accelerator costs 25 times less when its decode steps are shared with 31 others, and the output premium the machine produces falls with the batch. Utilization, batching, and scheduling are therefore properties of the provider's whole fleet that show up in every customer's unit cost.

2.4.3 Quantization, Speculative Decoding, and Mixture of Experts

Many engineering techniques lower the cost of a token. Three change the economics enough to deserve a description here, and they act on different terms. Table 4.2 lists a longer set with the bound each one relieves; surveys include Yuan et al. (2024), Zhou et al. (2024), and Miao et al. (2025).

Quantization. Weights are usually trained as 16-bit numbers. *Quantization* stores them, and sometimes the activations and the KV cache, with fewer bits: 8-bit (Dettmers et al., 2022; Xiao et al., 2023) or 4-bit (Frantar et al., 2023; Lin et al., 2024). Fewer bits per weight means a smaller b and so fewer bytes read per decode step and more room in memory for caches. At batch size one, halving b from 8 to 4 bits nearly halves the walkthrough's step time; at large batches, where caches account for much of the bytes, the gain is smaller (Exercise 2.5). Quantization can reduce quality, by an amount that depends on the model, the method, and the task, so it is a cost–quality trade-off to be measured rather than assumed.

Speculative decoding. Decode wastes arithmetic: each step reads all the weights but uses them for only one token per sequence. *Speculative decoding* spends that idle arithmetic on guesses (Leviathan et al., 2023; Chen et al., 2023). A cheap *draft* mechanism, such as a small model or extra prediction heads on the large one (Cai et al., 2024; Li et al., 2024), proposes the next γ tokens. The large model then checks all γ proposals in a single pass, which reads the weights once, as a decode step does, and accepts the longest correct prefix plus one token of its own. The acceptance rule is designed so that the output has exactly the distribution the large model alone would have produced. If each draft token is accepted independently with probability φ , the expected number of tokens produced per pass of the large model is

$$\frac{1 - \varphi^{\gamma+1}}{1 - \varphi}, \quad (2.4)$$

which is 3.36 for $\varphi = 0.8$ and $\gamma = 4$ (Leviathan et al., 2023). The method converts draft accuracy into fewer weight reads per token. Unlike most levers in this section, it does not trade quality for cost. Its costs are the draft computation and the work spent on rejected guesses, which consume capacity without reaching a user; Xia et al. (2024) survey the design space.

Mixture of experts. A *mixture-of-experts* model replaces each feed-forward block with many parallel blocks, the *experts*, and a small router that sends each token to a few of them (Shazeer et al., 2017; Fedus et al., 2022); Mixtral, for example, routes each token to two of eight experts per layer (Jiang et al., 2024). The model holds P parameters but activates only $P_{\text{act}} \ll P$ per token. Arithmetic per token is then about $2P_{\text{act}}$, but memory must still hold all P . Sparsity therefore changes the shape of serving more than it simply lowers cost. It lowers arithmetic, and it lowers bytes read per token at small batch sizes. At large batch sizes different tokens select different experts, so a step reads a growing fraction of all the weights, and the memory needed to hold the full model, often spread across several accelerators, becomes the binding input (Exercise 2.9).

✂ IN PRACTICE (AS OF SEPTEMBER 2026)

- *Sparse architectures.* OpenAI’s open-weight gpt-oss-120b has 117 billion parameters with 5.1 billion active per token, 128 experts with 4 selected per token, 36 layers with 64 query heads and 8 key–value heads, and a position limit of 131,072 tokens; its mixture-of-experts weights are stored in the 4-bit MXFP4 format, and the model card states it runs on a single 80 GB GPU (OpenAI, 2025b). DeepSeek-V3 has 671 billion parameters with 37 billion active per token and uses multi-head latent attention to compress the KV cache (DeepSeek-AI, 2024). Moonshot AI’s Kimi K2 has 1 trillion parameters with 32 billion activated, 384 experts with 8 selected per token, and a 128,000-token context (Moonshot AI, 2025). Active parameters are thus between roughly 3% and 6% of total parameters in these models.

- *Low-precision formats.* NVIDIA’s NVFP4 format stores values in 4 bits with an 8-bit scale shared by each block of 16 values; NVIDIA reports a memory footprint about 3.5 times smaller than 16-bit and 1.8 times smaller than 8-bit, and an accuracy loss of 1% or less on its evaluations of DeepSeek-R1-0528 when quantized from 8 to 4 bits, with hardware support in its Blackwell accelerators (Alvarez et al., 2025).
- *Speculative decoding in serving software.* The open-source vLLM engine documents several draft mechanisms, including separate draft models, EAGLE, multi-token prediction heads, n-gram matching, and suffix decoding, and describes speculative sampling as theoretically lossless up to the precision limits of hardware numerics (vLLM Project, 2026).

Accelerator memory and bandwidth figures are in the box in [Section 4.3](#); reasoning-effort controls are in the box in [Section 7.1](#).

2.5 Reasoning, Retrieval, and Compound Systems

Everything so far concerned a single model call. Few deployed products consist of one. This section describes the three patterns that multiply calls and tokens, and computes one compound request.

2.5.1 Chain-of-Thought and Hidden Reasoning Tokens

A model that writes out intermediate steps before its answer is more accurate on many multi-step problems. Prompting for such a *chain of thought* was the first demonstration (Wei et al., 2022). Current *reasoning models* are trained with reinforcement learning to generate long chains on their own before answering (Guo et al., 2025). Mechanically, reasoning tokens are ordinary output tokens: each is one decode step, reads the weights and the cache once, and adds an entry to the cache. What distinguishes them is economic. Providers commonly do not return them to the customer, or return only a summary, yet bill them at the output price (Definition 3.4). They also add latency by Eq. (2.1): 1,200 reasoning tokens in front of R1’s 300-token answer multiply decode time by five (Exercise 2.7).

Reasoning is a way to buy accuracy with inference compute instead of with a larger model. Allocating test-time compute well across prompts can outperform a model many times larger at matched compute (Snell et al., 2024). The economics of that purchase, including the optimal reasoning budget under diminishing returns, is the subject of [Section 7.1](#) (Theorem 7.2).

RUNNING EXAMPLE R2: WHERE DECODE TIME GOES IN THE INVOICE AGENT

In the real R2 experiment (Section 1.4), reasoning tokens were 61.6% of billed tokens and 71.9% of billed cost across 480 runs (Zhu, 2026c). The mechanics of this chapter explain why the cost share exceeds the token share. Every reasoning token is a decode step, billed at the output price, while the prompts were short and cheap to prefill. The invoice agent spends most of its machine time, and most of its customer’s money, on text nobody reads. Chapter 3 decomposes the bill cell by cell (Fig. 3.3).

2.5.2 Retrieval-Augmented Generation

A model’s weights cannot contain a firm’s current price list, contracts, or help-center articles. *Retrieval-augmented generation* (RAG) fetches relevant passages from an external index at request time and inserts them into the prompt (Lewis et al., 2020). The retrieval step itself usually involves no generative model call, only a search over precomputed vector representations, so its cost is small and largely fixed. Its effect on inference is entirely through the prompt. Five passages of 250 tokens each add 1,250 input tokens to an R1 call: more than doubling its prefill work, adding 205 MB of cache held through the whole decode, and adding \$0.0025 to its metered charge. Retrieval is nevertheless a device for *saving* tokens, because the alternative to sending five relevant passages is sending the whole manual or fine-tuning the model. How many passages to retrieve, and whether more context helps at all, are economic questions that Chapter 6 develops (Section 6.4), including the evidence that models use information in the middle of long contexts less well than information at its edges (Liu et al., 2024).

2.5.3 Tool Use and Agent Loops

A model can also be given *tools*: descriptions of functions, such as an order-lookup service or a web search, that it may ask the surrounding program to run. The model writes a structured request, the program executes it, and the result is appended to the context for another model call (Schick et al., 2023). Repeating this until the model declares the task done is an *agent loop* (Yao et al., 2023). A system built from several model calls, retrievers, tools, and checks is a *compound AI system* (Zaharia et al., 2024).

Every pass around the loop is a new request to the serving machinery of Section 2.3: a new prefill of the whole context, because the model’s interfaces are usually stateless, and a new decode. Two variables summarize the multiplication. The *call multiplier* k is the number of model calls per user-visible request. For agents that spawn other agents, the mean number of child invocations per invocation is m . Neither is a property of the model; both are properties of the program wrapped around it, and both are usually invisible in product analytics.

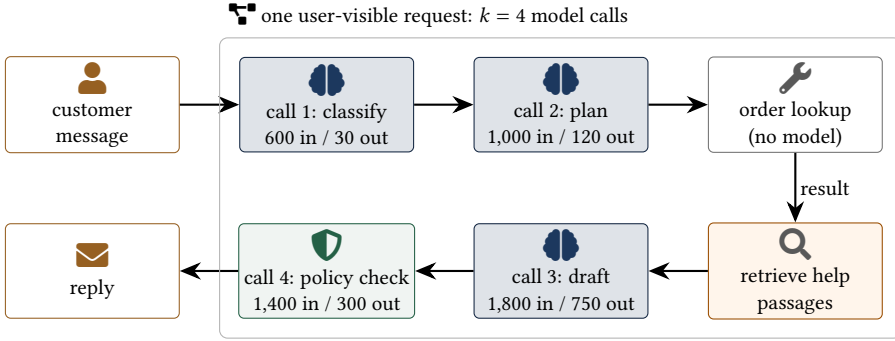


FIGURE 2.6. One R1 baseline request as a compound system. Four model calls (the policy check is a model acting as a verifier), one tool call, and one retrieval step sit behind a single reply. Token counts per call are a hypothetical decomposition whose averages equal R1’s 1,200 input and 300 output tokens per call. Costs and latencies are computed in Theorem 2.2.

EXAMPLE 2.2 (One R1 request becomes four calls). R1’s baseline design makes $k = 4$ model calls per customer message, averaging 1,200 input and 300 output tokens each (Section 1.4). Figure 2.6 shows one hypothetical decomposition consistent with those averages. Take the four-accelerator server of step 8 of Theorem 2.1 running at $B = 32$: prefill takes about 0.07 milliseconds per input token and decode 48.2 milliseconds per output token.

Call	Input	Output	Charge (\$)	Latency (s)
1. classify intent	600	30	0.00144	1.5
2. plan and request order lookup	1,000	120	0.00296	5.9
3. draft answer with passages	1,800	750	0.00960	36.3
4. policy check	1,400	300	0.00520	14.6
Request	4,800	1,200	0.01920	58.1

The request costs four times a single call, \$0.0192, or \$960 per day at 50,000 requests, R1’s baseline model charge. Its calls are sequential, because each consumes the previous one’s output, so their latencies add: 58.1 seconds of model time plus, say, 0.8 seconds for the order lookup and 0.3 seconds for retrieval (hypothetical), about 59 seconds, against 14.5 seconds for one call of average size. The customer waits for four prefills and 1,200 decode steps, and the drafting call alone accounts for 62% of the wait. None of this is visible in a metric that counts messages or averages tokens per call.

Two features of the example generalize. First, costs add across calls but latencies add only across *sequential* calls; calls that do not depend on one another can run in parallel, trading more simultaneous capacity for less waiting. Second, under the stateless interfaces used here each call re-sends its whole context. In an agent that appends every action and observation to its history, later calls re-send everything earlier calls admitted, so the tokens of a K -call loop grow with K^2 . Chapter 6 develops context replay

for conversations, and [Section 7.2](#) decomposes agent calls with an accounting identity ([Theorem 7.4](#)) and proves the replay law ([Theorem 7.6](#)); delegation, in which calls spawn further calls, is modeled as a branching process in [Section 7.3](#). [Chapter 8](#) composes these pieces into workload cost estimates.

💡 KEY IDEA

The number of model calls behind one user-visible request is set by the program around the model, not by the model. Costs add over every call, latency adds over every sequential call, and under context replay the tokens of a loop grow faster than its length. For agentic products, the call multiplier is usually a larger lever than the price of a token.

RUNNING EXAMPLE R3: A WORKFLOW METERED IN TOKENS PER DAY

In the security-operations workflow R3 ([Section 1.4](#)), 1,000 alerts a day pass through detection, triage, and investigation agents before a human recommends a response. The agents consume 20, 10, and 40 million tokens a day. At 1,000 alerts that is an average of 70,000 tokens per alert, the equivalent of about 47 single R1 calls, generated by loops of model calls, tool queries against logs and threat intelligence, and hand-offs between agents. The investigation agent alone accounts for 57% of the tokens. [Chapters 11](#) and [12](#) ask how a token budget should be allocated across such stages and how the workflow's value should be attributed to them.

2.6 From Mechanism to Economic Variable

[Table 2.3](#) collects the correspondence that the rest of the book relies on. The left column is a mechanism from this chapter; the middle column is the economic quantity it becomes; the right column is where that quantity is modeled.

The organizing observation is that the mechanisms fall into two groups that behave very differently. Hardware and serving mechanisms determine what a token costs to produce. They are controlled by whoever operates the capacity, they are bounded below by physics, and an entire industry is pushing on that bound. Application mechanisms determine how many tokens an outcome requires. They are controlled by whoever writes the program around the model, they have no natural bound, and they are where order-of-magnitude differences between two deployments of the same model on the same task arise. The walkthroughs of this chapter show both: batching changes the cost of an identical call by a factor of 25, and the program around R1 multiplies calls per request by four.

One further mechanism does not appear in any serving system but constrains every decision in the table: the measurement of quality. Static benchmarks are cheap and comparable but drift away from a firm's own tasks ([Liang et al., 2023](#)). Model-graded

evaluation scales but carries known biases (Zheng et al., 2023). Pairwise human preference platforms aggregate many judgments but measure preference, not correctness on a particular workload (Chiang et al., 2024). Held-out sets of a firm’s own tasks and human review are the most decision-relevant and the most expensive. Because every cost reduction must be checked against a quality floor, an organization that cannot measure quality at the resolution of the savings it seeks cannot safely accept any of them. [Chapter 9](#) makes this precise.

KEY TAKEAWAYS

- A language model predicts one token at a time with fixed weights. A dense model with P parameters performs about $2P$ operations per token, plus an attention term that grows with the square of the context length.
 - Serving has two phases. Prefill processes the whole prompt at once and is limited by arithmetic. Decode writes one token per step, reads all the weights each step, and is limited by memory bandwidth. Time to first token reflects prefill; time per output token reflects decode.
 - The KV cache stores $\kappa = 2n_\ell n_{kv} d_h b_{kv}$ bytes per token of context. It avoids recomputation but fills memory, which limits how many sequences share an accelerator. Grouped-query attention, paging, and eviction all enlarge that limit.
 - In the walkthrough, one R1 call uses 53 accelerator-seconds and costs six times its metered price when served alone, but 2.1 accelerator-seconds and a quarter of its price when its decode is shared with 31 others. Batching is the main source of efficiency in serving, and it trades per-user speed for throughput.
 - Quantization reduces bytes per weight, speculative decoding produces several tokens per weight read without changing the output distribution, and mixture-of-experts models cut arithmetic per token while still requiring memory for all parameters.
 - Reasoning tokens are ordinary decode steps that are billed but often not shown. Retrieval adds input tokens to save larger costs elsewhere. Tool use and agent loops multiply model calls, and under context replay their tokens grow faster than their length.
 - Hardware and serving mechanisms set the cost of a token; application mechanisms set the number of tokens per outcome. [Table 2.3](#) maps each mechanism to its economic variable and to the chapter that models it.
-

Notes and Further Reading

Models and their arithmetic. The transformer is due to Vaswani et al. (2017). The $2P$ rule for forward-pass operations and the attention correction follow the accounting of Kaplan

et al. (2020), whose scaling laws, revised for compute-optimal training by Hoffmann et al. (2022), relate size and data to quality; Sardana et al. (2024) add inference demand to the training objective. Sampling and its failure modes are discussed by Holtzman et al. (2020). Tokenization by byte-pair encoding is due to Sennrich et al. (2016), and SentencePiece (Kudo and Richardson, 2018) is the common toolkit; Petrov et al. (2023) document the unequal cost of tokenization across languages, which Chapter 3 develops.

Serving systems. Pope et al. (2023) give the clearest analysis of why decode is memory-bound and how partitioning trades latency for cost, building on the roofline model of Williams et al. (2009). Continuous batching was introduced in Orca (Yu et al., 2022), and paged KV-cache memory in vLLM (Kwon et al., 2023). FlashAttention (Dao et al., 2022) reduces the memory traffic of attention itself. Grouped-query attention is due to Ainslie et al. (2023); cache eviction and attention-sink methods include H₂O (Zhang et al., 2023) and StreamingLLM (Xiao et al., 2024). Disaggregation of prefill and decode is studied in DistServe (Zhong et al., 2024), Splitwise (Patel et al., 2024), and Mooncake (Qin et al., 2025), and chunked prefill in Sarathi-Serve (Agrawal et al., 2024). Gholami et al. (2024) argue that memory is the binding constraint of modern AI systems. Book-length treatments do not yet exist; the surveys of Yuan et al. (2024), Zhou et al. (2024), and Miao et al. (2025) are the best entry points.

Cheaper tokens. Post-training quantization of large models to 8 bits is studied by Dettmers et al. (2022) and Xiao et al. (2023), and to 4 bits by Frantar et al. (2023) and Lin et al. (2024). Speculative decoding was introduced independently by Leviathan et al. (2023) and Chen et al. (2023), from whom Eq. (2.4) is taken; Medusa (Cai et al., 2024) and EAGLE (Li et al., 2024) replace the separate draft model, and Xia et al. (2024) survey the field. Sparse mixture-of-experts layers go back to Shazeer et al. (2017) and were simplified by Fedus et al. (2022); Mixtral (Jiang et al., 2024) and DeepSeek-V3 (DeepSeek-AI, 2024) are influential open-weight examples, and the gpt-oss model card (OpenAI, 2025a) documents an open-weight sparse model with its serving format. Distillation (Hinton et al., 2015) and low-rank fine-tuning (Hu et al., 2022) lower serving cost by other routes and reappear in Chapter 10.

Compound systems. Chain-of-thought prompting is due to Wei et al. (2022); reinforcement-learned reasoning is described by Guo et al. (2025), and the compute-optimal allocation of test-time compute by Snell et al. (2024). Retrieval-augmented generation is due to Lewis et al. (2020), tool use by self-supervised learning to Toolformer (Schick et al., 2023), and the interleaving of reasoning and acting to ReAct (Yao et al., 2023). Zaharia et al. (2024) argue that state-of-the-art results increasingly come from compound systems rather than single models. The categories of tokens that such systems generate, and their relation to computation, are discussed by Zhu (2026d). For evaluation, see HELM (Liang et al., 2023), the LLM-as-a-judge study of Zheng et al. (2023), and Chatbot Arena (Chiang et al., 2024).

Exercises

EXERCISE 2.1 (Analysis). A dense model has 70 billion parameters served at 8 bits (1 byte per parameter). Assume that each decode step reads every weight exactly once, that arithmetic is never the bottleneck, and that the accelerator moves 400 GB per second. (a) Estimate the bytes read per decode step at batch size one, ignoring the KV cache, and the resulting decode throughput. (b) Repeat with the KV cache of a 1,500-token context at $\kappa = 163,840$ bytes per token. (c) What would the throughput be if the weights were stored at 16 bits? *Hint:* Throughput at batch size one is bandwidth divided by bytes per step; (a) gives about 5.7 tokens per second.

EXERCISE 2.2 (Analysis). Using the operation counts of [Section 2.1](#), explain why doubling the prompt length doubles the KV-cache memory of a request and its parameter term $2PS$ but roughly quadruples the attention term. For the model of [Table 2.1](#), compute the ratio of total prefill operations at $S = 2,400$ to those at $S = 1,200$, and find the prompt length at which the attention term equals the parameter term. What does this imply about per-token pricing of very long prompts?

EXERCISE 2.3 (Analysis). Suppose a request in another language carries the same content as the R1 call of [Theorem 2.1](#) but tokenizes into three times as many tokens: 3,600 input and 900 output tokens. At batch size one on the walkthrough accelerator, compute the prefill time, the decode time, the final KV cache, the attention term relative to the English request, and the metered charge. Which of these ratios exceed three, and why? Compare with the measured tokenization premiums of [Exercise 3.7](#). *Hint:* Decode time grows slightly more than threefold because every step reads a longer cache.

EXERCISE 2.4 (Analysis). An agent answers one request by making five tool calls and then a final answer, six model calls in all. Every call re-sends the full context: a system prompt with tool descriptions (1,000 tokens), the user's request (200 tokens), and all earlier model outputs and tool results. Each model call outputs 100 tokens, and each tool result adds 400 tokens. (a) Tabulate the input and output tokens of each call. (b) Compute the total tokens and the charge at R1's prices. (c) Recompute the input tokens if each call received only the system prompt and the most recent tool result, and discuss what the agent loses. *Hint:* Call i has $1,200 + 500(i - 1)$ input tokens; the totals are 14,700 input and 600 output tokens. Compare with [Theorem 7.6](#).

EXERCISE 2.5 (Analysis). Quantize the walkthrough model's weights from 8 to 4 bits ($b = 0.5$) while keeping the KV cache at 8 bits. With 1,500 tokens of context per sequence, compute (a) the decode step time at $B = 1$, (b) the new memory limit on the batch size, and (c) the step time at $B = 284$. Explain why the speedup at $B = 284$ is much smaller than at $B = 1$, and what this implies for adding up the advertised savings of several techniques. *Hint:* At $B = 284$ the caches account for half the bytes of the original step.

EXERCISE 2.6 (Analysis). A prompt of 128,000 tokens is sent to the walkthrough model. Compute the parameter and attention terms of its prefill and the resulting prefill time at $\Pi = 5 \times 10^{14}$ operations per second, the size of its KV cache, and the number of such sequences whose caches fit beside the weights in 140 GB. Use your results to explain why providers may charge a higher per-token price for long prompts, and why the surcharge may apply to output tokens as well.

EXERCISE 2.7 (Analysis). Take the R1 call on the four-accelerator server of [Theorem 2.1](#) (step 8), with a TPOT of 48.2 milliseconds. (a) Compute the decode time of the 300-token answer. (b) A reasoning model generates 1,200 hidden reasoning tokens before the same answer. Compute the new decode time and the new charge at R1's prices, billing reasoning as output. (c) With a delay cost of c_d dollars per second of waiting, for what values of c_d does the latency cost of reasoning exceed its token cost?

EXERCISE 2.8 (Analysis). In speculative decoding, a draft proposes $\gamma = 4$ tokens, each accepted independently with probability $\varphi = 0.8$. (a) Use [Eq. \(2.4\)](#) to compute the expected tokens per pass of the large model. (b) Suppose each draft token costs 5% of a large-model pass and the verification pass costs the same as one ordinary decode step. Compute the cost per generated token relative to ordinary decoding. (c) How do the answers change at $\varphi = 0.6$? Why might the benefit shrink at large batch sizes, where decode steps are already closer to compute-bound? *Hint:* In (b), divide the cost of one round, $1 + 4 \times 0.05$, by the expected tokens per round.

EXERCISE 2.9 (Analysis). A hypothetical mixture-of-experts model has 400 billion parameters, of which 20 billion are active per token, served at 8 bits. (a) Compare its arithmetic per token with that of a dense 400-billion-parameter model and a dense 20-billion-parameter model. (b) How much memory must hold its weights, and could it be served on one accelerator of [Table 2.1](#)? (c) At batch size one, if only active weights are read, what is the decode throughput at 400 GB/s? (d) Explain why, at large batch sizes, the bytes read per step approach those of the full model, and what this means for the economics of sparse models.

EXERCISE 2.10 (Design). Redesign the R1 request of [Theorem 2.2](#). Suppose intent classification and the policy check can be folded into the other calls, removing calls 1 and 4 and adding 400 input tokens to the drafting call (which then has 2,200 input and 750 output tokens). Compute the new charge per request, the charge per day at 50,000 requests, and the model latency on the same server. Then list what information you would need before adopting the redesign, and explain which of the other calls could run in parallel in the original design. *Hint:* The charge falls by about 30%; [Chapter 9](#) explains why that is not the end of the analysis.

TABLE 2.3. From mechanism to economic variable. Hardware and serving mechanisms (upper block) set what a token costs to produce and are controlled by whoever operates the capacity. Application mechanisms (lower block) set how many tokens an outcome requires and are controlled by whoever writes the program.

Mechanism (this chapter)	Economic variable	Modeled in
About $2P$ operations per token	Per-token cost proportional to active model size	Chapter 4 ; Chapter 10
Compute-bound prefill, memory-bound decode	Output–input cost ratio ρ ; price ratio $p_{\text{out}}/p_{\text{in}}$	Proposition 3.6 ; Chapter 13
Batching (effective batch size B)	Cost floor per token; throughput–latency frontier	Theorem 4.3 ; Section 4.3
KV cache κS per sequence	Memory limit on B ; long-context surcharges	Theorem 4.3 ; Chapter 6
TTFT and TPOT	Delay cost c_d ; latency service tiers; goodput	Section 3.4 ; Theorem 4.7 ; Chapter 13
Fleet utilization u	Utilization-adjusted unit cost $\bar{c}(u)$	Theorem 4.5 ; Chapter 5
Quantization, speculative decoding, MoE	Shifts of the cost floor; quality risk of cheaper serving	Table 4.2 ; Chapter 9
Prefill/decode disaggregation	Separate provisioning of compute and memory	Theorem 4.9 ; Chapter 5
Tokenizer	Tokens per task; non-comparability of unit prices	Section 3.2
Stochastic sampling	Quality as a distribution; quality floor $q_{\min}$	Section 9.3 ; Chapter 10
Prompt and context assembly, retrieval	Input tokens N_{in} ; context replay	Chapter 6
Prefix and prompt caching	Cache hit rate h ; cached-input price	Section 6.3 ; Definition 3.4
Reasoning tokens	Hidden output; optimal reasoning budget	Theorem 7.2 ; Section 3.6
Tool use and agent loops	Call multiplier k ; replay growth	Theorem 7.6 ; Chapter 8
Delegation to sub-agents	Branching factor m ; criticality at $m = 1$	Theorem 7.13 ; Chapter 18
Compound workflows	Stage budgets; attribution of value	Chapter 11 ; Chapter 12

The Token as a Unit of Account

Tokens are the unit on the invoice, but they are a poor unit of economic account. This chapter examines how tokens are constructed and why a token from one model is not comparable to a token from another. It shows that a single, fixed exchange rate between two tokenizers must misstate some texts by a margin that can be computed, and it derives the premium on output tokens from the balance between an accelerator's compute and its memory bandwidth. It then turns to what token counts miss (latency, quality, hidden reasoning, and adjacent costs) and to how token metrics invite strategic response. Because much of a modern bill consists of tokens the customer never sees, the chapter asks how such a bill can be verified and bounds what sampling audits can detect. It closes with what a quality-adjusted price index for inference would require. Real data from the invoice-reconciliation agent (R2), in which hidden reasoning accounts for 72% of the bill, run through the chapter.

LEARNING OBJECTIVES

- Explain how tokenizers turn text into billable units, and why tokens are not comparable across tokenizers, languages, models, and model versions.
- Compute the best achievable exchange rate between two tokenizers and the error that any fixed exchange rate must incur.
- Derive the input–output cost ratio from the machine balance of an accelerator, and explain how batch size, precision, and context length move it.
- Decompose a bill into input, cached, visible output, and reasoning tokens, and convert token charges into cost per accepted outcome.
- Identify the quantities that token counts miss, and design metric pairs that resist strategic response.
- Evaluate audit methods for hidden tokens and compute how many requests an audit must inspect to detect overbilling.
- Construct Laspeyres, Paasche, Fisher, and hedonic price indexes, and explain what a

price index for inference should hold fixed.

A provider’s invoice for a month of inference looks like a utility bill: a few lines of quantities, each multiplied by a unit price. Electricity is billed in kilowatt-hours, and a kilowatt-hour is the same physical quantity whether it lights a hospital or a billboard. Tokens invite the same reading, and the reading is wrong in instructive ways. A token is defined by a particular tokenizer, so its size varies from one model to another, from one language to another, and from one version of a model to the next. Its cost to the provider depends on whether it was read or written, on how many other requests shared the hardware at that moment, and on how long the context was. Its value to the customer depends on whether the answer it helped produce was any good. And a growing share of the tokens on a bill are tokens the customer never sees.

This chapter treats the token as what it is: a *unit of account*, the common denominator in which a buyer and a seller agree to record a transaction. [Section 3.1](#) explains how tokens are made. [Section 3.2](#) shows that they do not compare across tokenizers and quantifies the damage. [Section 3.3](#) decomposes the bill and explains why output tokens cost more than input tokens. [Section 3.4](#) lists what token counts leave out, [Section 3.5](#) what happens when token metrics become targets, and [Section 3.6](#) how a customer can verify a bill whose largest line is invisible. [Section 3.7](#) asks what it would mean to say that inference has become cheaper.

3.1 The Construction of Tokens

3.1.1 From Strings to Integers

A language model does not read characters. It reads a sequence of integers, each an index into a fixed vocabulary of a few tens or hundreds of thousands of entries. The program that converts text into that sequence is the *tokenizer*. Consider the English sentence “Please reconcile the attached invoice.” A byte-level tokenizer first splits it into pieces at spaces and punctuation (Please, _reconcile, _the, ...), then looks up each piece in its vocabulary. Common words are single entries. A rarer word is broken into fragments that are in the vocabulary, such as _recon and cile. Each fragment is one token, and the invoice counts them.

DEFINITION 3.1 (Tokenizer and token count). Let $\mathcal{X}$ be a set of inputs (for text, the finite strings over an alphabet) and $\mathcal{V}$ a finite vocabulary. A *tokenizer* is a map $\tau : \mathcal{X} \rightarrow \mathcal{V}^*$ from inputs to finite sequences of vocabulary items. The *token count* of an input x under τ is $N_\tau(x) = |\tau(x)|$, the length of that sequence.

The dominant construction is *byte-pair encoding* (Sennrich et al., 2016). Training starts with a vocabulary of single bytes and a large corpus. It counts which adjacent pair of symbols occurs most often, adds the merged pair to the vocabulary, and repeats until the

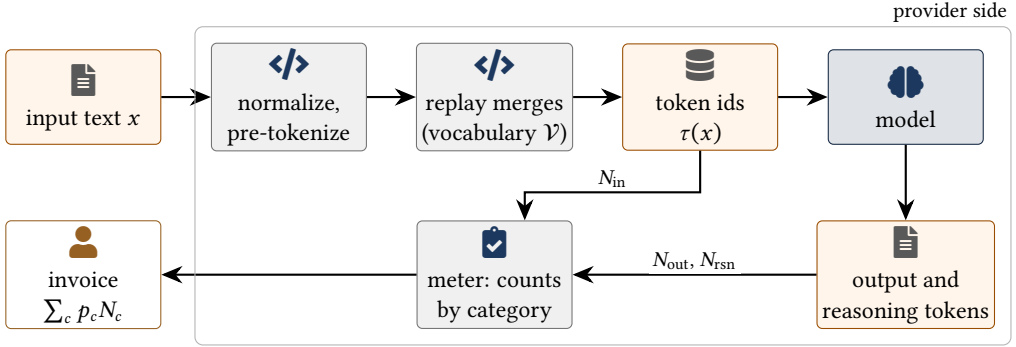


FIGURE 3.1. From text to invoice. Every box inside the frame is chosen and operated by the provider: the tokenizer that defines the unit, the model that generates output and hidden reasoning, and the meter that counts tokens by category. The customer supplies the text and receives the invoice. Section 3.2 is about the left half of the frame, Section 3.6 about the right half.

vocabulary reaches its target size. The ordered list of merges is the tokenizer. To encode new text, the merges are replayed in order. Unigram language-model segmentation (Kudo, 2018) reaches a similar vocabulary from the opposite direction, pruning a large candidate set, and both are commonly implemented through a shared toolkit (Kudo and Richardson, 2018). Chapter 2 describes how the resulting integers are consumed by the model; here what matters is the economics of the construction.

Two properties follow directly. First, a tokenizer is *learned from a corpus*, so it differs across model families, and it encodes the text that was common in its training data more compactly than text that was rare. Second, a token is a *compression unit*, not a linguistic one. It corresponds to no word, morpheme, or syllable with an independent definition; it is whatever the compressor found convenient. Everything in Section 3.2 is a consequence of these two properties.

3.1.2 Beyond Text

The same logic extends to other modalities, with even less connection to any natural unit. An image is typically cut into fixed-size patches, each mapped to one or more tokens, so its token count depends on resolution and on the provider's resizing rules rather than on its content. Audio is sliced into frames of fixed duration. Tool calls, function schemas, and structured outputs are serialized into text and tokenized like any other text, which is why a long tool description is billed on every call that carries it. A multimodal request therefore carries several token counts produced by several unrelated conversion rules and then added together as if they were one quantity.

3.1.3 Why Tokens Became the Billing Unit

Metering in tokens is a sensible commercial choice. A token count is cheap to compute, deterministic for a given tokenizer and input, and monotone in the work done: a longer

TABLE 3.1. One sentence in six languages: length in characters and bytes, and token counts under two tokenizers, also relative to English (script: `tokenizer_counts.py`).

Language	Chars	Bytes	Tokens		Relative to English		Ratio
			GPT-2	Qwen2.5	GPT-2	Qwen2.5	
English	85	85	15	15	1.00	1.00	1.000
Spanish	99	99	32	27	2.13	1.80	1.185
German	110	113	41	34	2.73	2.27	1.206
Greek	114	213	131	97	8.73	6.47	1.351
Hindi	63	165	95	57	6.33	3.80	1.667
Chinese	24	72	53	15	3.53	1.00	3.533

The last column is $r(s) = N_{\text{GPT-2}}(s)/N_{\text{Qwen2.5}}(s)$.

prompt or a longer answer costs the provider more. It lets a provider price a heterogeneous stream of requests with two or three numbers. The trouble begins when an administrative convenience is read as a physical measure of computation, as a measure of cost, or as a measure of value. It is none of these, and the rest of the chapter shows why.

💡 KEY IDEA

A token is a unit of account, not a unit of measurement. Like a currency, it is defined by its issuer, it can change in value when the issuer changes the tokenizer, and it does not convert at a fixed rate into the units of other issuers.

3.2 Non-Comparability across Models

3.2.1 One Sentence, Two Tokenizers, Six Languages

The quickest way to see the problem is to tokenize the same content several ways.

EXAMPLE 3.2 (Token counts across tokenizers and languages). The sentence “Please reconcile the attached invoice and report the total amount due, including tax” was translated into Spanish, German, Greek, Hindi, and Chinese, and each version was tokenized with two public byte-level BPE tokenizers: GPT-2’s, with a vocabulary of about 50,000 entries trained mostly on English web text (Radford et al., 2019), and Qwen2.5’s, with about 151,000 entries trained on a multilingual corpus. Table 3.1 reports the counts and Fig. 3.2 plots them.

Three things stand out. Within one tokenizer, the same request costs very different amounts in different languages: under GPT-2 the Greek sentence uses 8.7 times as many tokens as the English one, although it has only 34% more characters. Across tokenizers, the gap between languages narrows but does not close: Qwen2.5 still charges the Greek

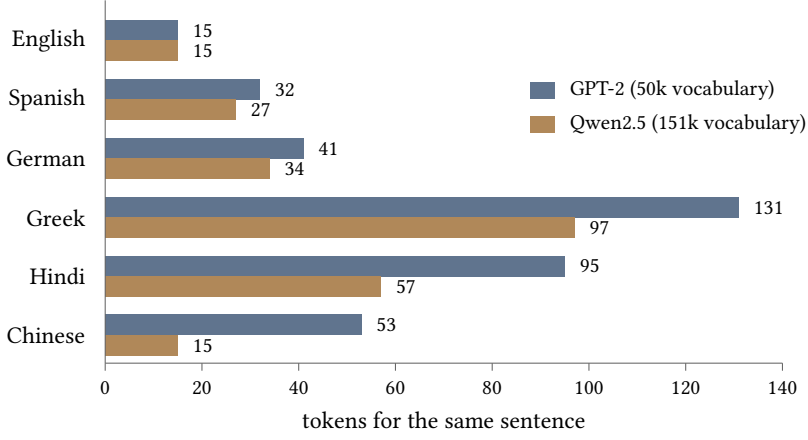


FIGURE 3.2. Token counts for one sentence in six languages (Table 3.1). A uniform price per token is a non-uniform price per unit of meaning, and the pattern of non-uniformity depends on the tokenizer. English is cheapest under both; the ranking of the other languages changes between tokenizers (Chinese is the third most expensive language under GPT-2 and ties English under Qwen2.5).

sentence 6.5 times the English count. And the ratio between the two tokenizers is not a constant. It is exactly 1 for English and 3.5 for Chinese, so no single conversion factor turns “GPT-2 tokens” into “Qwen tokens.”

These are single sentences, and translations of equal meaning have different natural lengths, so the example illustrates rather than measures. Systematic studies confirm the pattern at scale. [Petrov et al. \(2023\)](#) find that the same content can differ in token length by up to a factor of fifteen across languages under one tokenizer, and [Ahia et al. \(2023\)](#) show that commercial per-token prices therefore charge speakers of some languages several times more for equivalent requests, while also giving them less usable context. [Rust et al. \(2021\)](#) show that the quality of a tokenizer for a language affects the model’s performance in that language, so the speakers who pay more per unit of meaning can also receive less capable service.

3.2.2 No Common Unit

In the language of measurement theory, token count is a ratio scale *within* a tokenizer: forty tokens is twice twenty tokens ([Stevens, 1946](#); [Krantz et al., 1971](#)). *Across* tokenizers there is no shared unit. The following proposition makes this precise by asking how well any fixed exchange rate could do.

PROPOSITION 3.3 (No fixed exchange rate between tokenizers). *Let τ_A and τ_B be tokenizers and S a finite set of inputs with $N_A(s), N_B(s) > 0$ for all $s \in S$. Let $r(s) = N_B(s)/N_A(s)$, $r_{\min} = \min_s r(s)$, and $r_{\max} = \max_s r(s)$.*

- (i) For an exchange rate $\xi > 0$, predict $N_B(s)$ by $\xi N_A(s)$. The smallest achievable worst-case relative error is

$$\min_{\xi > 0} \max_{s \in S} \frac{|\xi N_A(s) - N_B(s)|}{N_B(s)} = \frac{r_{\max} - r_{\min}}{r_{\max} + r_{\min}},$$

attained uniquely at $\xi^* = 2r_{\min}r_{\max}/(r_{\min} + r_{\max})$.

- (ii) Let the two providers charge per-token prices $p_A, p_B > 0$. The ratio of their charges, $p_A N_A(s)/(p_B N_B(s))$, is the same for every $s \in S$ if and only if $r_{\min} = r_{\max}$. If $r_{\min} < p_A/p_B < r_{\max}$, provider A is strictly cheaper on some input and strictly more expensive on another.

Proof. (i) The relative error on s is $|\xi N_A(s) - N_B(s)|/N_B(s) = |\xi/r(s) - 1|$. For fixed ξ , ξ/r is monotone in r , so the maximum over S is attained at $r_{\min}$ or $r_{\max}$ and equals $g(\xi) = \max\{|\xi/r_{\min} - 1|, |\xi/r_{\max} - 1|\}$. If $\xi < r_{\min}$, then $g(\xi) \geq 1 - \xi/r_{\max} > 1 - r_{\min}/r_{\max}$; if $\xi > r_{\max}$, then $g(\xi) \geq \xi/r_{\min} - 1 > r_{\max}/r_{\min} - 1$. On $[r_{\min}, r_{\max}]$, $g(\xi) = \max\{\xi/r_{\min} - 1, 1 - \xi/r_{\max}\}$, the maximum of an increasing and a decreasing linear function, minimized uniquely where they are equal, at ξ^* , with value $\xi^*/r_{\min} - 1 = (r_{\max} - r_{\min})/(r_{\max} + r_{\min})$. This value is no larger than $1 - r_{\min}/r_{\max} = (r_{\max} - r_{\min})/r_{\max}$ or $r_{\max}/r_{\min} - 1$, so ξ^* is the global minimizer. (ii) The charge ratio on s is $(p_A/p_B)/r(s)$, which is constant on S exactly when r is. Provider A is strictly cheaper on s if and only if $p_A/p_B < r(s)$. If $r_{\min} < p_A/p_B < r_{\max}$, this holds at an input attaining $r_{\max}$ and fails strictly at an input attaining $r_{\min}$. ■

Applied to Table 3.1 with $A = \text{Qwen2.5}$ and $B = \text{GPT-2}$, $r_{\min} = 1.000$ (English) and $r_{\max} = 3.533$ (Chinese). The best single exchange rate is $\xi^* = 1.559$ GPT-2 tokens per Qwen token, and even that rate misstates some language's count by $(3.533 - 1)/(3.533 + 1) = 55.9\%$. Part (ii) says more: if the Qwen-based service charged anywhere between 1 and 3.5 times the GPT-2-based service's per-token price, it would be cheaper for the Chinese request and dearer for the English one. "Which provider is cheaper?" has no answer in tokens; it has an answer only for a stated workload.

Three corollaries are violated routinely in practice. *Cross-provider comparisons must be made in currency per task*, for a stated task mix, never in tokens. *Token budgets are not portable*: a context limit, a rate limit, or a spending cap written in tokens is silently redenominated when the underlying model or tokenizer changes. And *per-token pricing is not neutral across users*: by Proposition 3.3(ii) applied within one tokenizer to inputs of equal meaning, a uniform price per token is a price per unit of meaning that varies with language, script, and domain.

Tokenizers also create variation within a single request. The same output string can be produced by the model as different token sequences, and a per-token bill then depends on which sequence was sampled. Chatzi et al. (2025) document such tokenization multiplicity in proprietary and open-weight models and propose constraining generation to the canonical tokenization so that the price of a string is well defined.

3.2.3 Tokenizers Change under a Fixed Price

A provider can change the unit without changing the price list, which is the monetary equivalent of redefining the length of a meter. When a successor model ships with a new tokenizer, a customer’s per-token prices can stay the same while the bill for identical work moves.

IN PRACTICE (AS OF SEPTEMBER 2026)

Anthropic’s announcement of Claude Opus 4.7 (16 April 2026) states that the model uses an updated tokenizer and that the same input can map to roughly 1.0 to 1.35 times as many tokens, depending on content type (Anthropic, 2026c). If per-token prices are unchanged across such a transition, a customer whose workload sits at the top of that range pays up to 35% more for identical requests. The announcement presents the change as a tradeoff for better text processing. Whatever its merits, a per-token price series would record no change at all, which is why Section 3.7 insists that price indexes be computed per task.

3.3 Token Count, Cost, and the Input–Output Ratio

3.3.1 Categories of Tokens: Input, Cached, Output, Reasoning

Even for one model and one tokenizer, a bill is not a single quantity. Providers price several categories of tokens separately, because the categories cost different amounts to serve.

DEFINITION 3.4 (Metered bill). For a request or a billing period, let N_c be the number of tokens in category c and p_c its price per token. The categories used in this book are uncached input ($c = \text{in}$), input read from a prompt cache (cr), input written to a cache (cw), visible output (out), and reasoning output (rsn). The *metered bill* is

$$C_{\text{model}} = p_{\text{in}}N_{\text{in}} + p_{\text{cr}}N_{\text{cr}} + p_{\text{cw}}N_{\text{cw}} + p_{\text{out}}N_{\text{out}} + p_{\text{rsn}}N_{\text{rsn}}. \quad (3.1)$$

Table 3.2 summarizes what distinguishes the categories. *Input tokens* are processed in the prefill phase, many at once. *Cached input tokens* are a prefix that the provider has already processed for an earlier request and kept in memory, so the prefill work is skipped (Gim et al., 2024; Zheng et al., 2024); they are usually priced well below uncached input, and writing a prefix to the cache may carry a surcharge. *Output tokens* are generated one at a time in the decode phase and cost more to produce, for reasons derived in the next subsection. *Reasoning tokens* are output tokens that the model generates before or between its visible answers, in the manner of chain-of-thought (Wei et al., 2022), but that the provider does not return to the customer, or returns only in summary or encrypted form. They are produced by the same decode process as visible output, and

TABLE 3.2. Categories of tokens on a bill. The last column is what the customer can check directly, before any of the audit methods of Section 3.6.

Category	Served in	What drives its cost	Customer can check
Input	prefill	compute; attention over the context	count, by re-tokenizing
Cached input	(skipped prefill)	memory to hold the prefix	which prefixes were eligible
Visible output	decode	memory bandwidth; batch size	count, by re-tokenizing
Reasoning	decode	same as output, often longer	reported count only

providers commonly bill them at the output price, $p_{\text{rsn}} = p_{\text{out}}$ (see the box below); what distinguishes them is observability, not cost.

The existence of a cached-input price is itself evidence that tokens are not a homogeneous good. A homogeneous good does not sell for two prices from the same seller at the same moment. The seller is pricing the work behind the token, and the token is only the label.

✂ IN PRACTICE (AS OF SEPTEMBER 2026)

- *OpenAI*. Reasoning tokens are not visible through the API but occupy context and are billed as output tokens; their number is reported in the usage object under `output_tokens_details`. Reasoning can be returned as a summary or, in stateless use, as encrypted content to be passed back on later calls ([OpenAI, 2026d](#)). Cached input is discounted by up to 90% ([OpenAI, 2026a](#)).
- *Anthropic*. Thinking tokens are billed as output tokens and count toward `max_tokens`; on Claude Opus 5 thinking is on by default, and thinking blocks are returned with an empty text field under the default display setting ([Anthropic, 2026d](#)). The usage field `output_tokens_details.thinking_tokens` reports how many billed output tokens were reasoning ([Anthropic, 2026e](#)). Cache reads cost 0.1 times the base input price on most models, and cache writes 1.25 times (five-minute cache) or 2 times (one-hour cache) ([Anthropic, 2026a](#)).
- *Google*. For Gemini models with thinking enabled, response pricing is the sum of output and thinking tokens, and pricing is based on the full thought tokens even though only a summary is returned ([Google, 2026](#)).

In all three cases the customer receives a *count* of reasoning tokens reported by the provider, not the tokens themselves.

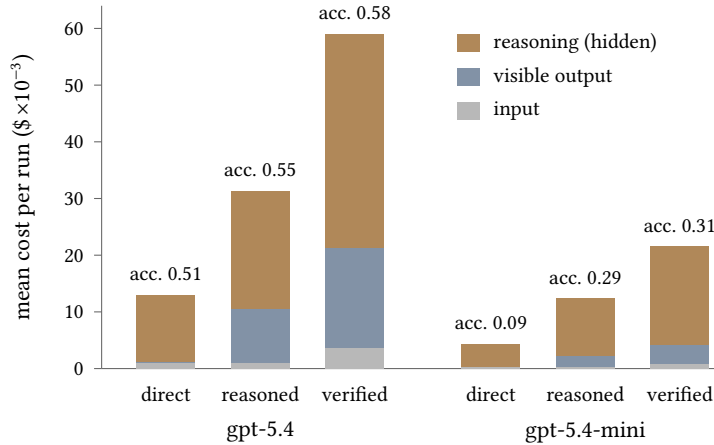


FIGURE 3.3. Running example R2 (real data, 80 runs per bar): mean cost per run by token category at the experiment’s calibration prices, with accuracy above each bar. Hidden reasoning (top segments) is the largest category in every cell and 71.9% of the total. Computed by `tools/numbers/token/chapter_numbers.py`.

RUNNING EXAMPLE R2: WHERE THE INVOICE-RECONCILIATION BILL GOES

Running example R2 (Section 1.4) is an agent that reconciles vendor invoices: it applies discounts, currency conversion, proration, credits, and tax in the right order to reach an exact amount payable. It is calibrated on a real experiment (Zhu, 2026c): 40 tasks, each run twice, under three strategies (*direct*: one call at low reasoning effort; *reasoned*: one call at medium effort with instructions to reason step by step; *verified*: a reasoned call followed by a verification call) on two models, gpt-5.4 and gpt-5.4-mini, for 480 runs in all. At the experiment’s calibration prices (\$2.50 and \$15.00 per million input and output tokens for gpt-5.4, \$0.75 and \$4.50 for gpt-5.4-mini), the mean cost of a run decomposes as in Fig. 3.3.

Across all 480 runs, reasoning tokens are 61.6% of billed tokens and 71.9% of billed cost. Their cost share exceeds their token share because they are billed at the output price, six times the input price for both models. The share is highest where the visible answer is shortest: under the direct strategy, where the model returns little more than a number, reasoning is 91% of gpt-5.4’s cost and 92% of gpt-5.4-mini’s. In every cell, most of what the customer pays for is text the customer never sees. Two further facts bear on Section 3.2. The larger model uses *fewer* billed tokens than the mini model under every strategy (1,193 against 1,294 per run for direct, 2,417 against 3,072 for reasoned, 5,152 against 5,802 for verified), so “tokens per task” is a property of the model, not just of the task. And the variation is large: under the reasoned strategy the standard deviation of gpt-5.4’s reasoning tokens is 38% of their mean, across tasks that differ only in their numbers.

3.3.2 Machine Balance and the Price Ratio

Why do output tokens cost several times as much as input tokens? The premium is often presented as a pricing convention. It is better understood as a property of the hardware, and deriving it shows which engineering choices move it. Chapter 2 describes the two phases of serving; the minimum needed here is this. In *prefill*, all S prompt tokens pass through the model together as large matrix multiplications, and the accelerator’s arithmetic is the bottleneck. In *decode*, each step produces one new token for each of the B sequences in a batch, and every step must stream the model’s weights and the sequences’ key–value (KV) caches from memory; memory bandwidth is the bottleneck (Pope et al., 2023; Kwon et al., 2023). The ratio of an accelerator’s arithmetic speed to its memory speed, called its *machine balance* in the roofline literature (Williams et al., 2009), then sets the price ratio.

ASSUMPTION 3.5 (Stylized serving model). A dense transformer has P parameters stored at b bytes each. It runs on an accelerator with effective compute throughput Π operations per second and memory bandwidth Ω bytes per second. (a) Prefill is compute-bound: each input token costs $2P$ operations, and attention compute is negligible. (b) Decode is bandwidth-bound: each step reads the weights once (bP bytes) and, for each of the B sequences in the batch, its KV cache of κS bytes, where κ is the KV-cache size per token and S the sequence’s context length. Each step emits one token per sequence. (c) Cost is proportional to accelerator time.

PROPOSITION 3.6 (Machine balance sets the output–input cost ratio). *Under Assumption 3.5, the ratio of the accelerator time per output token to that per input token is*

$$\rho = \frac{t_{\text{out}}}{t_{\text{in}}} = \frac{\Pi}{\Omega} \left(\frac{b}{2B} + \frac{\kappa S}{2P} \right). \quad (3.2)$$

The ratio decreases in the batch size B and increases in the machine balance Π/Ω , the precision b , and the context length S . As $B \rightarrow \infty$ it falls to the floor $\rho_\infty = (\Pi/\Omega) \kappa S / (2P)$. When the KV term is negligible, $\rho \approx (b/2B)(\Pi/\Omega)$.

Proof. By (a), an input token takes $t_{\text{in}} = 2P/\Pi$ seconds. By (b), a decode step moves $bP + B\kappa S$ bytes and takes $(bP + B\kappa S)/\Omega$ seconds; it yields B tokens, so $t_{\text{out}} = (bP + B\kappa S)/(B\Omega)$. Dividing, $\rho = \Pi(bP + B\kappa S)/(2PB\Omega)$, which is Eq. (3.2). The monotonicity claims and the limit are immediate. ■

EXAMPLE 3.7 (The ratio in numbers). Take hypothetical but representative values: machine balance $\Pi/\Omega = 500$ operations per byte, 16-bit weights ($b = 2$), and a 70-billion-parameter model ($P = 7 \times 10^{10}$) whose KV cache, with 80 layers, 8 key–value heads of dimension 128, and 16-bit storage, takes $\kappa = 2 \times 80 \times 8 \times 128 \times 2 = 327,680$ bytes per token. Ignoring the KV term, $\rho = 500/B$: 15.6 at $B = 32$, 3.9 at $B = 128$. With an average context of $S = 4,000$ tokens the floor is $\rho_\infty = 500 \times 327,680 \times 4,000 / (1.4 \times 10^{11}) = 4.68$,

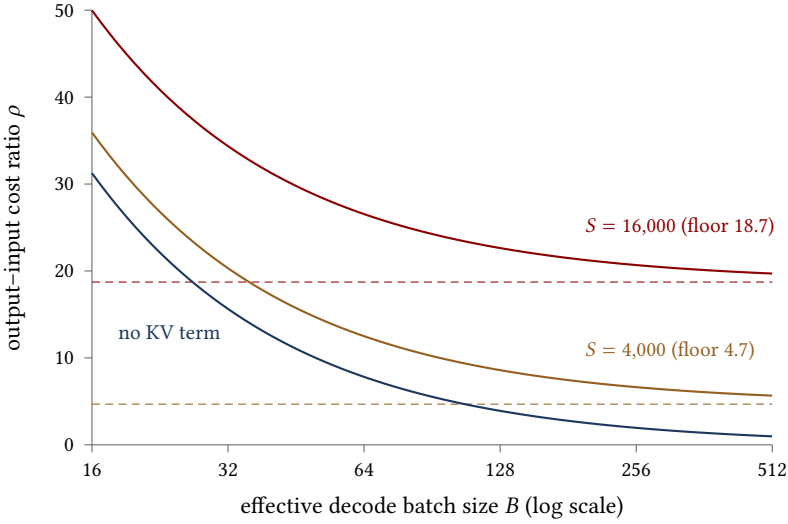


FIGURE 3.4. Proposition 3.6 with the parameters of Example 3.7 ($\Pi/\Omega = 500$, $b = 2$, $P = 7 \times 10^{10}$, $\kappa = 327,680$ bytes per token; all hypothetical). Larger batches amortize one read of the weights over more output tokens and shrink the premium on output, but the KV cache of long contexts sets a floor (dashed) that batching cannot remove.

and $\rho = 12.5$ at $B = 64$ and 8.6 at $B = 128$. The batch term equals the floor at $B \approx 107$. At $S = 16,000$ the floor rises to 18.7 , and batching can no longer bring the ratio below it.

The proposition organizes several facts that are otherwise puzzling. The premium on output is a hardware property: posted output–input price ratios fall in the range that Eq. (3.2) produces for plausible parameters (see the box below). A provider that raises its effective batch size through continuous batching (Yu et al., 2022) and paged KV memory (Kwon et al., 2023) changes the *relative* cost of the two things it sells. Weight quantization (Dettmers et al., 2022; Frantar et al., 2023; Lin et al., 2024) lowers b and so attacks decode cost specifically. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) yields several accepted tokens per read of the weights, which acts like a larger B . And long contexts raise the output premium through the floor, which is one reason why agentic workloads with long, growing contexts are expensive per output token. Chapter 4 adds utilization and memory capacity, which limits how large B can be.

✂ IN PRACTICE (AS OF SEPTEMBER 2026)

Posted prices per million tokens (input / output): OpenAI GPT-5.5 \$5.00 / \$30.00, GPT-5.4 \$2.50 / \$15.00, GPT-5.4-mini \$0.75 / \$4.50, with cached input at one tenth of the input price (\$0.50, \$0.25, \$0.075) (OpenAI, 2026c); Anthropic Claude Opus 5 \$5 / \$25 (Anthropic, 2026d). The output–input ratios are 6 and 5. The GPT-5.4 prices coincide with the calibration prices of running example R2.

3.3.3 From Tokens to Cost per Accepted Outcome

Token prices convert into cost per request, but a customer buys answers, not requests. If each attempt succeeds with probability s and failed attempts are detected and retried independently, the expected number of attempts per accepted answer is $1/s$.

DEFINITION 3.8 (Cost per accepted outcome). For a workload with mean cost per attempt C and acceptance probability $s > 0$ per attempt, the *cost per accepted outcome* is C/s . More generally, over a period with total cost C_{tot} and Y accepted outcomes, it is C_{tot}/Y .

EXAMPLE 3.9 (R2: the cheaper model is the more expensive one). Under the direct strategy in R2, a gpt-5.4 run costs \$0.01294 on average and is correct with probability 0.5125; a gpt-5.4-mini run costs \$0.00433 and is correct with probability 0.0875. The mini model is 3.0 times cheaper per run, and its per-token prices are 3.3 times lower. Per correct answer, gpt-5.4 costs $0.01294/0.5125 = \$0.0252$ and gpt-5.4-mini costs $0.00433/0.0875 = \$0.0495$, almost twice as much. In tokens, the mini model is also *less* efficient (1,294 against 1,193 per run). None of these rankings can be read from a price list.

The general point is that each step from a token to an outcome multiplies by a quantity the previous step does not record: tokens per call, calls per request, price per token, adjacent cost, and acceptance. [Chapter 8](#) develops this chain as an algebra; [Chapter 9](#) treats the harder question of what an accepted outcome is worth.

KEY IDEA

Compare models, providers, and designs in currency per accepted outcome for a stated workload. Token counts and per-token prices are inputs to that comparison, never substitutes for it.

3.4 What Token Counts Miss

3.4.1 Latency and Its Price

A token count records how much text moved, not how long it took. For an interactive request the delay is part of the cost to the customer: a support agent waiting for a draft, a user abandoning a chat, a trading system missing a window. Two quantities describe the delay: the *time to first token*, dominated by queueing and prefill, and the *time per output token*, set by decode speed. For G generated tokens (visible plus reasoning) at decode speed v tokens per second, the latency of a request is roughly $L = t_{\text{ft}} + G/v$, and with a delay cost of c_d per request per second the customer's full cost per request is $C_{\text{model}} + c_d L$.

EXAMPLE 3.10 (Latency in the support assistant). Take one model call of running example R1: 1,200 input and 300 visible output tokens at \$2 and \$8 per million, so the metered charge is \$0.0048. Option A serves it with no reasoning at 80 tokens per second after a 0.5-second first token, for a latency of $0.5 + 300/80 = 4.25$ seconds. Option B adds 1,200 reasoning tokens and runs at 60 tokens per second after a 1-second first token; its charge rises to \$0.0144 and its latency to $1 + 1,500/60 = 26$ seconds. With a hypothetical delay cost of $c_d = \$0.001$ per second (a customer's time valued at \$3.60 per hour), A's full cost is $0.0048 + 0.00425 = \$0.00905$ and B's is $0.0144 + 0.026 = \$0.0404$. Delay is 47% of A's full cost and 64% of B's, and none of it is on the invoice. (The speeds, the first-token times, the reasoning length, and c_d are hypothetical.)

Latency connects to the rest of the chapter through reasoning. In R2, gpt-5.4 generates 796 output tokens per run under the direct strategy and 2,018 under the reasoned strategy; at a hypothetical 60 tokens per second that is the difference between 13 and 34 seconds. Hidden reasoning is paid for twice, once on the invoice and once in waiting time. Providers also sell latency explicitly, through priority tiers and faster serving modes at higher per-token prices, and Chapter 4 derives the throughput–latency frontier behind such offers; Chapter 13 treats them as price discrimination.

IN PRACTICE (AS OF SEPTEMBER 2026)

Anthropic offers a research-preview “fast mode” for Claude Opus 5, which provides higher output speed at \$10 per million input tokens and \$50 per million output tokens, twice the model's standard rates (Anthropic, 2026d). The same tokens carry two prices that differ only in how quickly they are delivered.

3.4.2 Quality

A token count is silent about whether the answer was right. Cost per accepted outcome (Definition 3.8) repairs this only when acceptance is observable and its value is known. When quality is priced, a large reduction in tokens can be a loss.

RUNNING EXAMPLE R1: AN 89% TOKEN SAVING THAT LOSES MONEY

Running example R1 is a customer-support assistant handling 50,000 requests a day (all parameters hypothetical, Section 1.4). The baseline design makes $k = 4$ model calls per request with 1,200 input and 300 output tokens each; a redesign makes one call with 600 input and 200 output tokens, and a cache answers a quarter of requests without a model charge ($h = 0.25$). At \$2 and \$8 per million input and output tokens, the model charge falls from \$960 to \$105 a day (−89.1%), and tokens fall by 90%. Adjacent costs rise from \$300 to \$420, so total cost falls from \$1,260 to \$525, a saving of \$735 a day.

But the first-contact resolution rate slips from 0.78 to 0.76, and each unresolved request escalates to a human at \$4. The extra escalations cost $50,000 \times 0.02 \times \$4 = \$4,000$ a day, and the redesign loses \$3,265 a day. It would break even only if resolution fell by no more than $735 / (50,000 \times 4) = 0.37$ percentage points, or if an escalation cost no more than \$0.735. An evaluation that cannot detect a change of a few tenths of a point cannot tell whether the redesign should ship.

3.4.3 Hidden Reasoning and Adjacent Costs

Hidden reasoning changes the nature of the good. Visible output can be inspected: the customer can read it, re-tokenize it, and count. Reasoning tokens can be neither read nor counted by the customer; only their reported number arrives. For the customer, the *quantity* of reasoning becomes a credence attribute, something paid for but not verifiable even after consumption, like the number of hours a mechanic reports (Dulleck and Kerschbamer, 2006). The R2 data show why this matters economically. Reasoning is the largest line on the bill (Fig. 3.3), its standard deviation ranges from 24% to 43% of its mean across cells, for tasks that differ only in their numbers, and for gpt-5.4 it responds strongly to settings (the effort level, the instructions) whose effect on accuracy is small. A customer cannot tell from the invoice whether a month's increase in reasoning reflects harder tasks, a changed default, or overbilling. Section 3.6 takes up the verification problem; Chapter 14 examines the incentive problem it creates for contract design.

Finally, a token-denominated view omits everything outside the metered call: retrieval indexes and vector stores, orchestration, safety filters, evaluation harnesses (which consume inference of their own), observability, human review, and the engineering labor that keeps the system running. In R1 these adjacent costs are \$300 to \$420 a day against a model charge of \$105 to \$960, and in the redesign they are four times the model charge. Table 3.3 lists the common token-derived metrics, what each is good for, and what each cannot see. Moving down the table, decision relevance rises and measurement reliability falls.

3.5 Metrics under Strategic Response

Once a metric is used to allocate budgets or judge teams, it stops being a passive measurement. Goodhart's observation that a regularity used for control tends to break down (Goodhart, 1975), and Campbell's that a quantitative indicator used for decisions becomes subject to corrupting pressure (Campbell, 1979), apply with unusual force to token metrics, because the routes to gaming them are short. Manheim and Garrabrant (2018) distinguish four mechanisms by which optimizing a proxy fails: regressional (the proxy is a noisy measure of the goal), extremal (the relation between proxy and goal breaks at extreme values), causal (intervening on the proxy does not move the goal), and adversarial (other agents exploit the proxy). All four appear with tokens.

TABLE 3.3. Token-derived metrics: what each identifies and what it cannot identify.

Metric	Identifies	Cannot identify
Tokens per call	input and output intensity of one call	calls per request; cache state; quality
Tokens per request	intensity including the call multiplier k	whether k is necessary; value
Price per million tokens	unit price in one tokenizer's units	quantity; comparability across models
Cost per request	application-level unit cost	request value; adjacent cost; latency
Cost per accepted outcome	economic cost of a usable result	value; risk; delay
Total spend	budget magnitude	whether growth is waste or adoption

Suppose a team is told to cut tokens per request. It can truncate outputs, compress prompts past the point of sufficiency, remove verification passes, lower the reasoning effort, move work into a retrieval service or a human review queue booked to another budget, or switch to a model whose tokenizer yields fewer tokens for the same text (Section 3.2). Each of these improves the metric. Only some improve the business. Tokenizer switching is a pure causal failure: it changes the unit, not the work. Cost shifting is a failure of scope. Truncation and effort reduction are extremal failures, harmless in moderation and damaging past a threshold that the metric cannot see.

RUNNING EXAMPLE R2: A TOKEN TARGET THAT RAISES COST

Suppose the R2 team runs gpt-5.4-mini under the reasoned strategy and is told to cut tokens per task. Switching to the direct strategy does it spectacularly: billed tokens fall from 3,072 to 1,294 per task (−57.9%) and cost per run from \$0.01233 to \$0.00433 (−64.8%). Accuracy falls from 0.2875 to 0.0875, so cost per correct answer rises from \$0.0429 to \$0.0495 (+15.5%). If each correct reconciliation is worth \$400 (the value used in the R2 analysis), the change saves \$7.99 per thousand tasks and forgoes \$80,000 of expected value per thousand tasks. The token metric reports a success; the business has lost four orders of magnitude more than it saved.

The response is not to abandon token metrics but to deny them the degenerate solutions. Three design rules follow. *Pair every cost metric with a binding quality constraint*, so that truncation and effort reduction are infeasible once they reduce acceptance below a floor $q_{\min}$; this is why quality floors appear as constraints rather than objectives in Chapter 10. *Denominate in currency per accepted outcome rather than in tokens*, which removes the tokenizer-switching route entirely. *Draw the cost boundary around the whole system*, including adjacent costs, so that shifting work to another budget does not count

as saving. The provider side has its own version of the problem. Under per-token pricing, a provider earns more when a model reasons longer, and the customer cannot see the reasoning; Velasco et al. (2026) show formally that pay-per-token pricing gives a provider an incentive to misreport tokenization. Chapter 14 studies the resulting contracting problem; the next section asks what can be measured.

💡 KEY IDEA

Every token metric that is used as a target needs a partner metric that its cheapest improvements would damage. A cost metric without a quality floor rewards the destruction of quality.

3.6 Verifying the Bill

3.6.1 What the Customer Can and Cannot Observe

Return to Fig. 3.1. The customer sends the input and receives visible output and an invoice. Input and visible output can be checked by re-tokenizing them with the provider's tokenizer when it is public. Everything else is reported by the provider: the number of reasoning tokens, which input was served from cache, which model actually ran, and at what precision. Sun et al. (2025b) group the resulting risks into *quantity inflation* (token or call counts reported above what was used) and *quality downgrade* (a cheaper model or tool substituted for the one paid for), and argue that services whose operations are opaque to customers need auditing.

The incentive is not hypothetical. Velasco et al. (2026) show that even when the generative process is transparent, a provider can overcharge profitably by reporting a longer tokenization of the output string than the one generated, and they propose pricing per character, which removes the incentive while preserving average margins. Hoque et al. (2026) study hidden reasoning and report that, against the audit frameworks they evaluate, reasoning counts could be inflated by more than an order of magnitude on average without detection, and that tokenization ambiguity alone permits substantial over-reporting even when the full reasoning text is shown. Their central observation is that when all evidence comes from the provider, an audit reduces to a consistency check on the provider's own reports.

3.6.2 Four Families of Audit

Table 3.4 and Fig. 3.5 organize the proposed remedies by what the verifier must trust.

Predictive audits need nothing from the provider. PALACE (Wang et al., 2025) estimates the number of hidden reasoning tokens from the prompt and the visible answer, and flags bills far above the estimate. It is cheap and deployable today, but a provider that knows the predictor can inflate within its error. *Commitment-based audits* ask the provider to commit to the hidden content in a form that can later be checked in part.

TABLE 3.4. Families of audit for token bills. “Trusts” names what must be honest for the audit to mean anything.

Family	Checks	Trusts	Examples
Predictive	reported counts against counts predicted from prompt–answer pairs	the predictor; that inflation is not tailored to it	Wang et al. (2025)
Commitment	that reported tokens exist and relate to the task	that committed content is what was generated	Sun et al. (2025a)
Hardware attestation	which model ran and what it counted	the enclave vendor	Cai et al. (2025)
Proof or re-execution	that the output came from the claimed model and input	cryptographic assumptions; a verifier with compute	Ong et al. (2025); Sun et al. (2024)

CoIn (Sun et al., 2025a) builds a hash tree over fingerprints of token embeddings, so that an auditor can verify counts by spot-checking, and uses embedding-based relevance matching to detect reasoning content fabricated to pad the count; the authors report detecting token-count inflation with success rates up to 94.7% in their experiments. The provider still generates the committed evidence. *Hardware attestation* moves trust to a trusted execution environment that signs what model was loaded and what it counted. Cai et al. (2025) study model substitution, find that text-based statistical tests are query-intensive and miss subtle substitutions and that log-probability checks are defeated by nondeterminism in production inference, and propose trusted execution environments as a practical remedy with modest overhead. *Proofs of inference and re-execution* let a verifier check the computation itself. TOPLOC (Ong et al., 2025) commits to locality-sensitive hashes of intermediate activations, a few hundred bytes per 32 generated tokens, which a verifier can check faster than the original inference, and zkLLM (Sun et al., 2024) produces zero-knowledge proofs of a model’s forward pass at much higher cost. Moving down the table, the evidence becomes less controllable by the provider and more expensive to produce.

3.6.3 How Much Auditing Is Enough

Strong verification is costly, so it is applied to a sample. The following proposition shows that sampling is powerful when the per-request check is strong, and useless when it is not.

PROPOSITION 3.11 (Detection by sampling audits). *A provider bills a set of requests and overbills a fraction θ of them, chosen before the audit sample is drawn and without knowledge of it. An auditor draws n requests independently and uniformly at random and applies a check that flags an overbilled request with probability at least δ , independently across draws, and never flags a correctly billed one. Then:*

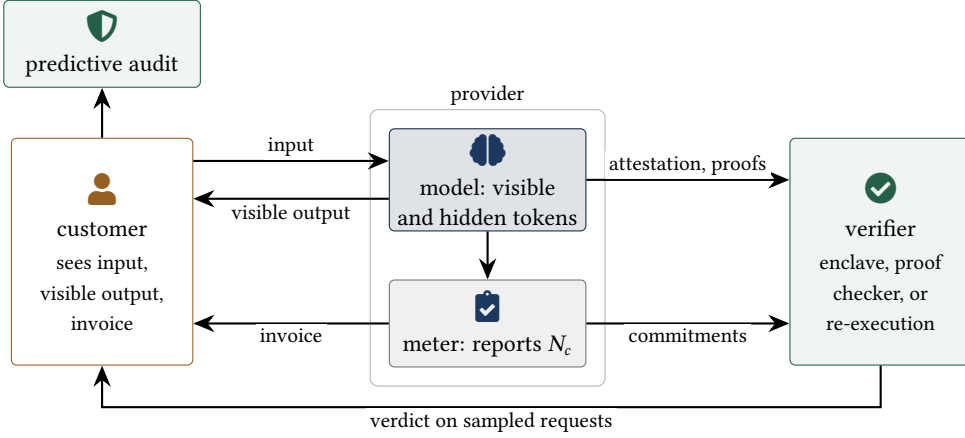


FIGURE 3.5. Who sees what in a token bill. The customer observes input, visible output, and the invoice, and can run a predictive audit on those alone. Stronger checks require evidence that leaves the provider's frame in a form the provider cannot forge: a signed attestation, a proof, or enough information for independent re-execution. Such checks are expensive, so they are applied to a sample of requests (Proposition 3.11).

- (i) the probability that the audit flags at least one request is at least $1 - (1 - \theta\delta)^n$;
- (ii) a sample of $n \geq \ln \beta / \ln(1 - \theta\delta)$ requests detects overbilling with probability at least $1 - \beta$;
- (iii) a provider that wants to escape detection with probability at least β can overbill a fraction of at most $\theta \leq (1 - \beta^{1/n})/\delta$ of requests.

Proof. Each draw lands on an overbilled request with probability θ and, given that, is flagged with probability at least δ ; correctly billed requests are never flagged. So each draw fails to flag with probability at most $1 - \theta\delta$, and by independence all n draws fail with probability at most $(1 - \theta\delta)^n$, which gives (i). Requiring $(1 - \theta\delta)^n \leq \beta$ and taking logarithms (note $\ln(1 - \theta\delta) < 0$) gives (ii). For (iii), escaping with probability at least β requires $\beta \leq (1 - \theta\delta)^n$, that is, $\theta \leq (1 - \beta^{1/n})/\delta$. ■

EXAMPLE 3.12 (Audit sample sizes). Suppose 5% of requests are overbilled ($\theta = 0.05$) and the auditor wants a 95% chance of catching it ($\beta = 0.05$). With a strong check such as attested re-execution ($\delta = 0.9$), $n \geq \ln 0.05 / \ln(1 - 0.045) = 65.1$, so 66 requests suffice. With a weak check ($\delta = 0.1$), 598 are needed; with $\delta = 0.01$, 5,990. Conversely, if 100 requests are audited with $\delta = 0.9$, a provider that wants at least an even chance of escaping can overbill no more than $(1 - 0.5^{1/100})/0.9 = 0.77\%$ of requests.

The proposition moves the difficulty to δ . The findings of Hoque et al. (2026) amount to the statement that checks built only on provider-controlled evidence have a detection probability δ that an adaptive provider can drive toward zero, and no sample size compensates for that. The assumption that the provider cannot see the sample matters

too. If the provider learns which requests will be audited, it can bill those honestly, and the effective θ within the sample is zero.

Predictive audits need a different calculation, because they detect a shift in averages rather than individual violations. Let $e_i = R_i/\hat{R}_i - 1$ be the relative gap between the reported reasoning count R_i and a prediction $\hat{R}_i$ that is unbiased for honest bills, with standard deviation σ . If the provider inflates all counts by a factor $1 + \iota$, the mean of e_i moves from 0 to about ι . By the standard sample-size formula for a one-sided test at the 5% level with 80% power, about $n \approx ((1.645 + 0.842)\sigma/\iota)^2$ requests are needed. In R2 the coefficient of variation of gpt-5.4's reasoning tokens under the reasoned strategy is 0.38, so a naive predictor that forecasts the mean detects 10% inflation after about 90 requests; a predictor with $\sigma = 0.20$ (hypothetical) needs 25, and with 100 requests it detects inflation of 5%. The weakness is the same as before in another form: a provider that knows the predictor can inflate only requests where the prediction is uncertain, or inflate by less than the detectable margin on every request.

KEY IDEA

A bill is only as verifiable as the evidence the provider cannot control. Sampling makes strong checks affordable, but it cannot make weak checks strong.

What can and cannot be verified, then? Counts of input and visible output can be verified today wherever the tokenizer is public. The quantity of hidden reasoning can be bounded statistically, but not verified request by request, unless the provider supplies commitments or attestations. The identity and precision of the model can be verified only with hardware attestation or proofs of inference. And no audit verifies that the reasoning was *necessary*, only that it happened; necessity is a question of value (Chapter 9) and of contract design (Chapter 14).

3.7 Toward a Price Index for Inference

“Inference is getting cheaper” is among the most repeated claims about AI. Sections 3.2 to 3.4 show why it is hard to state precisely: the unit changes across models, the good changes across versions, and the quantity of tokens needed for a task changes with the model. Economists have faced the same problem with computers, and the tools they developed apply.

3.7.1 Index-Number Theory in Brief

A price index compares the cost of a fixed bundle of goods at two dates. Let services $j = 1, \dots, J$ have prices $p_{j,t}$ and quantities $Q_{j,t}$ in period $t \in \{0, 1\}$.

DEFINITION 3.13 (Laspeyres, Paasche, and Fisher indexes). The Laspeyres index prices the base-period bundle, the Paasche index prices the current bundle, and the Fisher index

is their geometric mean:

$$I^L = \frac{\sum_j p_{j,1} Q_{j,0}}{\sum_j p_{j,0} Q_{j,0}}, \quad I^P = \frac{\sum_j p_{j,1} Q_{j,1}}{\sum_j p_{j,0} Q_{j,1}}, \quad I^F = \sqrt{I^L I^P}.$$

When buyers substitute toward goods whose prices fell most, the Laspeyres index, which ignores the substitution, overstates the price level in period 1, and the Paasche index understates it. The Fisher index is *superlative*: it is exact for a flexible class of preferences and lies between the two (Fisher, 1922; Diewert, 1976). Over many periods the indexes are chained, comparing each period with the one before.

EXAMPLE 3.14 (An index for three inference services). Consider three hypothetical services, priced per thousand tasks from a fixed task suite: a frontier model (\$40 in period 0, \$30 in period 1), a mid-tier model (\$10, \$6), and a small model (\$2, \$1.80). Buyers use 10, 50, and 100 thousand tasks in period 0 and 15, 90, and 60 thousand in period 1, moving toward the mid-tier model, whose price fell most. The base bundle costs \$1,100 at period-0 prices and \$780 at period-1 prices, so $I^L = 0.7091$. The period-1 bundle costs \$1,620 and \$1,098, so $I^P = 0.6778$. The Fisher index is $I^F = 0.6933$. The three measures of price change are -29.1% , -32.2% , and -30.7% . The gap between the first two is the substitution effect.

The unit of the index matters more than the formula. **Example 3.14** prices *tasks*. Had it priced tokens, every effect in Sections 3.2 and 3.3 would contaminate the result: a new tokenizer that yields 30% more tokens at unchanged per-token prices would register as no change, while the cost of each task rose by 30%.

3.7.2 Hedonic Adjustment for Capability

The harder problem is that goods change. The frontier model of period 1 is not the frontier model of period 0: it is more accurate, has a longer context, and may reason more. A price comparison that ignores quality misses part of the decline when quality rises at a fixed price. *Hedonic regression* addresses this by treating a good as a bundle of characteristics (Griliches, 1961; Triplett, 2006). With a quality score z_j for model j ,

$$\ln p_{j,t} = \gamma_t + \psi z_j + e_{j,t}, \quad (3.3)$$

where ψ is the log-price premium per unit of quality, $e_{j,t}$ is an error, and $\exp(\gamma_1 - \gamma_0)$ is the quality-adjusted price ratio between the periods.

EXAMPLE 3.15 (A hedonic index from eight models). Four hypothetical models are sold in each period, with prices per task and quality scores (benchmark accuracy in percent): in period 0, (50, \$2.00), (60, \$4.00), (70, \$9.00), (80, \$20.00); in period 1, (60, \$1.50), (70, \$3.20), (80, \$7.00), (90, \$15.00). Least squares on Eq. (3.3) gives $\hat{\psi} = 0.0770$ (ten quality points multiply price by 2.16) and $\hat{\gamma}_1 - \hat{\gamma}_0 = -1.033$, so the quality-adjusted price ratio is $\exp(-1.033) = 0.356$, a decline of 64.4%. The naive comparison of geometric mean

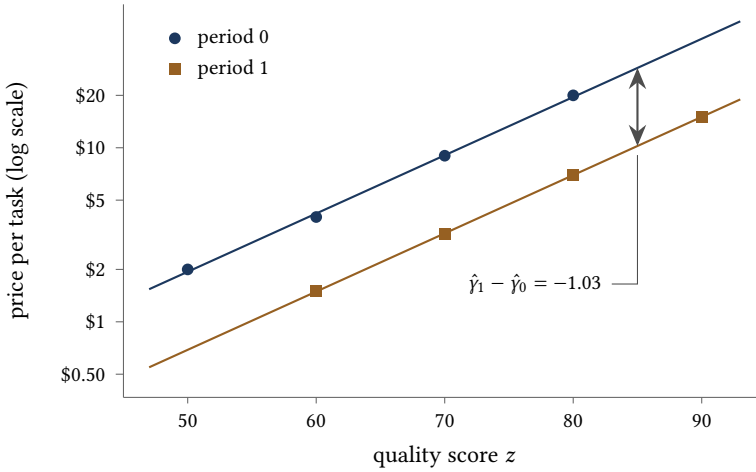


FIGURE 3.6. The hedonic regression of [Example 3.15](#) (hypothetical data). Each period's models lie on a line of common slope $\hat{\psi}$; the vertical distance between the lines is the quality-adjusted log price change. A comparison of average prices mixes this shift with the movement of the models along the line toward higher quality.

prices, which ignores that average quality rose from 65 to 75, shows a decline of only 23.1%. Matching models of equal quality where both periods have one (60, 70, and 80) gives ratios of 0.375, 0.356, and 0.350, with geometric mean 0.360, close to the hedonic estimate.

Hedonic methods have a long record in computing. [Berndt and Rappaport \(2001\)](#) document quality-adjusted price declines for personal computers over a quarter century, and [Nordhaus \(2007\)](#) traces the cost of computation over two centuries. Two precedents are especially close to inference. [Byrne et al. \(2018\)](#) construct hedonic price indexes for cloud compute, storage, and database services from 2009 to 2016 and find prices falling throughout, at double-digit annual rates from early 2014; the cloud, like inference, is a metered service whose unit (an instance-hour) hides changing hardware. [Coyle and Hampton \(2024\)](#) argue that the cost of computation has continued to fall rapidly once innovation in chip types and cloud computing is taken into account, though more slowly since the mid-2010s. For language models, [Cottier et al. \(2025\)](#) measure the price of reaching fixed benchmark thresholds and find declines ranging from 9 to 900 times per year depending on the task, and [Gundlach et al. \(2025\)](#) find that the price of a given level of benchmark performance has fallen roughly 5 to 10 times per year for frontier models, while the prices of frontier models themselves have risen. Both are fixed-quality measures in the spirit of [Eq. \(3.3\)](#), and both price tasks or benchmark scores rather than raw tokens.

3.7.3 What an Inference Price Index Should Measure

There is at present no accepted quality-adjusted price index for inference, and building one is a research problem, not a routine calculation. The chapter's results say what it should hold fixed. The unit should be a *task at a stated quality*, not a token (Proposition 3.3). The price should be the full cost per accepted outcome, including reasoning tokens, which can dominate the bill (Fig. 3.3), and ideally latency (Example 3.10). The characteristics should include accuracy on a task distribution of interest, effective context length, tail latency at a stated load, tool-use reliability, and refusal behavior. Public evaluations supply some of these (Liang et al., 2023; Chiang et al., 2024), with known limitations, including contamination and the instability of model-graded judgments (Zheng et al., 2023). And the index must survive the provider's control of the unit: tokenizer changes, changed reasoning defaults, and silent model updates all move cost per task without moving any posted price.

IN PRACTICE (AS OF SEPTEMBER 2026)

Claude Opus 5 is priced at \$5 per million input tokens and \$25 per million output tokens, unchanged from Claude Opus 4.8, but thinking is on by default. Anthropic's documentation notes that a workload that ran without thinking on the earlier model can therefore produce more output tokens per request at the same per-token rates (Anthropic, 2026d). A per-token price index records no change across this transition; an index per task records whatever change in tokens per task, and in quality, the customer's workload experiences.

KEY IDEA

"Inference got cheaper" is a claim about a price index, and a meaningful one prices tasks at fixed quality. Per-token price series confound price change with changes in the unit, in verbosity, and in quality.

KEY TAKEAWAYS

- A token is a unit of account defined by a provider's tokenizer. Token counts are comparable within a tokenizer and not across tokenizers, languages, or model versions; any fixed exchange rate between two tokenizers misstates some text by $(r_{\max} - r_{\min}) / (r_{\max} + r_{\min})$ (Proposition 3.3).
- A uniform price per token is a non-uniform price per unit of meaning. In the six-language example, the same sentence costs up to 8.7 times the English token count under one tokenizer.
- The output premium is set by machine balance, batch size, precision, and context

length (Proposition 3.6). Larger batches shrink it; long contexts put a floor under it.

- Bills contain input, cached, visible output, and reasoning tokens. In R2, hidden reasoning is 61.6% of billed tokens and 71.9% of billed cost.
- Token counts miss latency, quality, the necessity of reasoning, and adjacent cost. Compare designs in currency per accepted outcome; in R2 the model that is three times cheaper per run is twice as expensive per correct answer.
- Token targets invite gaming. Pair each cost metric with a quality floor, report cost per accepted outcome, and draw the cost boundary around the whole system. As a target for teams, use the failure-priced economic cost per resolved case (Definition 9.2); cost per accepted outcome still rewards under-spending (Proposition 17.13).
- Hidden tokens are credence goods. Sampling audits detect overbilling efficiently only when each check relies on evidence the provider cannot control (Proposition 3.11).
- A price index for inference should price tasks at fixed quality using superlative or hedonic methods, following the precedents of computer and cloud price measurement.

Notes and Further Reading

Tokenization. Byte-pair encoding entered neural language processing through Sennrich et al. (2016); Kudo (2018) introduced unigram segmentation and subword regularization, and Kudo and Richardson (2018) the toolkit through which both are widely used. The economic consequences of tokenization were first studied through fairness across languages: Petrov et al. (2023) measure tokenization disparities across languages and tokenizers, and Ahia et al. (2023) connect them directly to commercial per-token prices and context limits. Rust et al. (2021) show that the quality of a multilingual model’s tokenizer for a language affects its downstream performance in that language. On the pricing side, Velasco et al. (2026) analyze the incentive to misreport tokenization under pay-per-token pricing and propose pay-per-character pricing, and Chatzi et al. (2025) show that tokenization multiplicity makes the price of a string arbitrary. Measurement-theoretic language for units and scales goes back to Stevens (1946) and is developed axiomatically in Krantz et al. (1971).

The cost of a token. The derivation of the output premium follows the roofline model of Williams et al. (2009), applied to transformer inference by Pope et al. (2023). The systems that raise effective batch size, continuous batching (Yu et al., 2022) and paged KV memory (Kwon et al., 2023), and that cut decode cost through quantization (Dettmers et al., 2022; Frantar et al., 2023; Lin et al., 2024) and speculative decoding (Leviathan et al., 2023; Chen et al., 2023), are the levers Proposition 3.6 identifies; Chapters 2 and 4 treat them in detail. Prompt caching as a serving technique is described by Gim et al. (2024)

and Zheng et al. (2024). The categories of tokens and the observation that reasoning tokens are largely invisible yet billed are discussed in Zhu (2026d), and the R2 experiment comes from Zhu (2026c).

Metrics and strategic response. Goodhart’s law originates in monetary policy (Goodhart, 1975); Campbell’s parallel law concerns social indicators (Campbell, 1979); Manheim and Garrabrant (2018) classify the mechanisms. The multitask principal–agent model of Holmström and Milgrom (1991) explains why rewarding a measurable dimension of effort degrades unmeasured ones, which is the formal core of the argument for quality floors. Dulleck and Kerschbamer (2006) survey the economics of credence goods, the category into which hidden reasoning falls.

Verification. Sun et al. (2025b) set out the audit problem for opaque AI services and a taxonomy of approaches. CoIn (Sun et al., 2025a) and PALACE (Wang et al., 2025) are commitment-based and predictive audits of hidden reasoning, and Hoque et al. (2026) show how audits built on provider-controlled evidence can be circumvented. Cai et al. (2025) study model substitution and argue for trusted execution environments. TOPLOC (Ong et al., 2025) and zkLLM (Sun et al., 2024) represent the lightweight and the cryptographically strong ends of verifiable inference. Proposition 3.11 is the elementary acceptance-sampling calculation; its role here is to show that the sample size is secondary to the strength of the per-request check.

Price measurement. Fisher (1922) and Diewert (1976) are the classic references on index formulas and superlative indexes; Griliches (1961) introduced hedonic price indexes and Triplett (2006) is the standard handbook. For computing, see Berndt and Rappaport (2001) and Nordhaus (2007); for cloud services, Byrne et al. (2018) and Coyle and Hampton (2024). Recent measurements of inference prices at fixed capability are Cottier et al. (2025) and Gundlach et al. (2025), and Demiret et al. (2026) describe the market in which these prices are set. Chapter 13 studies how providers design the price lists that such indexes summarize, and Chapter 21 returns to the inference price index as an open problem.

Exercises

EXERCISE 3.1 (Analysis). Model A is priced at \$1 and model B at \$3 per million output tokens. Model A’s answers average 500 output tokens and half its attempts are accepted, so it needs on average one retry per accepted answer. Model B’s answers are 40% shorter, and it needs half as many retries per accepted answer. Ignoring input tokens, compute the expected cost per accepted answer for each model. Holding model B’s cost per accepted answer fixed, at what acceptance probability for model A would the two be equally expensive? *Hint:* With acceptance probability s , the expected number of attempts is $1/s$ and the expected number of retries is $1/s - 1$.

EXERCISE 3.2 (Analysis). Construct Laspeyres and Paasche price indexes for inference across three models and two periods, using prices per task and quantities of your choice in which buyers substitute toward the model whose price fell most. Compute the Fisher index and explain why the Laspeyres and Paasche indexes disagree. Then change the quantities so that buyers substitute toward the model whose price fell least, and explain what happens to the ordering.

EXERCISE 3.3 (Design). Propose an audit procedure that lets a customer verify a bill that includes hidden reasoning tokens. Specify what the provider must supply, what the customer or a third party computes, how requests are sampled, and what the procedure can and cannot establish. Discuss how your procedure fares against a provider that knows the procedure. *Hint:* Separate the count of reasoning tokens, the identity of the model, and the necessity of the reasoning; Table 3.4 and Proposition 3.11 address different parts.

EXERCISE 3.4 (Analysis). Using Table 3.1, restrict attention to the four languages written in Latin or Greek script. Compute $r_{\min}$, $r_{\max}$, the best exchange rate ξ^* , and the minimax relative error of Proposition 3.3. Explain why the error is smaller than for all six languages, and what this implies for a firm whose traffic is almost entirely European.

EXERCISE 3.5 (Analysis). Under a single tokenizer with price p per token, define a language's *tokenization premium* as its token count divided by the English count for the same content. Using the GPT-2 and Qwen2.5 columns of Table 3.1, compute the price a Hindi speaker pays relative to an English speaker under each tokenizer. What per-character price would make the charges equal for this sentence, and what are its drawbacks? *Hint:* Compare the character and byte columns; neither is a language-neutral measure of meaning either.

EXERCISE 3.6 (Analysis). In Example 3.7 with $S = 4,000$, find the batch size at which the batch term $b/(2B)$ of Eq. (3.2) falls to 10% of the KV term $\kappa S/(2P)$. Is that batch size likely to fit in memory? Then compute how ρ at $B = 128$ changes if weights are quantized to 4 bits ($b = 0.5$) while the KV cache stays at 16 bits. *Hint:* Solve $b/(2B) = 0.1 \kappa S/(2P)$ for B .

EXERCISE 3.7 (Implementation). Tokenize the same paragraph (about 100 words) in three languages with two public tokenizers, and compare token counts. Report the ratios as in Table 3.1, compute the minimax exchange-rate error, and state what your results imply for per-token pricing across languages. Repeat with a paragraph of source code and a paragraph of numbers.

EXERCISE 3.8 (Analysis). The reasoning share of R2's bill depends on prices. Suppose both models' output prices were three times, rather than six times, their input prices, with input prices unchanged. Using the category means in Fig. 3.3 or the data file, recompute the share of reasoning in total billed cost across all runs. Explain why the

share falls less than proportionally. *Hint:* The share across all runs requires the totals, not the average of the cell shares; reasoning is a large fraction of output in every cell.

EXERCISE 3.9 (Analysis). An auditor can afford to re-execute 40 requests a month in an attested environment ($\delta = 0.95$). (a) If a provider overbills 2% of requests, what is the probability of detection in a month? In a year of independent monthly audits? (b) What fraction of requests could a provider overbill while keeping at least a 90% chance of escaping detection in a single month? (c) Which assumption of [Proposition 3.11](#) would a provider attack first?

EXERCISE 3.10 (Design). A platform team is rewarded for reducing “tokens per resolved ticket” in running example R1. List five ways to improve the metric, classify each by the Goodhart mechanism it exploits, and design a pair of metrics that makes each of them unprofitable. Estimate, using the R1 numbers, how large a drop in resolution rate would offset a 10% token saving.

EXERCISE 3.11 (Analysis). In [Example 3.15](#), a new model with quality 95 and price \$40 per task enters in period 1. Re-estimate [Eq. \(3.3\)](#) with the new observation and report the change in the quality-adjusted index. Why can a matched-model index not use this observation at all, and why is that a problem when quality improves mainly through new models? *Hint:* Add a row to the design matrix with the period-1 dummy set to one; a least-squares routine does the rest.

EXERCISE 3.12 (Analysis). In [Example 3.10](#), find the delay cost c_d per hour of waiting at which option B’s full cost is twice option A’s. Then suppose option B raises first-contact resolution enough to avoid one escalation (\$4) in every 200 requests. At what delay cost does B become the cheaper option per request, counting escalations?

Bibliography

- Daron Acemoglu. The simple macroeconomics of AI. *Economic Policy*, 40(121):13–58, 2025.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? Tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore, 2023. Association for Computational Linguistics.
- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Blake Alcott. Jevons’ paradox. *Ecological Economics*, 54(1):9–21, 2005.
- Eduardo Alvarez, Omri Almog, Eric Chung, Simon Layton, Dusan Stosic, Ronny Krashinsky, and Kyle Aubrey. Introducing NVFP4 for efficient and accurate low-precision inference. <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>, 2025. NVIDIA Technical Blog. Accessed: 2026-09-15.
- Anthropic. Prompt caching. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>, 2026a. Claude API documentation. Accessed: 2026-09-15.
- Anthropic. Models overview. <https://platform.claude.com/docs/en/about-claude/models/overview>, 2026b. Claude API documentation. Accessed: 2026-09-15.
- Anthropic. Introducing Claude Opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, April 2026c. Accessed: 2026-09-15.
- Anthropic. What’s new in Claude Opus 5. <https://platform.claude.com/docs/en/models/opus-5/whats-new-opus-5>, 2026d. Claude API documentation. Accessed: 2026-09-15.
- Anthropic. Extended thinking.

- <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>, 2026e. Claude API documentation. Accessed: 2026-09-15.
- Ernst R. Berndt and Neal J. Rappaport. Price and quality of desktop and mobile personal computers: A quarter-century historical overview. *American Economic Review*, 91(2):268–273, 2001.
- Erik Brynjolfsson, Daniel Rock, and Chad Syverson. The productivity J-curve: How intangibles complement general purpose technologies. *American Economic Journal: Macroeconomics*, 13(1):333–372, 2021.
- Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. Generative AI at work. *The Quarterly Journal of Economics*, 140(2):889–942, 2025.
- David Byrne, Carol Corrado, and Daniel E. Sichel. The rise of cloud computing: Minding your P’s, Q’s and K’s. NBER Working Paper 25188, National Bureau of Economic Research, 2018.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.
- Will Cai, Tianneng Shi, Xuandong Zhao, and Dawn Song. Are you getting what you pay for? auditing model substitution in LLM APIs, 2025.
- Donald T. Campbell. Assessing the impact of planned social change. *Evaluation and Program Planning*, 2(1):67–90, 1979.
- Ivi Chatzi, Nina Corvelo Benz, Stratis Tsirtsis, and Manuel Gomez-Rodriguez. Tokenization multiplicity leads to arbitrary price variation in LLM-as-a-service, 2025.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Yuxi Chen, Junming Chen, Chenyu He, Yiwei Li, Yicheng Ji, Yifan Wu, Dingyu Yang, et al. Token economics for LLM agents: A dual-view study from computing and economics. *arXiv preprint arXiv:2605.09104*, 2026.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, et al. Chatbot arena: An open platform for evaluating LLMs by human preference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Ben Cottier, Ben Snodin, David Owen, and Tom Adamczewski. LLM inference prices have fallen rapidly but unequally across tasks. Epoch AI Data Insight, <https://epoch.ai/data-insights/llm-inference-price-trends>, March 2025.
- Diane Coyle and Lucy Hampton. 21st century progress in computing. *Telecommunications Policy*, 48(1):102649, 2024. doi: 10.1016/j.telpol.2023.102649. Working-paper version: “Twenty-First Century Progress in Computing,” Bennett Institute for Public Policy, 2023.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- DeepSeek-AI. DeepSeek-V3 technical report, 2024.
- Mert Demirer, Andrey Fradkin, and Nadav Tadelis. The emerging market for intelligence: How

- firms buy and sell AI. *Journal of Economic Perspectives*, 40(3):23–46, 2026. doi: 10.1257/jep.20261506.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- W. Erwin Diewert. Exact and superlative index numbers. *Journal of Econometrics*, 4(2):115–145, 1976.
- Uwe Dulleck and Rudolf Kerschbamer. On doctors, mechanics, and computer specialists: The economics of credence goods. *Journal of Economic Literature*, 44(1):5–42, 2006. doi: 10.1257/002205106776162717.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- FinOps Foundation. The FinOps framework. <https://www.finops.org/framework/>, 2024. Accessed 18 August 2026.
- Irving Fisher. *The Making of Index Numbers: A Study of Their Varieties, Tests, and Reliability*. Houghton Mifflin, Boston, 1922.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. AI and memory wall. *IEEE Micro*, 2024. arXiv:2403.14123.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024.
- Charles A. E. Goodhart. Problems of monetary management: The UK experience. *Papers in Monetary Economics, Reserve Bank of Australia*, 1, 1975.
- Google. Gemini thinking. Gemini API documentation, 2026. URL <https://ai.google.dev/gemini-api/docs/thinking>. Accessed 15 September 2026.
- Zvi Griliches. Hedonic price indexes for automobiles: An econometric analysis of quality change. *The Price Statistics of the Federal Government, NBER*, pages 173–196, 1961.
- Hans Gundlach, Jayson Lynch, Matthias Mertens, and Neil Thompson. The price of progress: Price performance and the future of AI, 2025.
- Daya Guo et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi: 10.1038/s41586-025-09422-z. arXiv:2501.12948, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- Bengt Holmström and Paul Milgrom. Multitask principal-agent analyses: Incentive contracts, asset ownership, and job design. *Journal of Law, Economics, and Organization*, 7:24–52, 1991.

- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- Shahinul Hoque, Jinghui Zhang, Jinyuan Sun, and Fnu Suya. Token inflation: How dishonest providers can overcharge for large language model usage, 2026.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- William Stanley Jevons. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan and Co., London, 1865.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, et al. Mixtral of experts, 2024.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- David H. Krantz, R. Duncan Luce, Patrick Suppes, and Amos Tversky. *Foundations of Measurement, Volume I: Additive and Polynomial Representations*. Academic Press, New York, 1971.
- Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75. Association for Computational Linguistics, 2018.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71. Association for Computational Linguistics, 2018.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.15077.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight

- quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s law, 2018.
- Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, et al. Artificial intelligence index report 2025. Stanford Institute for Human-Centered Artificial Intelligence, 2025.
- Nicholas Merizzi, Tim Smith, Diana Kearns-Manolatos, Nitin Mittal, and Gaurav Churiwala. The pivot to tokenomics: Navigating AI’s new spend dynamics. Industry report, Deloitte, January 2026a. URL <https://www.deloitte.com>. Accessed January 2026.
- Nicholas Merizzi, Tim Smith, Nitin Mittal, Gaurav Churiwala, and Diana Kearns-Manolatos. AI tokens: How to navigate AI’s new spend dynamics. Deloitte Insights, <https://www.deloitte.com/us/en/insights/topics/emerging-technologies/ai-tokens-how-to-navigate-spend-dynamics.html>, January 2026b. Published 19 January 2026; accessed 15 September 2026. Web version of the report cited as `deloitte2026tokenomics`.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 2025. arXiv:2312.15234.
- Moonshot AI. Kimi-K2-Instruct. <https://huggingface.co/moonshotai/Kimi-K2-Instruct>, 2025. Hugging Face model card. Accessed: 2026-09-15.
- William D. Nordhaus. Two centuries of productivity growth in computing. *The Journal of Economic History*, 67(1):128–159, 2007.
- Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference, 2025.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025a.
- OpenAI. gpt-oss-120b. <https://huggingface.co/openai/gpt-oss-120b>, 2025b. Hugging Face model card and configuration. Accessed: 2026-09-15.
- OpenAI. Prompt caching. <https://developers.openai.com/api/docs/guides/prompt-caching>, 2026a. OpenAI API documentation. Accessed: 2026-09-15.
- OpenAI. GPT-5.5 model. <https://developers.openai.com/api/docs/models/gpt-5.5>, 2026b. OpenAI API documentation. Accessed: 2026-09-15.
- OpenAI. API pricing. <https://developers.openai.com/api/docs/pricing>, 2026c. Accessed: 2026-09-15.
- OpenAI. Reasoning models. OpenAI API documentation, 2026d. URL <https://developers.openai.com/api/docs/guides/reasoning>. Accessed 15 September 2026.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2024. doi: 10.1109/ISCA59077.2024.00019.

- David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David R. So, Maud Texier, and Jeff Dean. The carbon footprint of machine learning training will plateau, then shrink. *Computer*, 55(7):18–28, 2022.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Sundar Pichai. Google I/O 2026: Sundar pichai’s opening keynote. The Keyword (Google blog), <https://blog.google/innovation-and-ai/sundar-pichai-io-2026/>, May 2026. Published 19 May 2026; accessed 15 September 2026.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- Mohamad Salim, Jasmine Latendresse, SayedHassan Khatoonabadi, and Emad Shihab. Tokenomics: Quantifying where tokens are used in agentic software engineering. In *International Conference on Mining Software Repositories (MSR ’26)*, 2026.
- Nikhil Sardana, Jacob Portes, Sasha Doubrov, and Jonathan Frankle. Beyond Chinchilla-optimal: Accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of PMLR, pages 43445–43460, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics.
- Carl Shapiro and Hal R. Varian. *Information Rules: A Strategic Guide to the Network Economy*. Harvard Business School Press, 1998.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

- S. S. Stevens. On the theory of scales of measurement. *Science*, 103(2684):677–680, 1946.
- J. R. Stormont and Mike Fuller. *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management*. O’Reilly Media, 2nd edition, 2023.
- Guoheng Sun, Ziyao Wang, Bowei Tian, Meng Liu, Zheyu Shen, Shwai He, Yexiao He, Wanghao Ye, Yiting Wang, and Ang Li. CoIn: Counting the invisible reasoning tokens in commercial opaque LLM APIs, 2025a.
- Guoheng Sun, Ziyao Wang, Xuandong Zhao, Bowei Tian, Zheyu Shen, Yexiao He, Jinming Xing, and Ang Li. Invisible tokens, visible bills: The urgent need to audit hidden operations in opaque LLM services, 2025b.
- Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 4405–4419, 2024. doi: 10.1145/3658644.3670334.
- Tokenomics Foundation. Tokenomics and FinOps: How do they fit together? <https://www.tokeneconomics.com/insights/tokenomics-and-finops-how-do-they-fit-together/>, 2026a. Accessed 18 August 2026.
- Tokenomics Foundation. AI tokenomics: Definition and overview. <https://www.tokeneconomics.com/>, 2026b. Accessed 18 August 2026; the Foundation labels its definition a working draft.
- Tokenomics Foundation. You cannot govern what you cannot see: What enterprise tokenomics looks like in practice. <https://www.tokeneconomics.com/insights/enterprise-tokenomics-in-practice-at-metlife/>, 2026c. Accessed 18 August 2026; fireside discussion with S. Niswander (MetLife), presented as an illustrative operating model rather than a standard.
- Tokenomics Foundation. Production, consumption, value. <https://www.tokeneconomics.com/docs/overview/production-consumption-value/>, 2026d. Accessed 18 August 2026.
- Tokenomics Foundation. The five-layer tokenomics stack. <https://www.tokeneconomics.com/projects/the-five-layer-tokenomics-stack/>, 2026e. Accessed 18 August 2026; presented as a starting model.
- Tokenomics Foundation. What is tokenomics? <https://www.tokeneconomics.com/docs/overview/what-is-tokenomics/>, 2026f. Accessed 18 August 2026.
- Tokenomics Foundation and Dan Neff. Big-T notation: An open framework for token-consumption growth. <https://www.tokeneconomics.com/projects/big-t-notation/>, 2026. Accessed 18 August 2026; described by its authors as an intentionally informal reasoning framework.
- Jack E. Triplett. *Handbook on Hedonic Indexes and Quality Adjustments in Price Indexes*. OECD Publishing, Paris, 2006.
- Tim Tully, Joff Redfern, Deedy Das, and Derek Xiao. 2025: The state of generative AI in the enterprise. Menlo Ventures, <https://menlovc.com/perspective/2025-the-state-of-generative-ai-in-the-enterprise/>, December 2025. Published 9 December 2025; accessed 15 September 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez,

- Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5998–6008, 2017.
- Ander Artola Velasco, Stratis Tsirtsis, Nastaran Okati, and Manuel Gomez-Rodriguez. Is your LLM overcharging you? tokenization, transparency, and incentives. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. Oral; arXiv:2505.21627.
- vLLM Project. Speculative decoding.
https://docs.vllm.ai/en/stable/features/speculative_decoding/, 2026. vLLM documentation. Accessed: 2026-09-15.
- Ziyao Wang, Guoheng Sun, Yexiao He, Zheyu Shen, Bowei Tian, and Ang Li. Predictive auditing of hidden tokens in LLM APIs via reasoning length estimation, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, et al. LLM inference unveiled: Survey and roofline model insights. arXiv preprint arXiv:2402.16363, 2024.
- Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, et al. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research (BAIR) Blog, 2024. URL
<https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,

- Zi Lin, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.
- Quanyan Zhu. Agentomics: Economic foundations for the valuation, attribution, and pricing of AI agents in Human-AI workflows. *arXiv preprint arXiv:2606.14769*, 2026a. URL <https://arxiv.org/abs/2606.14769>.
- Quanyan Zhu. Foundations of AI tokenomics: The economics of production, consumption, and value in token-based AI systems. Monograph manuscript, New York University, 2026b.
- Quanyan Zhu. Pricing agentic AI: Token-based, value-based, and hybrid contracts. Working paper, 2026c.
- Quanyan Zhu. AI tokenomics: The economics of tokens, computation, and pricing in foundation models. *arXiv preprint arXiv:2606.24616*, 2026d.